



Module Code	:	CT0106-3-2-SNA – System and Network Administration
Intake Code	:	APD2F2509CS(CYB)
Lecturer Name	:	Mr. Shahab Alizadeh and Ms. Nik Sakinah Nik Ab Aziz
Hand in Date	:	22 nd September 2025
Hand out Date	:	1 st December 2025
Group No.	:	Group SNA35
Group Leader	:	AHMED MIRAHUSAIN ALVI

Student ID	Student Name
TP084807	AHMED MIRAHUSAIN ALVI
TP082459	MOHAMED ABDIFATAH ALI
TP085327	SULTAN ABDULLA OMAR TAKRORI
TP085031	OMAR KONDAKJI
TP080822	ABDUL AZIZ VAYANI

TABLE OF CONTENTS

ACKNOWLEDGEMENT	1
CHAPTER 1: INTRODUCTION	2
CHAPTER 2: NETWORK CONFIGURATION	3
2.1 Hostname Configuration on Rocky	3
2.2 Static IP Address on Rocky	5
2.3 Testing Hostname on Rocky	6
2.4 Hostname Configuration on Ubuntu	7
2.5 Verifying Connectivity between Rocky and Ubuntu	8
2.6 Summary	8
CHAPTER 3: INSTALLING & CONFIGURING A DNS SERVER	9
3.1 Introduction to DNS	9
3.2 Introduction to BIND9	10
3.3 DNS Configuration on Rocky Server 1. Verify OS and Hostname	11
3.4 DNS Configuration on Ubuntu Client.....	26
3.5 Testing the DNS Service	28
3.5.1 Nslookup Method	28
3.5.2 Ping Method	29
3.5.3 Dig Method.....	30
CHAPTER 4: INSTALLING & CONFIGURING AN EMAIL SERVER	33
4.1 An Overview	33
4.2 What is a Mail Server?	33
4.3 SMTP, IMAP and POP3 Explained	34
4.3 System Setup and Configuration.....	38

4.4 Introduction to Mail Server	39
4.5 Introduction to Postfix.....	39
4.6 Introduction to Dovecot	40
4.7 Introduction to OpenSSL	40
4.8 Installation of Postfix, Dovecot and OpenSSL	41
4.9 SSL/TLS Certificate Configuration	44
4.10 Introduction to Thunderbird	46
4.11 Setting up Thunderbird.....	47
4.12 Verification and Certificate Trust	51
4.13 Finalizing the Setup.....	52
4.14 Local Mail Delivery Testing	52
CHAPTER 5: INSTALLING & CONFIGURING A WEB SERVER USING APACHE	55
5.1 Introduction to Apache.....	55
5.2 Switching to Root in Rocky	56
5.3 Creating Users on Rocky Server	56
5.4 Setting Hostnames in /etc/hosts directory	57
5.5 Installing Apache on Rocky	58
5.6 Allowing Web Traffic Through Firewall	60
5.9 Setting Apache Server Name	62
5.10 Installing Curl on Rocky and Ubuntu	64
5.11 Testing HTTP Connection	64
5.11.1 Using Curl on Rocky Terminal	64
5.11.2 Using the Mozilla Firefox Browser	65
5.12 Adding SSL / TLS on Rocky	65
5.13 Edit SSL Config to use it	68

5.14 Testing HTTPS from Ubuntu.....	70
5.15 Personal Reflection	73
CHAPTER 6: ADDITIONAL SERVICE CONFIGURATION.....	74
6.1 Samba File Server	74
6.1.1 Introduction to Samba	74
6.1.2 Installation of Samba.....	74
6.1.4 Configuring Samba Shared Directory	76
6.1.5 Restarting Samba File services.....	77
6.1.6 Allowing Samba through Firewall	78
6.1.7 Connecting to Samba share	79
6.1.8 Uploading a file from Ubuntu Client.....	81
6.1.9 Downloading a file from Samba.....	82
6.1.10 Personal Reflection.....	83
6.2 MariaDB.....	84
6.2.1 Introduction to MariaDB	84
6.2.2 MariaDB Installation	85
6.2.3 Adding support for all Unicode Characters.....	86
6.2.4 Enabling MariaDB permanently.....	87
6.2.5 Configuring Firewall to whitelist MariaDB	87
6.2.6 Securing MariaDB Installation	88
6.2.6 Verifying MariaDB Service.....	90
6.2.7 Verifying Port Connections	91
6.2.8 Configuring MariaDB to listen on all interfaces	92
6.2.9 Whitelisting Port 3306.....	93
6.2.10 Logging into MariaDB	94

6.2.11 Creating and using the database	95
6.2.12 Verifying Database Creation	96
6.2.13 Creating and Inserting Data into Table.....	97
6.2.14 Verifying Table Creation.....	98
6.2.15 Installing MariaDB Client on Ubuntu	99
6.2.16 Remote Connection to MariaDB from Ubuntu Client.....	100
6.2.17 Verifying Databases from the Ubuntu Client	101
6.2.18 Querying Table Data from Ubuntu Client.....	102
6.2.19 Web-based database output (PHP Integration)	103
6.2.20 Personal Reflection.....	105
6.3 Advanced Apache Server Features	106
6.3.1 Virtual Hosting (Hosting Custom Websites)	106
6.3.2 Force HTTPS (Redirect HTTP → HTTPS)	110
6.3.3 Securing the website using Basic Authentication.....	113
6.3.4 Custom 404 Error Page.....	117
6.4 Personal Reflection	122
CHAPTER 7: TROUBLESHOOTING	125
7.1 Stuck on a Grey Screen	125
7.2 Unable to install dhcp-server.....	136
7.3 Mail Server Troubleshooting	148
7.3.1 Failed Mail Delivery.....	148
7.3.2 Thunderbird Self-signed Certificate Warning	149
7.4 MariaDB Troubleshooting	153
CONCLUSION.....	154
REFERENCES	155

APPENDIX A: GROUP PHOTO.....	156
A.1 Group Photo 1	156
A.2 Group Photo 2	157
A.3 Group Photo 3	158
APPENDIX B: WORKLOAD MATRIX.....	159

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to **Mr. Shahab Alizadeh and Ms. Nik Sakinah Nik Ab Aziz** for their valuable guidance, support, and encouragement throughout the preparation and completion of our **System and Network Administration (SNA)** assignment. Their insights, clear explanations, and constructive feedback greatly helped us in understanding the core concepts and applying them effectively in our work. This assignment not only enhanced our technical knowledge but also improved our analytical and problem-solving skills. We are sincerely grateful for the opportunity to learn under their supervision and for the continuous support they have shown us. We also express our deep gratitude to our colleagues and peers, whose collaboration and exchange of ideas have greatly enriched our understanding of the subject. A heartfelt thank you to our friends: Sultan Abdulla Omar Takrori, Mohamed Abdifatah Ali, Ahmed Mirahusain Alvi, Abdul Aziz Vayani and Omar Kondakji for their continuous support, motivation, and cooperation throughout this journey.

CHAPTER 1: INTRODUCTION

Building a working server–client environment from scratch is not just about running commands; it is about understanding how different network services connect and support each other. For this assignment, our group was responsible for creating a realistic enterprise-style network using Rocky Linux as the main server and Ubuntu Linux as the client. The goal was to design a fully functional system where multiple services operate together smoothly, communicate properly, and follow secure practices that match real-world standards.

We also established various core services that are fundamental in any organization during the course of the project. The purpose of BIND9 was to manage DNS resolution of internal and external networks, DHCP was used to automatically assign IP addresses, Postfix and dovecot served to create the secure email system and Apache was installed with HTTPS so that people could access the internet through encrypted web-services. As an improvement, we have also implemented Samba File Server and MariaDB so that we could share files centrally and manage databases, which is an addition to the capability of the system and makes it rather simulate the real company network.

This report explains how each service was set up, why each configuration was chosen, and how everything was tested inside a shared NAT network on VirtualBox. It also highlights the issues we faced, the troubleshooting steps we took, and how the final result reflects real Linux server administration and teamwork.

CHAPTER 2: NETWORK CONFIGURATION

This chapter describes the network configuration implemented on the Rocky Linux Server and Ubuntu Client systems used in this project. The purpose of the configuration is to ensure that both machines are operating under the same network environment, using static IP addressing and proper hostname identification to enable smooth communication.

The domain name selected for this setup is **distinction.club**. Therefore, both systems were assigned Fully Qualified Domain Names (FQDN) within this domain to support hostname.

2.1 Hostname Configuration on Rocky

First, we check the current hostname of the Rocky Linux server using the `hostname` command. In our setup, **we want the hostname to be in the format of a Fully Qualified Domain Name (FQDN)**, so it can be recognized properly on the network. If the hostname is not already set to the FQDN, we update it in the hostname configuration files. The FQDN **server.distinction.club** allows the server to be uniquely identified by other machines on the same network.

Figure 1 shows the Static Hostname of our Rocky Linux Server

A terminal window with a yellow background and a dark purple title bar. The title bar contains the text 'ahmedsna@server:~' and standard window controls. The terminal shows a command prompt 'ahmedsna@server:~\$' followed by the command 'hostname'. The output is 'server.distinction.club', which is highlighted in green. A red arrow points to the output text. The prompt 'ahmedsna@server:~\$' appears again on the next line.

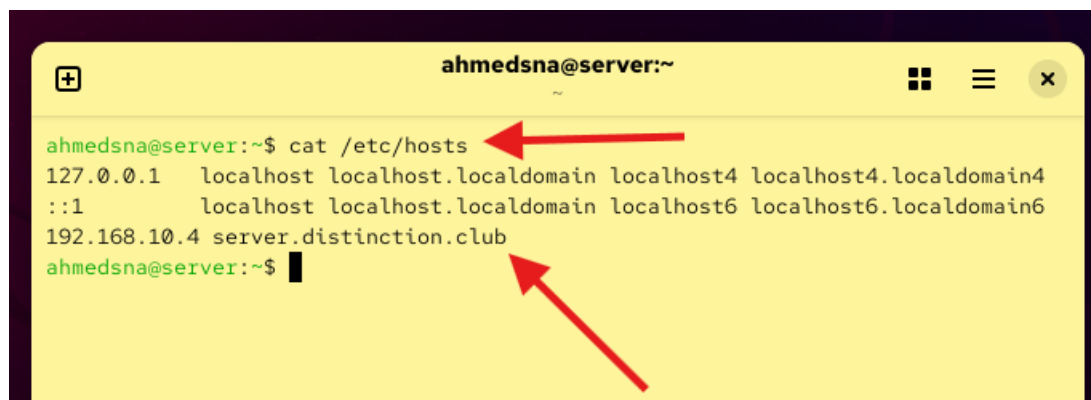
```
ahmedsna@server:~$ hostname
server.distinction.club
ahmedsna@server:~$
```

This output confirms that the Rocky server hostname is now correctly set to `server.distinction.club`.

Figure 2 shows the content of hostname fileA terminal window titled 'ahmedsna@server:~' with a yellow background. The command 'cat /etc/hostname' has been executed, and the output 'server.distinction.club' is displayed. A red rectangular box highlights the command and its output.

```
ahmedsna@server:~$ cat /etc/hostname
server.distinction.club
```

The /etc/hostname file contains the persistent hostname. The correct FQDN stored here ensures the hostname remains the same after reboot.

Figure 3 checks hosts configuration fileA terminal window titled 'ahmedsna@server:~' with a yellow background. The command 'cat /etc/hosts' has been executed, showing the contents of the file. Two red arrows point to specific lines: one points to the first line '127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4' and the other points to the third line '192.168.10.4 server.distinction.club'.

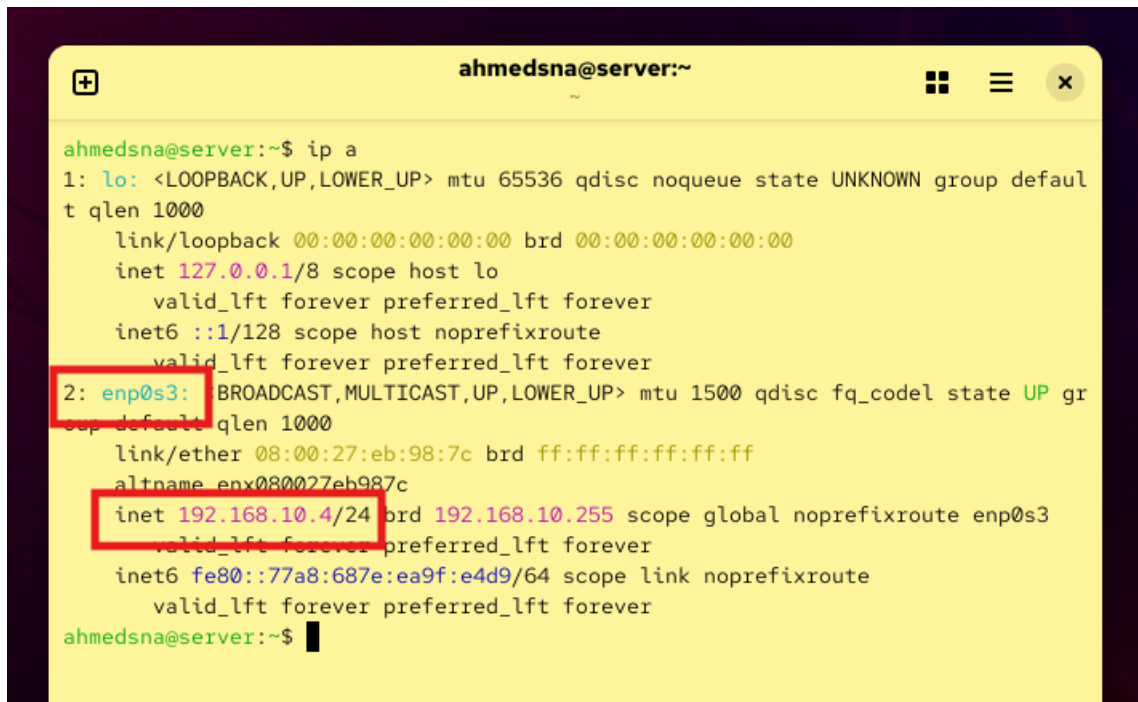
```
ahmedsna@server:~$ cat /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.10.4 server.distinction.club
ahmedsna@server:~$
```

The /etc/hosts file maps the hostname to the static IP 192.168.10.4 , enabling proper local name resolution and system identification.

2.2 Static IP Address on Rocky

The Rocky Server must have a static IP address so that other machines can consistently reach it. We verify the interface information and **ensure the assigned IP is correctly configured on the active network interface**. This makes the server reliable for all network-based operations.

Figure 4 checks IP address of Rocky



```

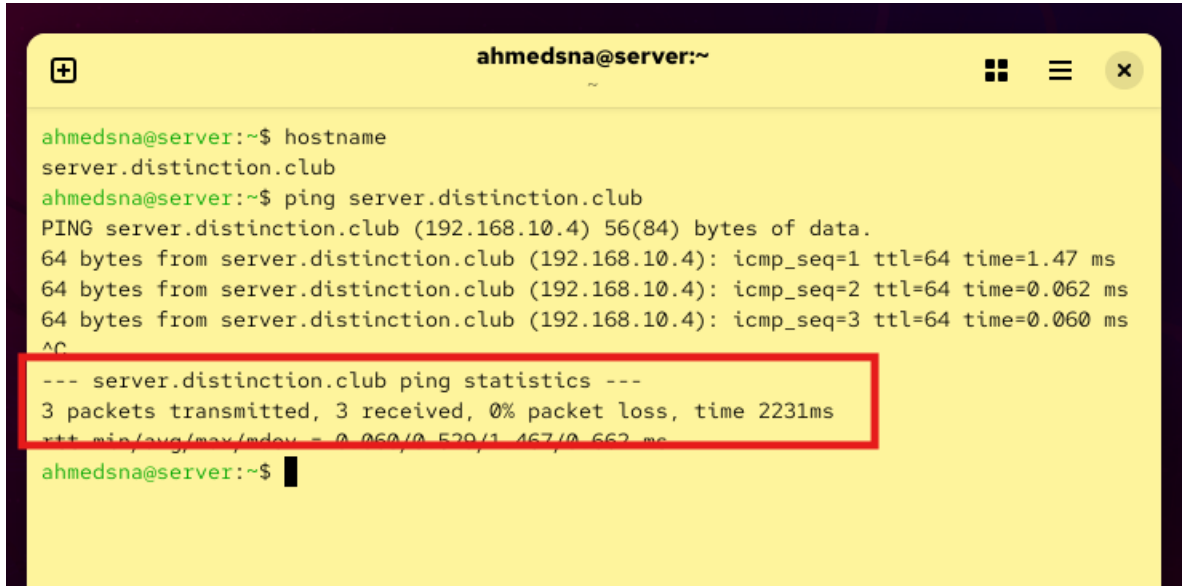
ahmedsna@server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:eb:98:7c brd ff:ff:ff:ff:ff:ff
    altname enx080027eb987c
    inet 192.168.10.4/24 brd 192.168.10.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::77a8:687e:ea9f:e4d9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
ahmedsna@server:~$
  
```

The output shows that the interface enp0s3 has been successfully assigned the static IP 192.168.10.4/24, confirming the network configuration is active

2.3 Testing Hostname on Rocky

To check if the hostname is functioning properly, we can ping it using the domain name.

Figure 5 checks Domain Name Connectivity



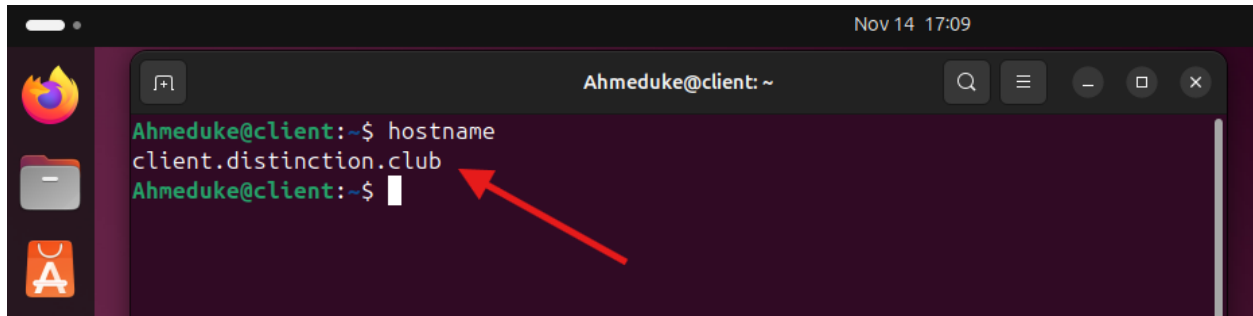
```
ahmedsna@server:~$ hostname
server.distinction.club
ahmedsna@server:~$ ping server.distinction.club
PING server.distinction.club (192.168.10.4) 56(84) bytes of data:
64 bytes from server.distinction.club (192.168.10.4): icmp_seq=1 ttl=64 time=1.47 ms
64 bytes from server.distinction.club (192.168.10.4): icmp_seq=2 ttl=64 time=0.062 ms
64 bytes from server.distinction.club (192.168.10.4): icmp_seq=3 ttl=64 time=0.060 ms
^C
--- server.distinction.club ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2231ms
rtt min/avg/max/mdev = 0.060/0.530/1.467/0.662 ms
ahmedsna@server:~$
```

The results confirm that the hostname is responding without issues.

2.4 Hostname Configuration on Ubuntu

Assigning a proper FQDN to the client allows it to be uniquely identified when communicating with the server and other devices.

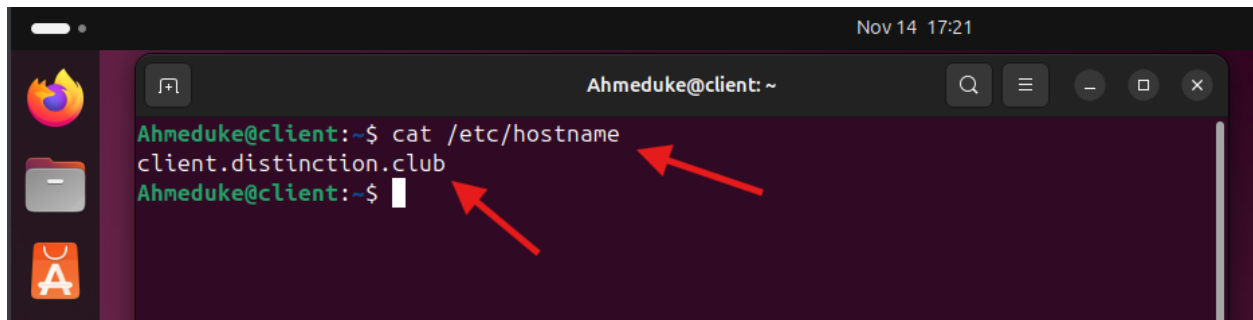
Figure 6 checks hostname of Ubuntu Client

A terminal window titled 'Ahmeduke@client: ~' with a search bar and window controls. The prompt is 'Ahmeduke@client:~\$'. The command 'hostname' has been entered, and the output 'client.distinction.club' is displayed on the next line. A red arrow points from the output text to the terminal window.

```
Ahmeduke@client:~$ hostname
client.distinction.club
Ahmeduke@client:~$
```

The hostname is successfully configured as **client.distinction.club**, confirming correct identification on the network.

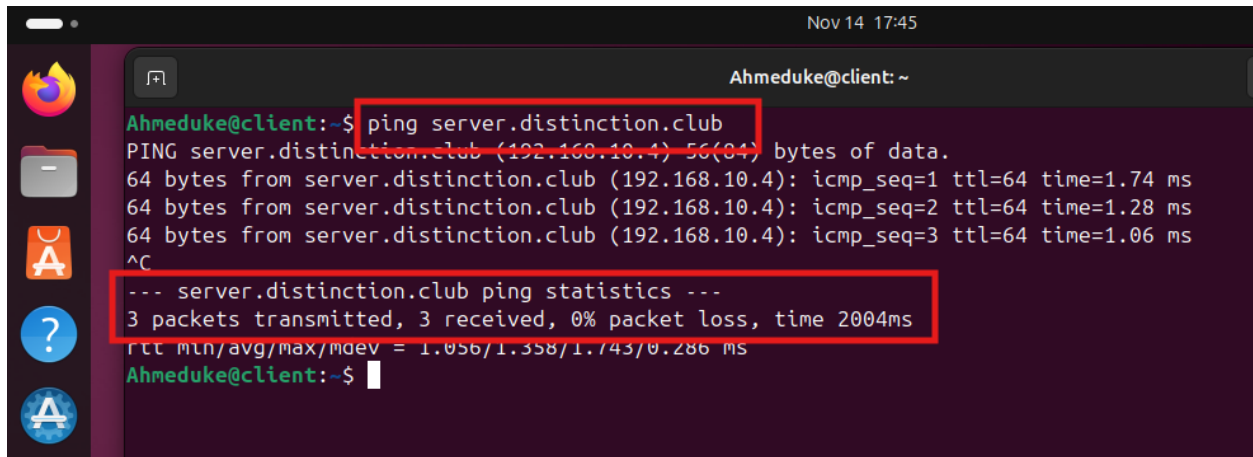
Figure 7 checks content of hostname file on ubuntu

A terminal window titled 'Ahmeduke@client: ~' with a search bar and window controls. The prompt is 'Ahmeduke@client:~\$'. The command 'cat /etc/hostname' has been entered, and the output 'client.distinction.club' is displayed on the next line. Two red arrows point from the output text to the terminal window.

```
Ahmeduke@client:~$ cat /etc/hostname
client.distinction.club
Ahmeduke@client:~$
```

The `/etc/hostname` file shows the permanent hostname stored on the system, which loads during boot.

2.5 Verifying Connectivity between Rocky and Ubuntu



A terminal window titled 'Ahmeduke@client: ~' showing a successful ping command. The command 'ping server.distinction.club' is entered and executed. The output shows three successful ping responses from 192.168.10.4 with times of 1.74 ms, 1.28 ms, and 1.06 ms. A summary line indicates 3 packets transmitted, 3 received, 0% packet loss, and a time of 2004ms. The statistics line shows rtt min/avg/max/mdev = 1.056/1.358/1.743/0.286 ms. The terminal window has a dark purple background and a sidebar with application icons on the left.

```
Ahmeduke@client:~$ ping server.distinction.club
PING server.distinction.club (192.168.10.4) 56(84) bytes of data:
64 bytes from server.distinction.club (192.168.10.4): icmp_seq=1 ttl=64 time=1.74 ms
64 bytes from server.distinction.club (192.168.10.4): icmp_seq=2 ttl=64 time=1.28 ms
64 bytes from server.distinction.club (192.168.10.4): icmp_seq=3 ttl=64 time=1.06 ms
^C
--- server.distinction.club ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.056/1.358/1.743/0.286 ms
Ahmeduke@client:~$
```

The successful ping results confirm that the Ubuntu Client can reach the Rocky Server, verifying network communication between both systems.

2.6 Summary

Both the Rocky Linux Server and the Ubuntu Client were configured with static IP addresses and correct FQDN hostnames within the **distinction.club** domain. The successful ping tests between the two systems confirm that the network configuration is functioning properly, allowing stable and consistent communication across the environment.

CHAPTER 3: INSTALLING & CONFIGURING A DNS SERVER

3.1 Introduction to DNS



The Domain Name server (DNS) is a core network service that translates human readable domain names to machine readable IP address, instead of us remembering the numerical IPs, This process allows devices on networks to locate and communicate with each other efficiently and improving usability, for our case instead of remembering the numeric IP address 192.168.200.4, users and systems can refer to host using simple names such as server.distinction.club.

3.2 Introduction to BIND9



BIND9 (Berkeley Internet Name Domain version 9) is the most used DNS server software on Linux systems, It provides full authoritative DNS functionality which means it can store and answer DNS records for an organization's own domain, The BIND9 uses configuration files like named.conf and zone files to define the forward lookup zones hostnames to IP and reverse the lookup zones IP to hostnames, In our case BIND9 is used on Rocky Linux server to manage the name resolution for the distinction.club domain, Which allows both the server and the client to resolve each other's hostname reliably.

3.3 DNS Configuration on Rocky Server

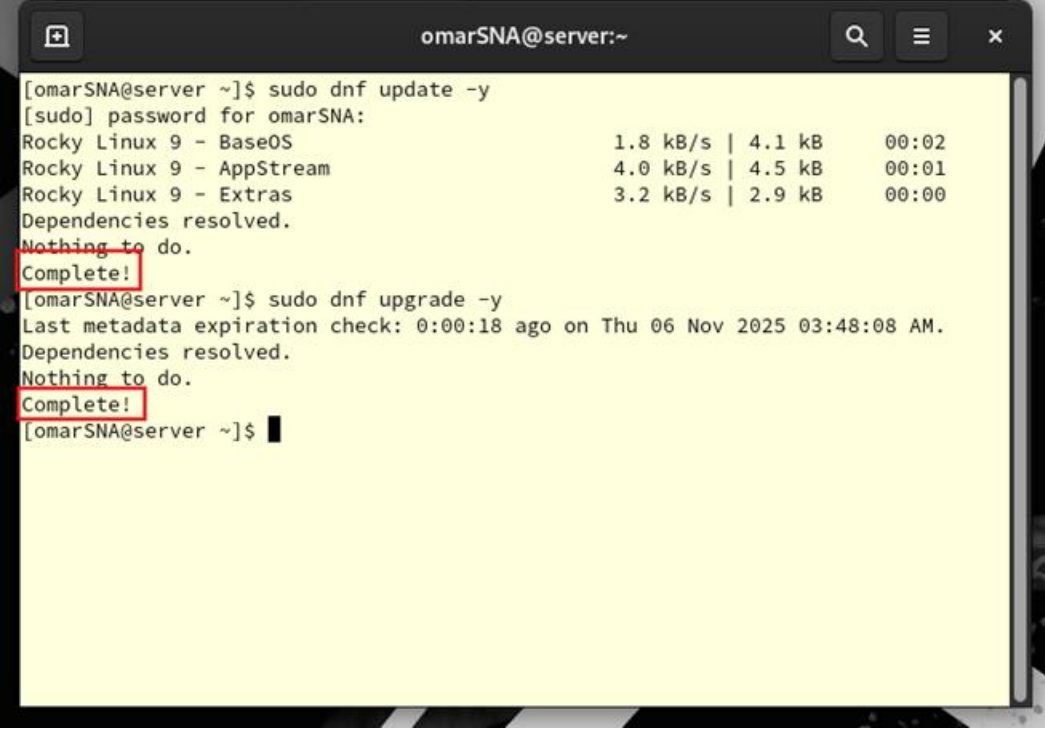
1. Verify OS and Hostname

A terminal window titled 'omarSNA@server:~' with search, menu, and close icons. The command 'cat /etc/os-release' is executed, displaying system information for Rocky Linux 9.6 (Blue Onyx). The output includes fields like NAME, VERSION, ID, ID_LIKE, VERSION_ID, PLATFORM_ID, PRETTY_NAME, ANSI_COLOR, LOGO, CPE_NAME, HOME_URL, VENDOR_NAME, VENDOR_URL, BUG_REPORT_URL, SUPPORT_END, ROCKY_SUPPORT_PRODUCT, ROCKY_SUPPORT_PRODUCT_VERSION, REDHAT_SUPPORT_PRODUCT, and REDHAT_SUPPORT_PRODUCT_VERSION. The command 'hostname' is then executed, showing the system's hostname as 'server.distinction.club'.

```
[omarSNA@server ~]$ cat /etc/os-release
NAME="Rocky Linux"
VERSION="9.6 (Blue Onyx)"
ID="rocky"
ID_LIKE="rhel centos fedora"
VERSION_ID="9.6"
PLATFORM_ID="platform:el9"
PRETTY_NAME="Rocky Linux 9.6 (Blue Onyx)"
ANSI_COLOR="0;32"
LOGO="fedora-logo-icon"
CPE_NAME="cpe:/o:rocky:rocky:9::baseos"
HOME_URL="https://rockylinux.org/"
VENDOR_NAME="RESF"
VENDOR_URL="https://resf.org/"
BUG_REPORT_URL="https://bugs.rockylinux.org/"
SUPPORT_END="2032-05-31"
ROCKY_SUPPORT_PRODUCT="Rocky-Linux-9"
ROCKY_SUPPORT_PRODUCT_VERSION="9.6"
REDHAT_SUPPORT_PRODUCT="Rocky Linux"
REDHAT_SUPPORT_PRODUCT_VERSION="9.6"
[omarSNA@server ~]$ hostname
server.distinction.club
[omarSNA@server ~]$
```

We first check that my Rocky Linux server was installed correctly and that the hostname matched my assigned group FQDN. This step ensures that the DNS configuration will reflect the correct domain name, The output showed NAME=Rocky Linux and my hostname server.distinction.club, confirming that the system setup is consistent.

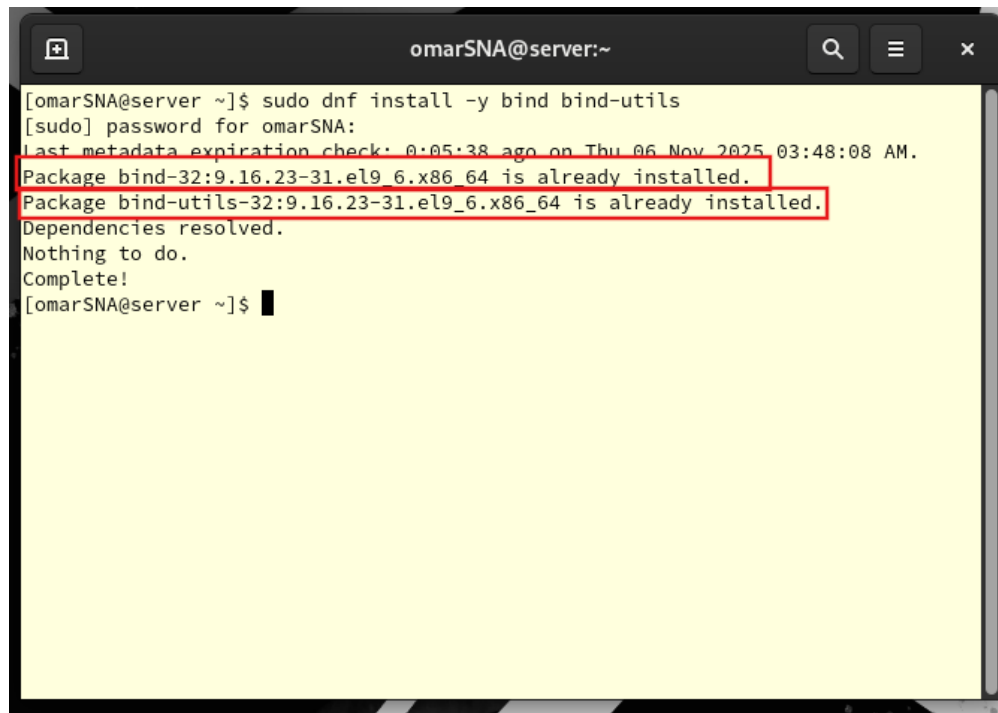
2. Update and Upgrade System Packages

A terminal window titled 'omarSNA@server:~' with search, menu, and close icons in the title bar. The terminal output shows the execution of 'sudo dnf update -y' and 'sudo dnf upgrade -y'. The first command lists updates for Rocky Linux 9 packages (BaseOS, AppStream, Extras) with their respective sizes and download times. Both commands result in 'Nothing to do.' and 'Complete!' messages, which are highlighted with red boxes. The prompt returns to the user after each command.

```
[omarSNA@server ~]$ sudo dnf update -y
[sudo] password for omarSNA:
Rocky Linux 9 - BaseOS                1.8 kB/s | 4.1 kB      00:02
Rocky Linux 9 - AppStream             4.0 kB/s | 4.5 kB      00:01
Rocky Linux 9 - Extras                3.2 kB/s | 2.9 kB      00:00
Dependencies resolved.
Nothing to do.
Complete!
[omarSNA@server ~]$ sudo dnf upgrade -y
Last metadata expiration check: 0:00:18 ago on Thu 06 Nov 2025 03:48:08 AM.
Dependencies resolved.
Nothing to do.
Complete!
[omarSNA@server ~]$
```

Before installing BIND9, we update and upgrade our Rocky server to ensure all system packages and dependencies are current. Keeping the system updated helps prevent compatibility issues when setting up network services like DNS.

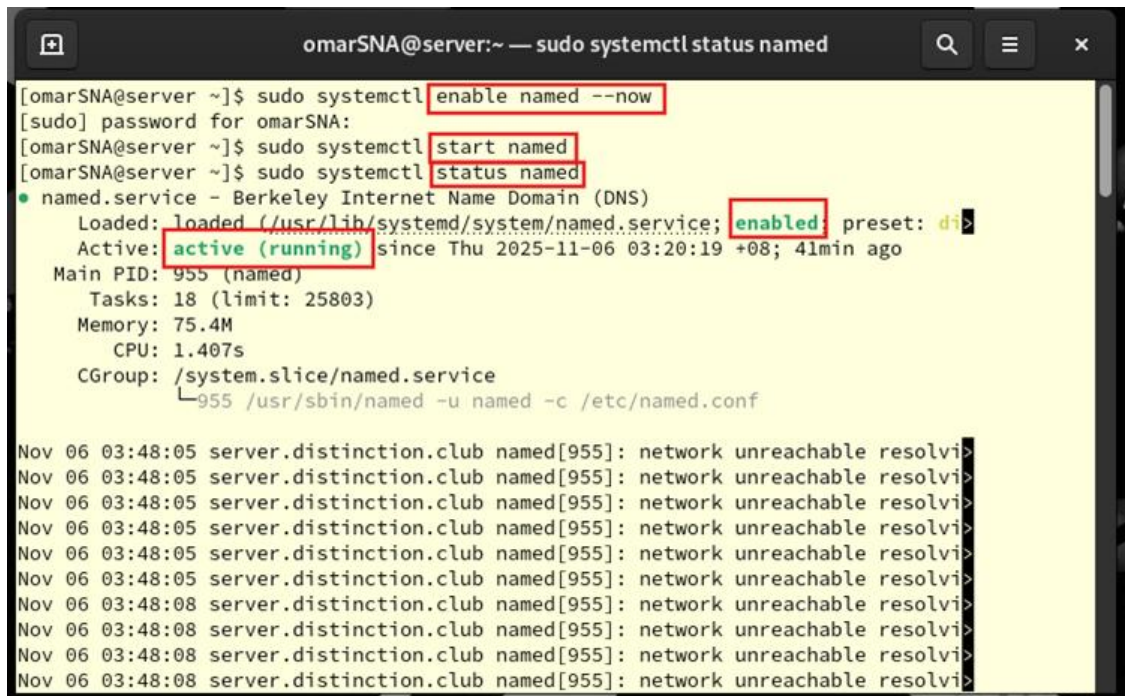
3. Install BIND9



```
omarSNA@server:~  
[omarSNA@server ~]$ sudo dnf install -y bind bind-utils  
[sudo] password for omarSNA:  
Last metadata expiration check: 0:05:38 ago on Thu 06 Nov 2025 03:48:08 AM.  
Package bind-32:9.16.23-31.el9_6.x86_64 is already installed.  
Package bind-utils-32:9.16.23-31.el9_6.x86_64 is already installed.  
Dependencies resolved.  
Nothing to do.  
Complete!  
[omarSNA@server ~]$
```

Then we install the BIND9 package, which provides the DNS service (named) along with useful tools like dig and nslookup for testing, which we use later. This software allows our Rocky server to resolve and manage domain names for our internal networks.

4. Enable and Start the DNS Service

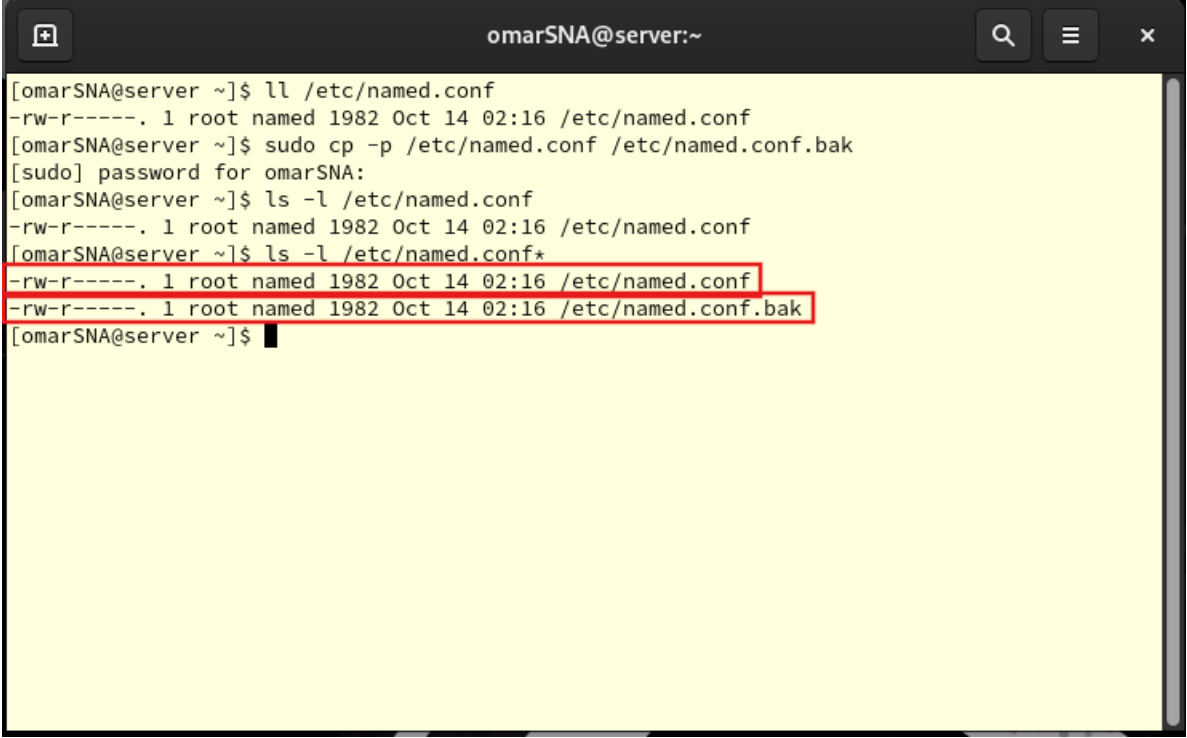


```
omarSNA@server:~ — sudo systemctl status named
[omarSNA@server ~]$ sudo systemctl enable named --now
[sudo] password for omarSNA:
[omarSNA@server ~]$ sudo systemctl start named
[omarSNA@server ~]$ sudo systemctl status named
● named.service - Berkeley Internet Name Domain (DNS)
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: disabled)
   Active: active (running) since Thu 2025-11-06 03:20:19 +08; 41min ago
     Main PID: 955 (named)
       Tasks: 18 (limit: 25803)
        Memory: 75.4M
          CPU: 1.407s
      CGroup: /system.slice/named.service
              └─955 /usr/sbin/named -u named -c /etc/named.conf

Nov 06 03:48:05 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:05 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:05 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:05 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:05 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:05 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:08 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:08 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:08 server.distinction.club named[955]: network unreachable resolv>
Nov 06 03:48:08 server.distinction.club named[955]: network unreachable resolv>
```

After the installation process, we enable and start the named DNS service, which is the main DNS process. Enabling it ensures the service starts automatically every time the server boots. We confirm that the service is active and running by checking the status of named.

5. Backup the Default Configuration File

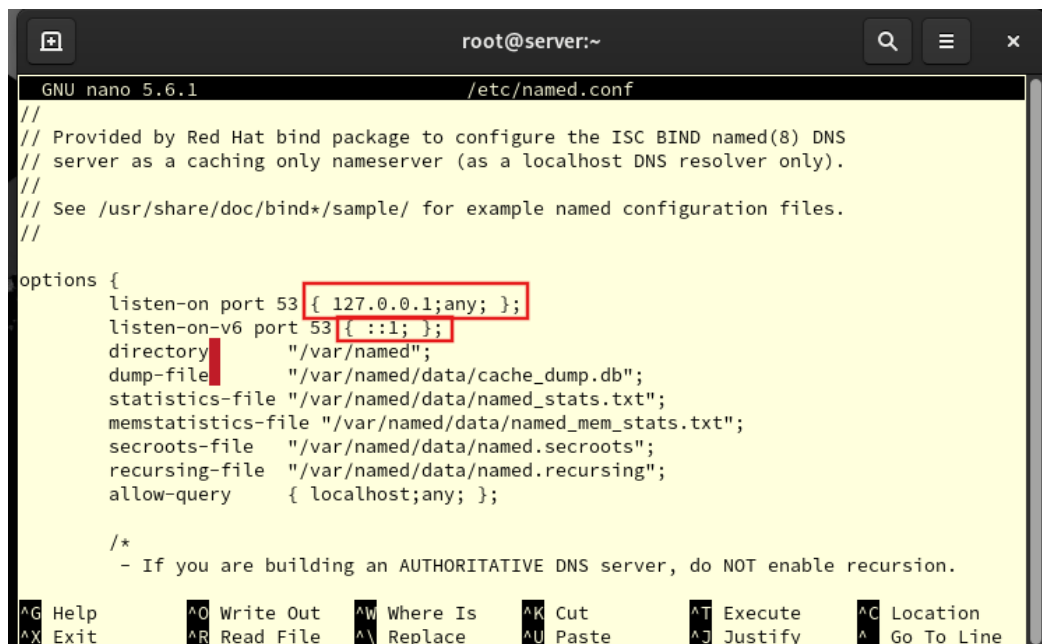
A terminal window titled 'omarSNA@server:~' with search, menu, and close icons in the top right. The terminal shows the following commands and output:

```
[omarSNA@server ~]$ ll /etc/named.conf
-rw-r-----. 1 root named 1982 Oct 14 02:16 /etc/named.conf
[omarSNA@server ~]$ sudo cp -p /etc/named.conf /etc/named.conf.bak
[sudo] password for omarSNA:
[omarSNA@server ~]$ ls -l /etc/named.conf
-rw-r-----. 1 root named 1982 Oct 14 02:16 /etc/named.conf
[omarSNA@server ~]$ ls -l /etc/named.conf*
-rw-r-----. 1 root named 1982 Oct 14 02:16 /etc/named.conf
-rw-r-----. 1 root named 1982 Oct 14 02:16 /etc/named.conf.bak
[omarSNA@server ~]$
```

The last two lines of the output are highlighted with a red box.

Before making any changes, we create a backup of the main configuration file named `named.conf` to avoid losing the default settings. This backup is important in case we need to restore the original configuration later.

6. Edit named.conf to Allow Queries and Define Zones

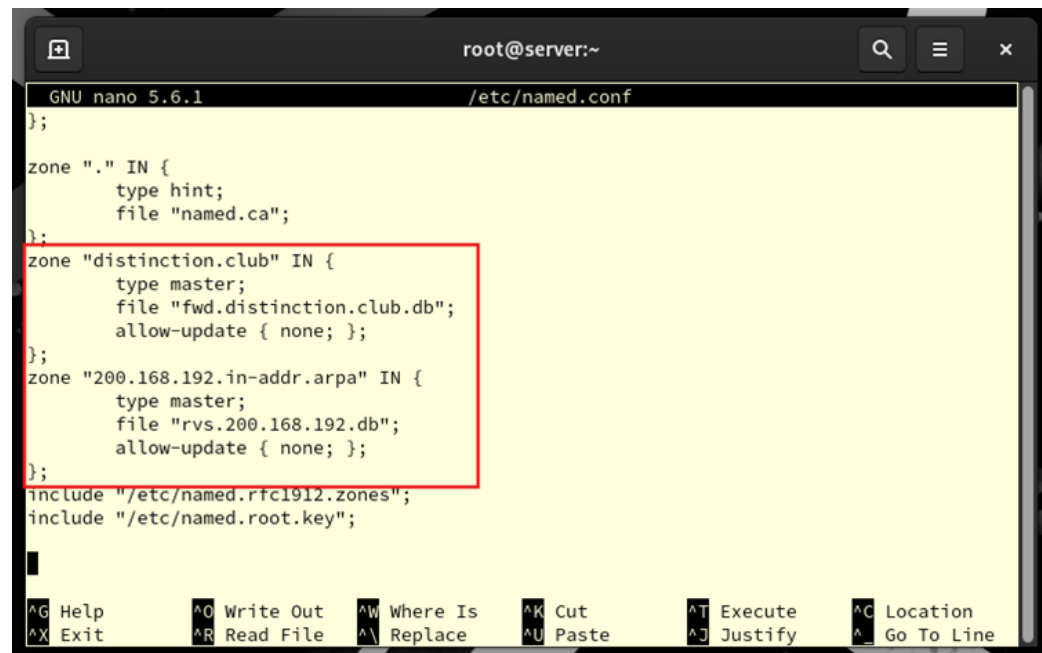


```

GNU nano 5.6.1 /etc/named.conf
//
// Provided by Red Hat bind package to configure the ISC BIND named(8) DNS
// server as a caching only nameserver (as a localhost DNS resolver only).
//
// See /usr/share/doc/bind*/sample/ for example named configuration files.
//
options {
    listen-on port 53 { 127.0.0.1; any; };
    listen-on-v6 port 53 { ::1; };
    directory "/var/named";
    dump-file "/var/named/data/cache_dump.db";
    statistics-file "/var/named/data/named_stats.txt";
    memstatistics-file "/var/named/data/named_mem_stats.txt";
    secroots-file "/var/named/data/named.secroots";
    recursing-file "/var/named/data/named.recursing";
    allow-query { localhost; any; };

    /*
     - If you are building an AUTHORITATIVE DNS server, do NOT enable recursion.

```



```

GNU nano 5.6.1 /etc/named.conf
};

zone "." IN {
    type hint;
    file "named.ca";
};

zone "distinction.club" IN {
    type master;
    file "fwd.distinction.club.db";
    allow-update { none; };
};

zone "200.168.192.in-addr.arpa" IN {
    type master;
    file "rvs.200.168.192.db";
    allow-update { none; };
};

include "/etc/named.rfc1912.zones";
include "/etc/named.root.key";

```

In this step, we enter as root using the command `su -`. We configure the DNS service to listen on all interfaces and to accept queries from any host on the network. We also define the forward and reverse zone files, which handle hostname to IP and IP to hostname resolution for our internal domain `distinction.club`.

7. Create the Forward Lookup Zone File

```

root@server:/var/named
[root@server ~]# cd /var/named
[root@server named]# nano fwd.distinction.club.db
[root@server named]#

```

```

GNU nano 5.6.1 fwd.distinction.club.db
$TTL 86400
@      IN      SOA      server.distinction.club.  root.distinction.club. (
                        2001062501 ; serial
                        21600      ; refresh after 6 hours
                        3600       ; retry after 1 hour
                        604800     ; expire after 1 week
                        86400      ; minimum TTL of 1 day
)

@      IN      NS       server.distinction.club.

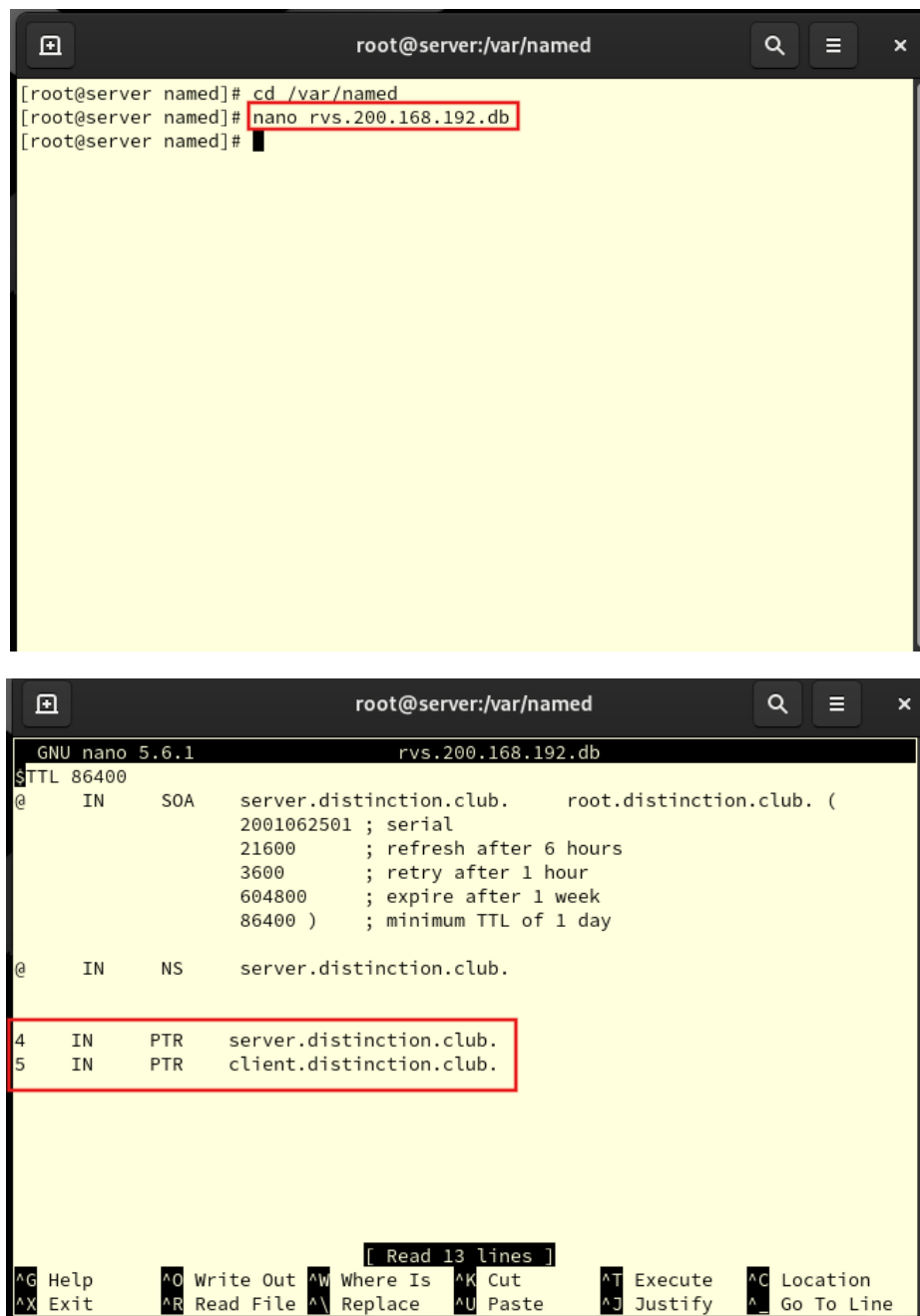
server IN      A        192.168.200.4
client IN     A        192.168.200.5

[ Read 33 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

The forward zone file maps hostnames to their IP addresses. We create the file fwd.distinction.club.db inside the /var/named directory and define server.distinction.club and client.distinction.club with their corresponding IP addresses. This allows other systems on the network to find these hosts by name.

8. Create the Reverse Lookup Zone File



The first screenshot shows a terminal window with the prompt `root@server:/var/named`. The user enters `cd /var/named` and then `nano rvs.200.168.192.db`, which is highlighted with a red box. The second screenshot shows the same terminal window with the nano editor open, editing the file `rvs.200.168.192.db`. The file content is as follows:

```
GNU nano 5.6.1 rvs.200.168.192.db
$TTL 86400
@      IN      SOA      server.distinction.club.  root.distinction.club. (
                                2001062501 ; serial
                                21600      ; refresh after 6 hours
                                3600       ; retry after 1 hour
                                604800    ; expire after 1 week
                                86400 )   ; minimum TTL of 1 day

@      IN      NS       server.distinction.club.

4      IN      PTR      server.distinction.club.
5      IN      PTR      client.distinction.club.
```

The last two lines of the file are highlighted with a red box. The terminal window also shows a status bar at the bottom with various nano editor commands.

In the same directory, we create the reverse zone file `rvs.200.168.192.db`. This reverse zone performs the opposite function, resolving IP addresses back to hostnames. We map .4 and .5 in our subnet to `server.distinction.club` and `client.distinction.club` respectively, completing the bidirectional DNS setup.

9. Validate the Zone Files

```

root@server:/var/named
[omarSNA@server ~]$ su -
Password:
[root@server ~]# cd /var/named
[root@server named]# ls -l
total 24
drwxrwx---. 2 named named  49 Nov  5 05:43 data
drwxrwx---. 2 named named  60 Nov  6 04:31 dynamic
-rw-r--r--. 1 root  root   502 Oct 14 03:09 fwd.distinction.club.db
-rw-r-----. 1 root  named 2112 Jul 30 03:50 named.ca
-rw-r-----. 1 root  named  152 Jul 30 03:50 named.empty
-rw-r-----. 1 root  named  152 Jul 30 03:50 named.localhost
-rw-r-----. 1 root  named  168 Jul 30 03:50 named.loopback
-rw-r--r--. 1 root  root   475 Oct 14 03:25 rvs.200.168.192.db
drwxrwx---. 2 named named   6 Jul 30 03:49 slaves
[root@server named]#

```

```

root@server:/var/named
[omarSNA@server ~]$ su -
Password:
[root@server ~]# cd /var/named
[root@server named]# ls -l
total 24
drwxrwx---. 2 named named  49 Nov  5 05:43 data
drwxrwx---. 2 named named  60 Nov  6 04:31 dynamic
-rw-r--r--. 1 root  root   502 Oct 14 03:09 fwd.distinction.club.db
-rw-r-----. 1 root  named 2112 Jul 30 03:50 named.ca
-rw-r-----. 1 root  named  152 Jul 30 03:50 named.empty
-rw-r-----. 1 root  named  152 Jul 30 03:50 named.localhost
-rw-r-----. 1 root  named  168 Jul 30 03:50 named.loopback
[root@server named]# named-checkzone distinction.club /var/named/fwd.distinction.club.db
zone distinction.club/IN: loaded serial 2001062501
OK
[root@server named]# named-checkzone rvs.200.168.192.db /var/named/rvs.200.168.192.db
zone rvs.200.168.192.db/IN: loaded serial 2001062501
OK
[root@server named]#

```

We verify that both zone files exist and pass the syntax check using named-checkzone. This ensures that no configuration or formatting errors prevent the DNS service from starting and working properly.

10. Restart and Verify DNS Service

```

omarSNA@server:~ — systemctl status named
[omarSNA@server ~]$ systemctl restart named
[omarSNA@server ~]$ systemctl status named
● named.service - Berkeley Internet Name Domain (DNS)
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: disabled)
   Active: active (running) since Thu 2025-11-06 05:20:13 +08; 14s ago
     Process: 3251 ExecStartPre=/bin/bash -c if [ ! "$DISABLE_ZONE_CHECKING" == >
     Process: 3253 ExecStart=/usr/sbin/named -u named -c ${NAMEDCONF} $OPTIONS (>
    Main PID: 3254 (named)
      Tasks: 14 (limit: 25803)
     Memory: 38.5M
        CPU: 128ms
    CGroup: /system.slice/named.service
            └─3254 /usr/sbin/named -u named -c /etc/named.conf

Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: network unreachable resolv>
Nov 06 05:20:13 server.distinction.club named[3254]: managed-keys-zone: Key 203>
Nov 06 05:20:13 server.distinction.club named[3254]: managed-keys-zone: Key 386>

```

After creating and validating the zone files, we restart the DNS service to apply the new configuration. We then check the status of the named DNS service to make sure it is still active and running

11. Allow DNS Through Firewall

A terminal window titled 'omarSNA@server:~' with search, menu, and close icons in the top right. The terminal shows the following commands and output:

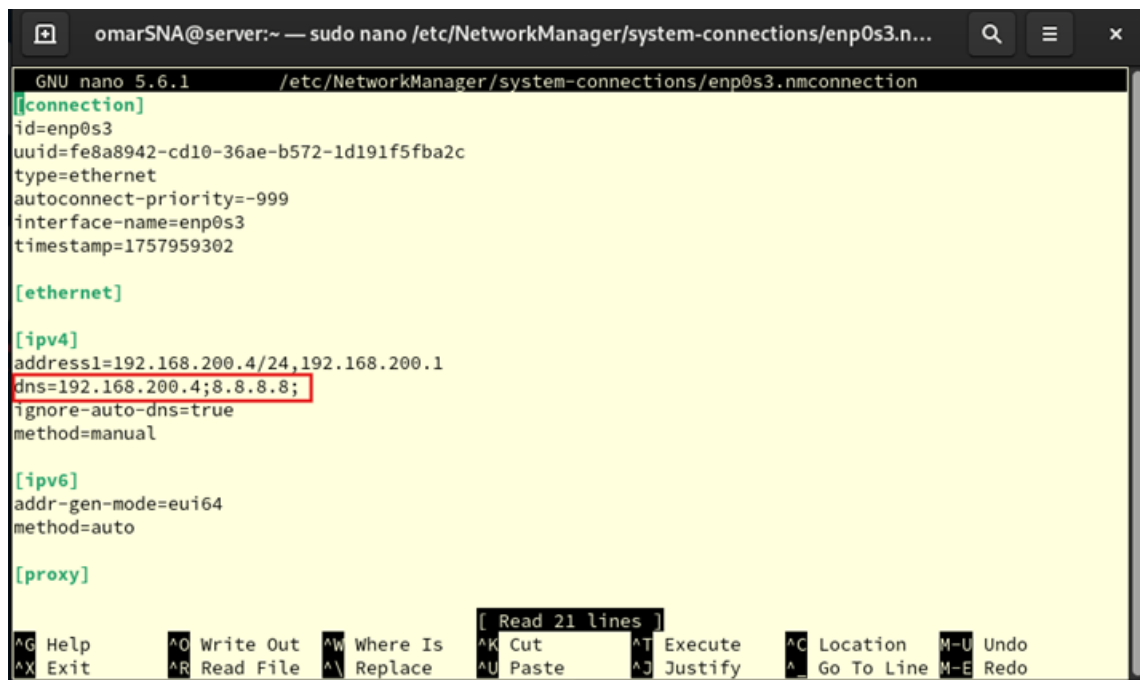
```
[omarSNA@server ~]$ sudo firewall-cmd --permanent --add-port=53/udp
[sudo] password for omarSNA:
Warning: ALREADY_ENABLED: 53:udp
success
[omarSNA@server ~]$ sudo firewall-cmd --permanent --add-port=53/tcp
Warning: ALREADY_ENABLED: 53:tcp
success
[omarSNA@server ~]$ sudo firewall-cmd --reload
success
[omarSNA@server ~]$
```

The words 'success' are highlighted with red boxes in the original image.

Since DNS uses both UDP and TCP on port 53, we configure the firewall to allow incoming traffic for both protocols. Reloading the firewall applies the rules, enabling clients on the network to query the DNS server.

12. Configure NetworkManager DNS Settings

```
omarSNA@server:~  
[omarSNA@server ~]$ sudo nano /etc/NetworkManager/system-connections/enp0s3.nmconnection  
[sudo] password for omarSNA:  
[omarSNA@server ~]$ S
```

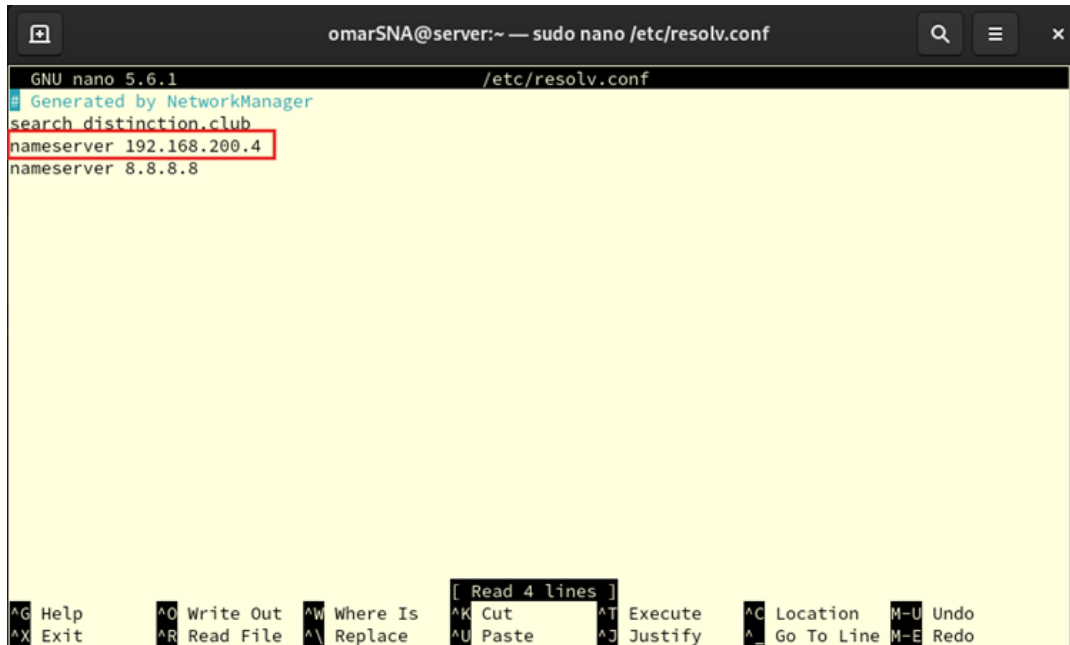


```
GNU nano 5.6.1 /etc/NetworkManager/system-connections/enp0s3.nmconnection  
[connection]  
id=enp0s3  
uuid=fe8a8942-cd10-36ae-b572-1d191f5fba2c  
type=ethernet  
autoconnect-priority=-999  
interface-name=enp0s3  
timestamp=1757959302  
  
[ethernet]  
  
[ipv4]  
address1=192.168.200.4/24,192.168.200.1  
dns=192.168.200.4;8.8.8.8;  
ignore-auto-dns=true  
method=manual  
  
[ipv6]  
addr-gen-mode=eui64  
method=auto  
  
[proxy]  
  
[ Read 21 lines ]  
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

A terminal window titled 'omarSNA@server:~' with search, menu, and close buttons in the title bar. The terminal shows the command 'sudo systemctl restart NetworkManager' being entered. The word 'restart' is highlighted with a red rectangular box. The prompt '[omarSNA@server ~]\$' is visible on two lines.

```
[omarSNA@server ~]$ sudo systemctl restart NetworkManager
[omarSNA@server ~]$
```

In this step, we configure the NetworkManager connections to use our own DNS server (192.168.200.4) as the primary resolver, with Google DNS (8.8.8.8) as a backup. Restarting NetworkManager applies these changes.

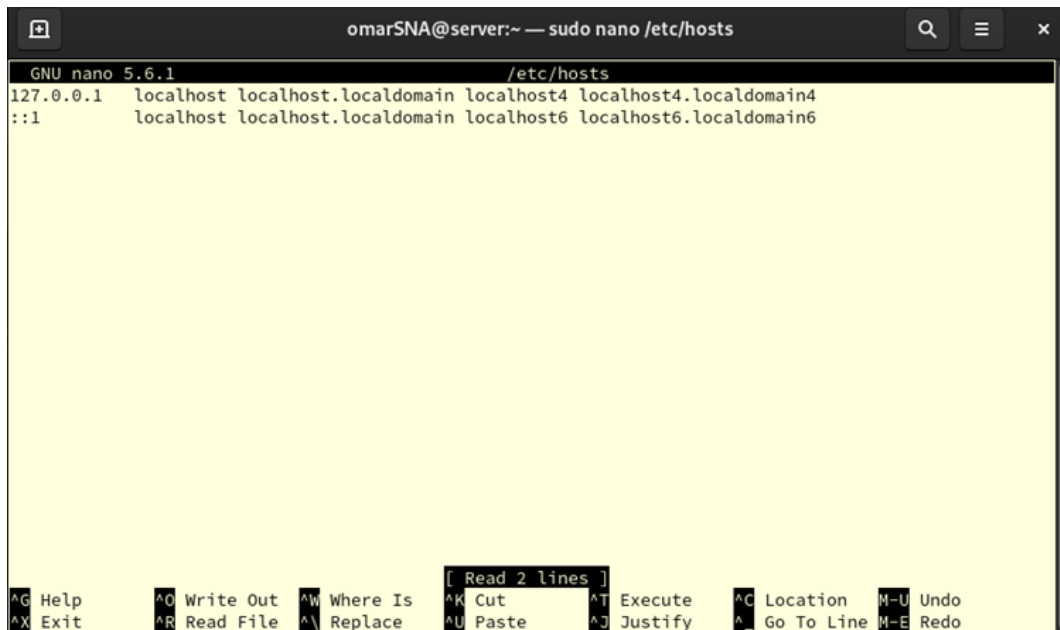
13. Verify /etc/resolv.conf

```
omarSNA@server:~ — sudo nano /etc/resolv.conf
GNU nano 5.6.1 /etc/resolv.conf
# Generated by NetworkManager
search distinction.club
nameserver 192.168.200.4
nameserver 8.8.8.8

[ Read 4 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

The /etc/resolv.conf file is automatically generated and used by the system for name resolution.

We confirm that the listed nameserver is 192.168.200.4, ensuring that **our** Rocky DNS server **is** the one used for all DNS queries.

14. Clean /etc/hosts

The screenshot shows a terminal window titled 'omarSNA@server:~ — sudo nano /etc/hosts'. The window displays the contents of the /etc/hosts file in a yellow background. The file contains two lines of text: '127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4' and '::1 localhost localhost.localdomain localhost6 localhost6.localdomain6'. The nano editor interface is visible, including the top bar with 'GNU nano 5.6.1' and the bottom bar with various keyboard shortcuts like '^G Help', '^O Write Out', '^W Where Is', '^K Cut', '^T Execute', '^C Location', 'M-U Undo', '^X Exit', '^R Read File', '^_ Replace', '^U Paste', '^J Justify', '^_ Go To Line', and 'M-E Redo'. A status bar at the bottom indicates '[Read 2 lines]'.

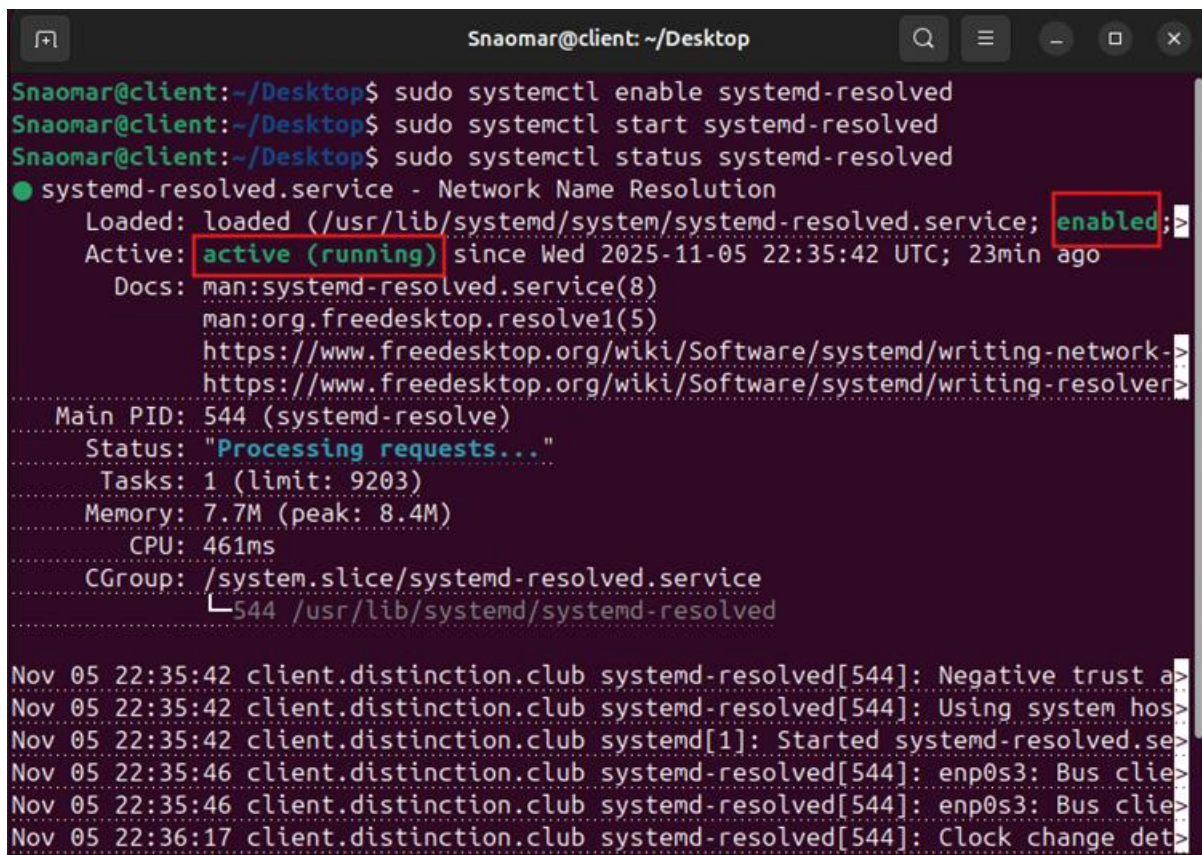
We remove any existing static hostnames for our server and client from /etc/hosts file. This ensures the system resolves hostnames using the DNS server instead of relying on local file entries.

3.4 DNS Configuration on Ubuntu Client

15. Configure Ubuntu Client DNS via GUI

The screenshot shows the 'Wired' network settings window. The 'IPv4' tab is active. In the 'IPv4 Method' section, 'Automatic (DHCP)' is selected. In the 'DNS' section, the 'Automatic' toggle is turned off, and the text input field contains '192.168.200.4, 8.8.8.8', which is highlighted with a red box. Below this, the 'Routes' section has an 'Automatic' toggle turned on. At the bottom, there is a checkbox for 'Use this connection only for resources on its network' which is unchecked.

On the Ubuntu client, we set our Rocky server as the primary DNS server using the GUI network settings. This allows the client to send all DNS queries to 192.168.200.4, ensuring that hostname resolution happens through the DNS service we configured.

16. Enable systemd-resolved on Ubuntu

```
Snaomar@client: ~/Desktop
Snaomar@client:~/Desktop$ sudo systemctl enable systemd-resolved
Snaomar@client:~/Desktop$ sudo systemctl start systemd-resolved
Snaomar@client:~/Desktop$ sudo systemctl status systemd-resolved
● systemd-resolved.service - Network Name Resolution
   Loaded: loaded (/usr/lib/systemd/system/systemd-resolved.service; enabled;
   Active: active (running) since Wed 2025-11-05 22:35:42 UTC; 23min ago
     Docs: man:systemd-resolved.service(8)
           man:org.freedesktop.resolve1(5)
           https://www.freedesktop.org/wiki/Software/systemd/writing-network-
           https://www.freedesktop.org/wiki/Software/systemd/writing-resolver
   Main PID: 544 (systemd-resolve)
     Status: "Processing requests..."
    Tasks: 1 (limit: 9203)
   Memory: 7.7M (peak: 8.4M)
      CPU: 461ms
   CGroup: /system.slice/systemd-resolved.service
           └─544 /usr/lib/systemd/systemd-resolved

Nov 05 22:35:42 client.distinction.club systemd-resolved[544]: Negative trust a
Nov 05 22:35:42 client.distinction.club systemd-resolved[544]: Using system hos
Nov 05 22:35:42 client.distinction.club systemd[1]: Started systemd-resolved.se
Nov 05 22:35:46 client.distinction.club systemd-resolved[544]: enp0s3: Bus clie
Nov 05 22:35:46 client.distinction.club systemd-resolved[544]: enp0s3: Bus clie
Nov 05 22:36:17 client.distinction.club systemd-resolved[544]: Clock change det
```

We enable systemd-resolved on Ubuntu to manage the system's DNS resolution persistently. This ensures that the DNS configuration remains active even after a reboot.

3.5 Testing the DNS Service

3.5.1 Nslookup Method

Rocky:



```
Activities Terminal Nov 17 07:01
omarSNA@server:~
[omarSNA@server ~]$ nslookup server.distinction.club
Server:      192.168.200.4
Address:     192.168.200.4#53

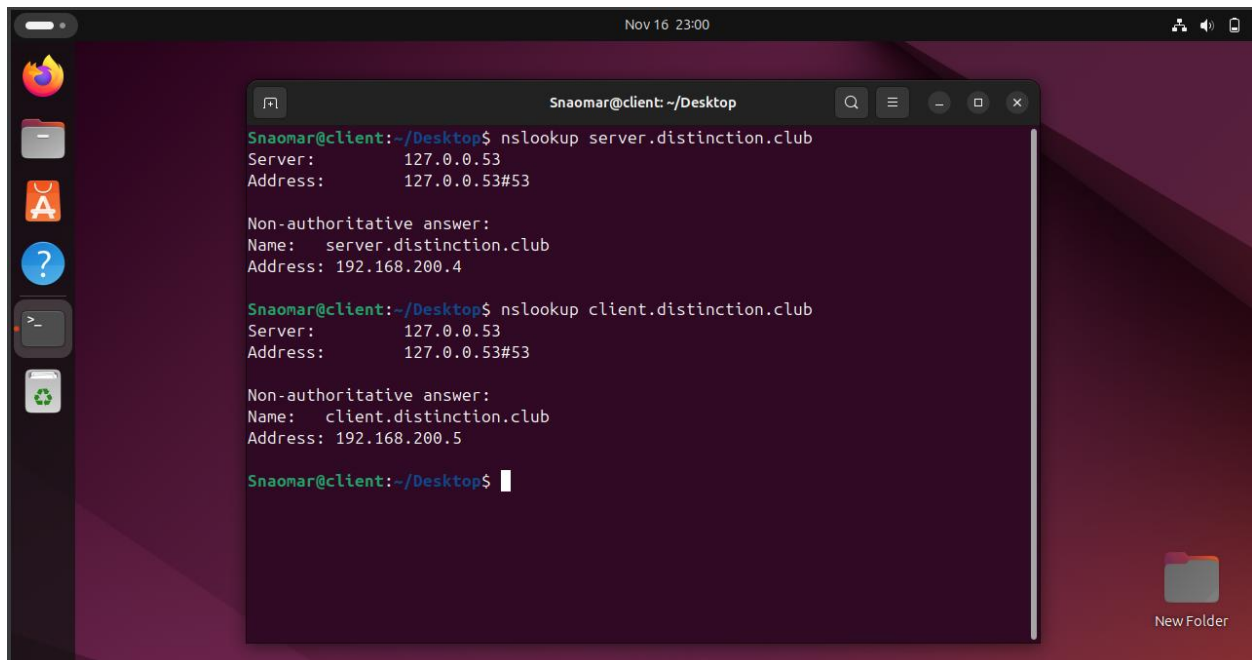
Name:   server.distinction.club
Address: 192.168.200.4

[omarSNA@server ~]$ nslookup client.distinction.club
Server:      192.168.200.4
Address:     192.168.200.4#53

Name:   client.distinction.club
Address: 192.168.200.5

[omarSNA@server ~]$
```

Ubuntu:



```
Nov 16 23:00
Snaomar@client: ~/Desktop
Snaomar@client:~/Desktop$ nslookup server.distinction.club
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   server.distinction.club
Address: 192.168.200.4

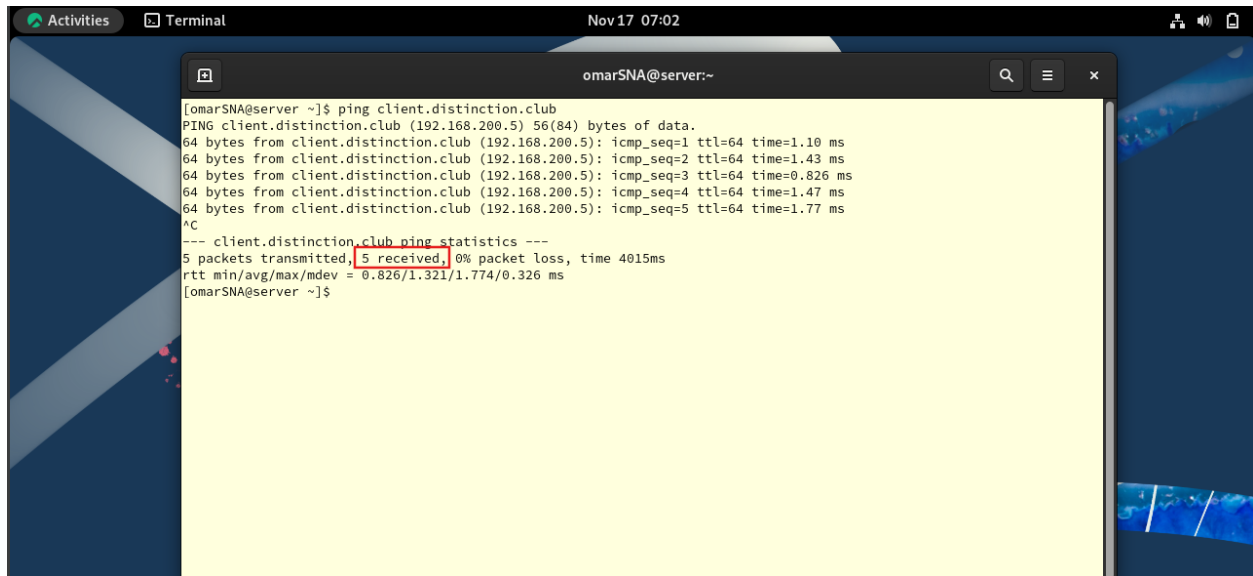
Snaomar@client:~/Desktop$ nslookup client.distinction.club
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   client.distinction.club
Address: 192.168.200.5

Snaomar@client:~/Desktop$
```

3.5.2 Ping Method

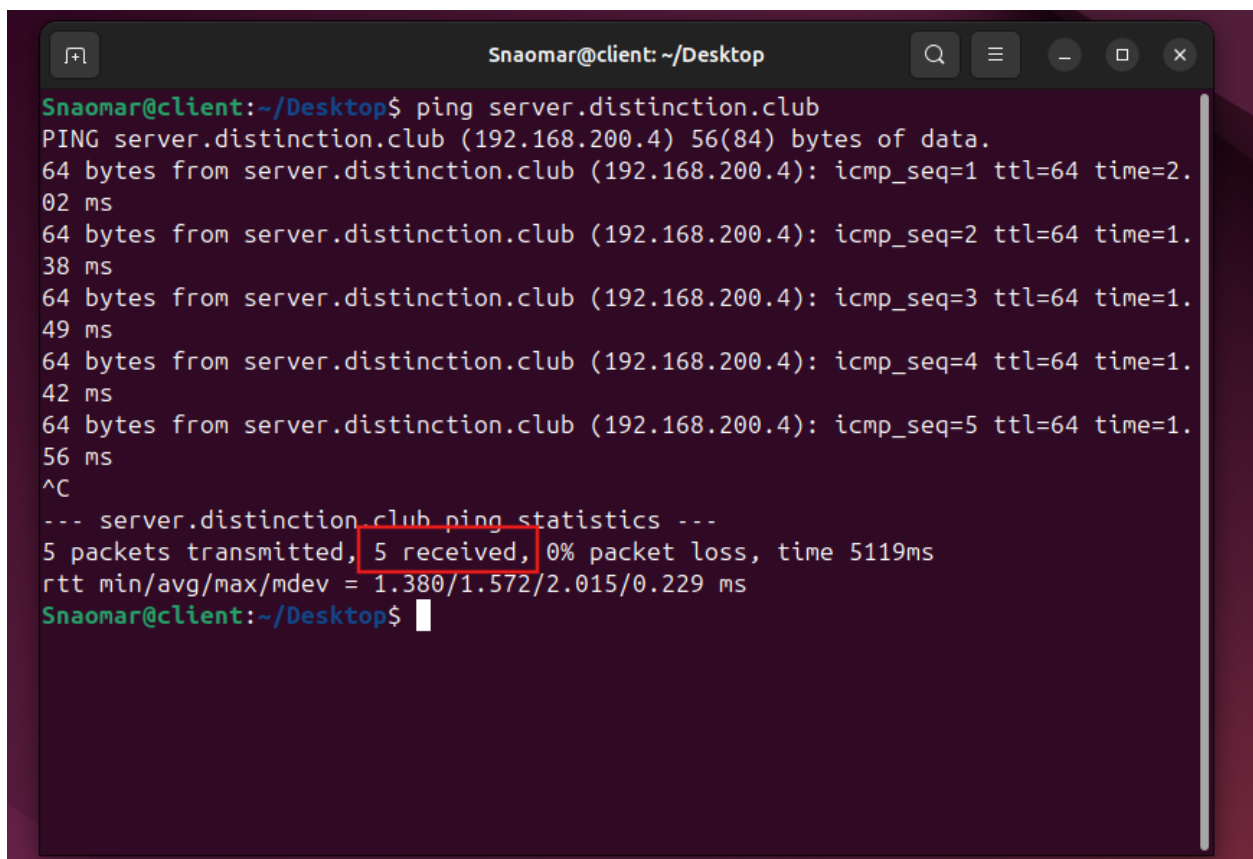
Rocky:



A screenshot of a terminal window on Rocky Linux. The window title is "omarsNA@server:~". The terminal shows the command `ping client.distinction.club` being executed. The output displays five successful ping responses from `client.distinction.club (192.168.200.5)` with varying times. Below the responses, the ping statistics are shown: "5 packets transmitted, 5 received, 0% packet loss, time 4015ms". The text "5 received" is highlighted with a red box. The terminal background is a dark blue with a light blue geometric pattern.

```
[omarsNA@server ~]$ ping client.distinction.club
PING client.distinction.club (192.168.200.5) 56(84) bytes of data:
64 bytes from client.distinction.club (192.168.200.5): icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from client.distinction.club (192.168.200.5): icmp_seq=2 ttl=64 time=1.43 ms
64 bytes from client.distinction.club (192.168.200.5): icmp_seq=3 ttl=64 time=0.826 ms
64 bytes from client.distinction.club (192.168.200.5): icmp_seq=4 ttl=64 time=1.47 ms
64 bytes from client.distinction.club (192.168.200.5): icmp_seq=5 ttl=64 time=1.77 ms
^C
--- client.distinction.club ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4015ms
rtt min/avg/max/mdev = 0.826/1.321/1.774/0.326 ms
[omarsNA@server ~]$
```

Ubuntu:



A screenshot of a terminal window on Ubuntu. The window title is "Snaomar@client: ~/Desktop". The terminal shows the command `ping server.distinction.club` being executed. The output displays five successful ping responses from `server.distinction.club (192.168.200.4)` with varying times. Below the responses, the ping statistics are shown: "5 packets transmitted, 5 received, 0% packet loss, time 5119ms". The text "5 received" is highlighted with a red box. The terminal background is a dark purple with a light purple geometric pattern.

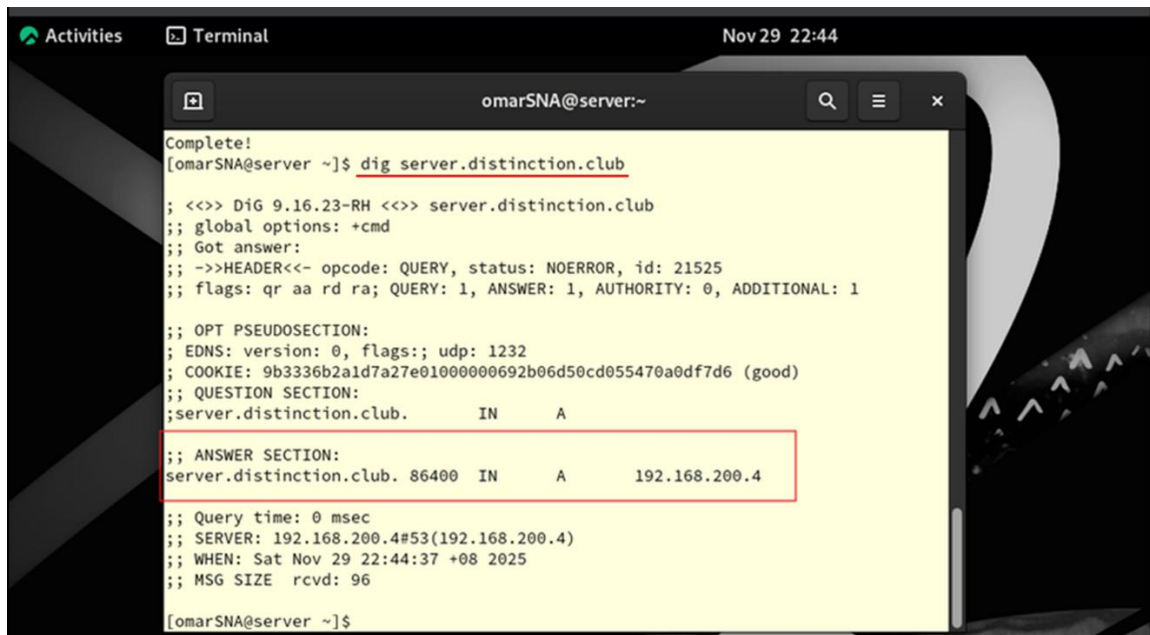
```
Snaomar@client:~/Desktop$ ping server.distinction.club
PING server.distinction.club (192.168.200.4) 56(84) bytes of data:
64 bytes from server.distinction.club (192.168.200.4): icmp_seq=1 ttl=64 time=2.02 ms
64 bytes from server.distinction.club (192.168.200.4): icmp_seq=2 ttl=64 time=1.38 ms
64 bytes from server.distinction.club (192.168.200.4): icmp_seq=3 ttl=64 time=1.49 ms
64 bytes from server.distinction.club (192.168.200.4): icmp_seq=4 ttl=64 time=1.42 ms
64 bytes from server.distinction.club (192.168.200.4): icmp_seq=5 ttl=64 time=1.56 ms
^C
--- server.distinction.club ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 5119ms
rtt min/avg/max/mdev = 1.380/1.572/2.015/0.229 ms
Snaomar@client:~/Desktop$
```

3.5.3 Dig Method

Rocky:

- \$ dig server.distinction.club
- \$ dig client.distinction.club

Figure 8 dig method verification of server.distinction.club domain on Rocky server



```

Complete!
[omarSNA@server ~]$ dig server.distinction.club

; <<>> DiG 9.16.23-RH <<>> server.distinction.club
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 21525
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

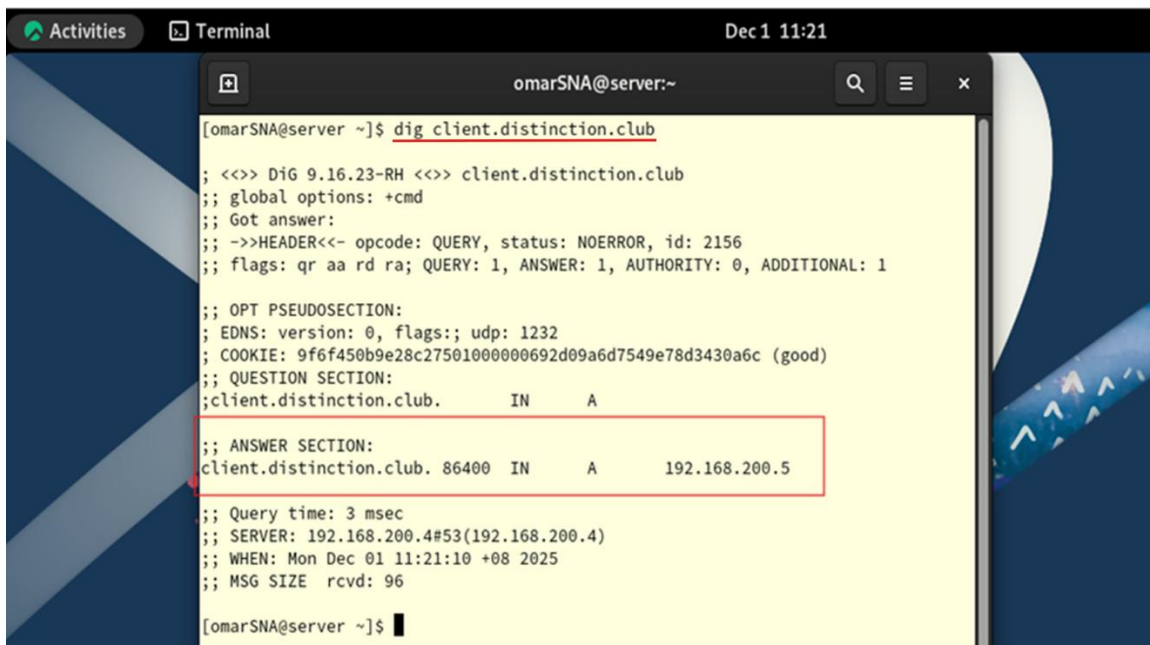
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 9b3336b2ald7a27e01000000692b06d50cd055470a0df7d6 (good)
;; QUESTION SECTION:
;server.distinction.club.      IN      A

;; ANSWER SECTION:
server.distinction.club. 86400 IN      A      192.168.200.4

;; Query time: 0 msec
;; SERVER: 192.168.200.4#53(192.168.200.4)
;; WHEN: Sat Nov 29 22:44:37 +08 2025
;; MSG SIZE rcvd: 96

[omarSNA@server ~]$
  
```

Figure 9 dig method verification of client.distinction.club domain on Rocky server



```

[omarSNA@server ~]$ dig client.distinction.club

; <<>> DiG 9.16.23-RH <<>> client.distinction.club
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 2156
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 9f6f450b9e28c27501000000692d09a6d7549e78d3430a6c (good)
;; QUESTION SECTION:
;client.distinction.club.      IN      A

;; ANSWER SECTION:
client.distinction.club. 86400 IN      A      192.168.200.5

;; Query time: 3 msec
;; SERVER: 192.168.200.4#53(192.168.200.4)
;; WHEN: Mon Dec 01 11:21:10 +08 2025
;; MSG SIZE rcvd: 96

[omarSNA@server ~]$
  
```

Ubuntu:

- o \$ dig server.distinction.club

- o \$ dig client.distinction.club

Figure 10 dig method verification of server.distinction.club domain on Ubuntu client

```

Snaomar@client: ~/Desktop
Snaomar@client:~/Desktop$ dig server.distinction.club

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> server.distinction.club
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39160
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;server.distinction.club.      IN      A

;; ANSWER SECTION:
server.distinction.club. 86400 IN      A      192.168.200.4

;; Query time: 5681 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Dec 01 03:26:07 UTC 2025
;; MSG SIZE rcvd: 68

Snaomar@client:~/Desktop$

```

Figure 11 dig method verification of client.distinction.club on Ubuntu client

```

Snaomar@client: ~/Desktop
Snaomar@client:~/Desktop$ dig client.distinction.club

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> client.distinction.club
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 64173
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;client.distinction.club.     IN      A

;; ANSWER SECTION:
client.distinction.club. 86400 IN      A      192.168.200.5

;; Query time: 3 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Mon Dec 01 03:27:31 UTC 2025
;; MSG SIZE rcvd: 68

Snaomar@client:~/Desktop$

```

3.6 Personal Reflection

I have successfully configured both DNS with BIND9 which helped me to understand how the essential name resolution is in a network, configuring both the forward and reverse zones has taught me that hostnames and IPs are interconnected and that a simple error can lead to lookup verification failures. Using nslookup and dig helped me to improve my troubleshooting capabilities also helped me to verify each configuration step that I did, Overall, this assignment task has enhanced my understanding of DNS and its importance towards other services in the system.

CHAPTER 4: INSTALLING & CONFIGURING AN EMAIL SERVER

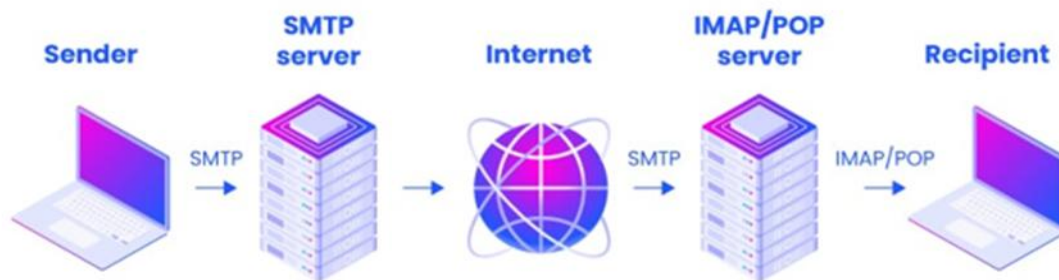
4.1 An Overview

This section documents the process of building a secure email system using Postfix, Dovecot, and Thunderbird. The configuration was performed on a Rocky Linux server with an Ubuntu client, aiming to establish encrypted mail transmission and storage through SSL/TLS protocols. The implementation includes network setup, user creation, SSL certificate generation, service configuration, troubleshooting, and validation.

The aim of this project is to implement a secure, fully functional mail system for local users using Postfix (SMTP server) and Dovecot (IMAP/POP3 server). Thunderbird is used as the client on Ubuntu to send and receive messages securely via SSL/TLS encryption.

4.2 What is a Mail Server?

A mail server also known as an email server is a software program that sends and receives email. It is frequently used as a catch-all term for both mail transfer agents (MTA) and mail delivery agents (MDA), each of which serves a slightly different purpose.



The above linear diagram illustrates the technical journey of an email from a sender to a recipient. Starting on the left, a "Sender" laptop transmits a message via the SMTP protocol to an SMTP server. The data then travels across the Internet to an IMAP/POP server, where it is finally retrieved by the "Recipient" laptop using either IMAP or POP protocols.

4.3 SMTP, IMAP and POP3 Explained

Think of the email system like the physical postal service. When you send a letter, you drop it in a mailbox, and a mail truck takes it to a sorting center. Eventually, it arrives at the recipient's local post office, where they can choose how to pick it up.

In this analogy:

SMTP is the Mail Truck (It sends the mail).

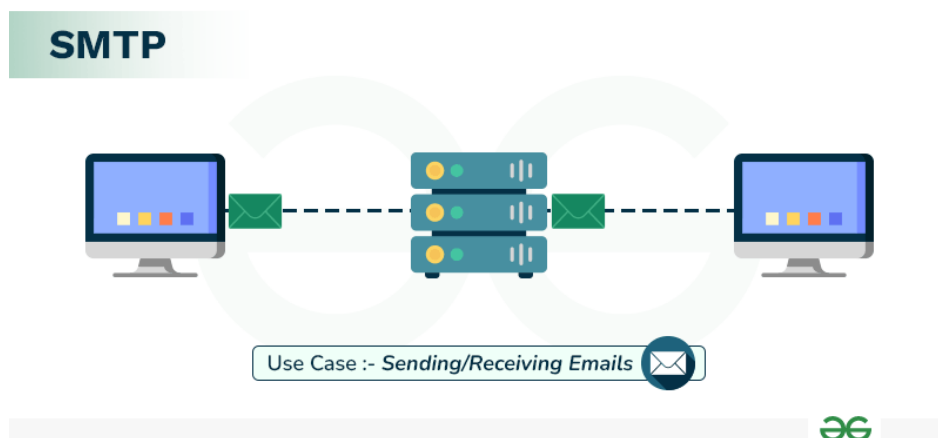
POP3 and IMAP are two different ways to check your mailbox at the post office.

SMTP (Simple Mail Transfer Protocol)

"The Sender"

SMTP is the industry standard protocol **used for sending emails**. It is responsible for pushing your email from your computer to your email server, and then from your server to the recipient's server.

- **Role:** Sending and Relaying.
- **Direction:** Push (Client → Server → recipient Server).
- **Analogy:** The mail carrier who takes your letter from your house and drives it across the country to the recipient's local post office.
- **Key Behavior:** SMTP only handles "outbound" traffic. It cannot be used to read or retrieve emails from an inbox.

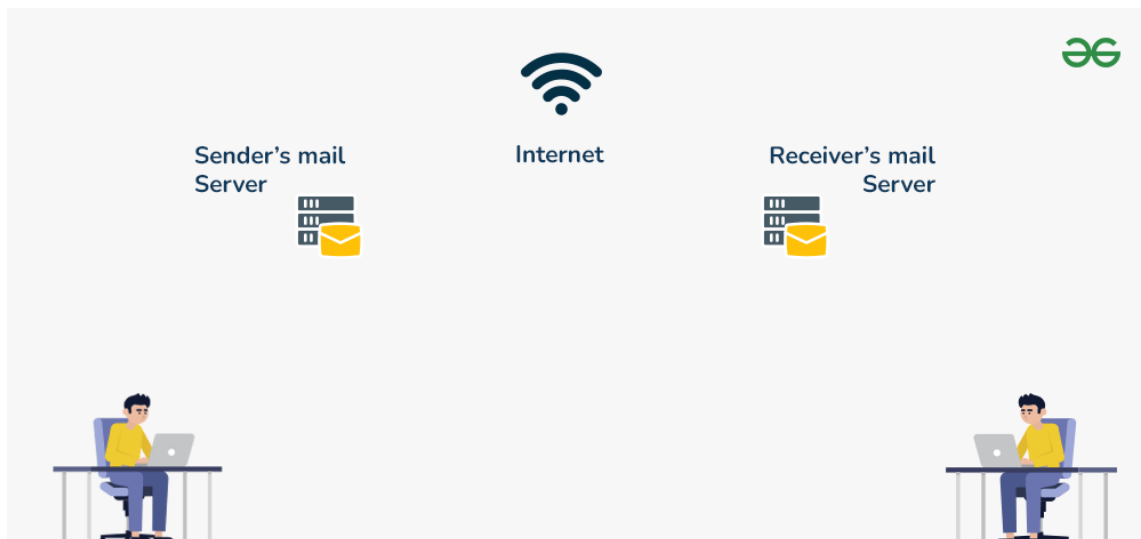


IMAP (Internet Message Access Protocol)

"The Syncer"

IMAP is a **method for receiving email** where the **email stays on the server**. When you check your email, your device effectively "looks through a window" at the server.

- **Role:** Retrieving/Viewing.
- **Key Feature: Synchronization.** If you read, delete, or organize an email on your phone, it also happens on your laptop and the server instantly.
- **Best For:** People who use multiple devices (e.g., a smartphone, a work laptop, and a home tablet).
- **Pros:** Your email is backed up on the server; you see the same inbox everywhere.
- **Cons:** You can fill up your server's storage limit quickly.

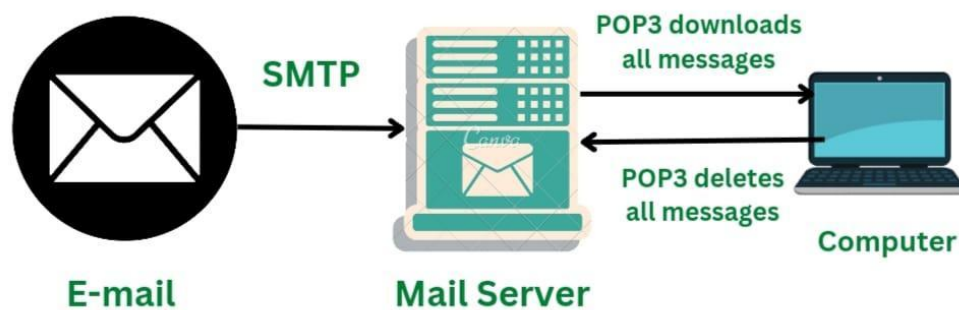


POP3 (Post Office Protocol 3)

"The Downloader"

POP3 is an older method for receiving email. It was designed when people only had one computer and internet storage was expensive. It connects to the server, **downloads all your emails** to your specific device, and then (usually) **deletes** them from the server.

- **Role:** Retrieving/Downloading.
- **Key Feature: Local Storage.** Once downloaded, the email exists *only* on that specific computer.
- **Best For:** Privacy advocates or people with very limited server storage who use only one specific computer to check mail.
- **Pros:** Works well without an internet connection (once downloaded); doesn't use up server space.
- **Cons:** If your computer hard drive fails, you lose all your emails. If you check email on your phone, it won't be there when you get to your computer.



Feature	SMTP	IMAP	POP3
Primary Function	Sending email	Accessing email	Downloading email
Direction	Outgoing (Push)	Incoming (Pull/Sync)	Incoming (Pull/Move)
Where is email stored?	N/A (Transit only)	Server (Cloud)	Local Device (Hard Drive)
Multi-device support?	N/A	Excellent (Syncs everywhere)	Poor (Fragmented)
Offline Access	No	Limited (Header only, usually)	Yes (Full content)

The Email Flow Diagram

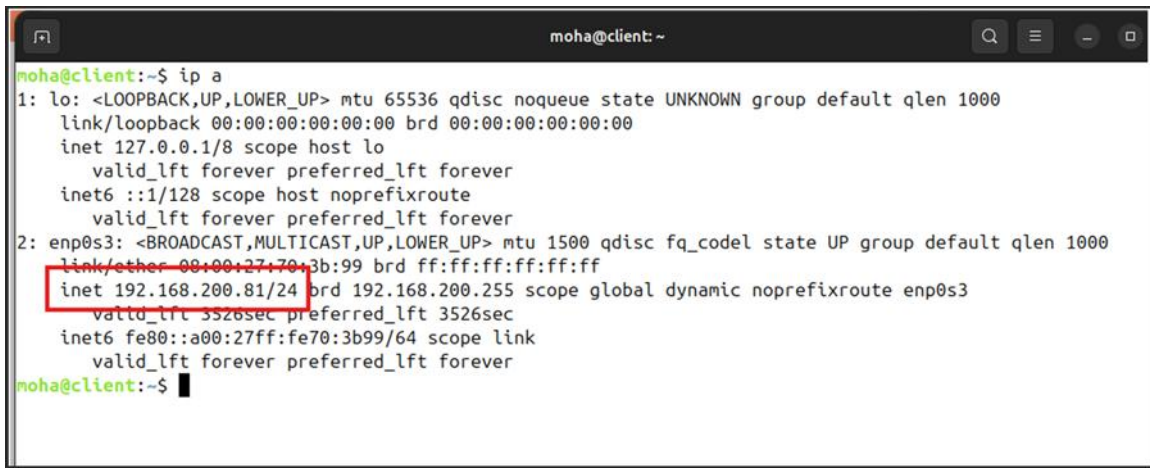
To visualize how they work together:

1. You (Sender) write an email and click send.
2. Your computer uses SMTP to push the email to your Google/Outlook Server.
3. Your server uses SMTP to relay the email to the Recipient's Server.
4. The email sits in the Recipient's Server waiting.
5. The Recipient opens their laptop:
 - If they use IMAP, they view the email while it stays on the server.
 - If they use POP3, they download the email to their laptop and remove it from the server.

4.3 System Setup and Configuration

Both Rocky Linux and Ubuntu were configured within the **same NAT network**. IP connectivity was verified using ping tests.

Figure 12 Ubuntu client network interface (192.168.200.81)



```
moha@client:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:70:3b:99 brd ff:ff:ff:ff:ff:ff
    inet 192.168.200.81/24 brd 192.168.200.255 scope global dynamic noprefixroute enp0s3
        valid_lft 3526sec preferred_lft 3526sec
    inet6 fe80::a00:27ff:fe70:3b99/64 scope link
        valid_lft forever preferred_lft forever
moha@client:~$
```

This screenshot confirms that the Ubuntu client has been assigned the IP address 192.168.200.81 on the NAT network. At this stage, you should verify that the client is in the same subnet as the Rocky server so that IMAP and SMTP traffic can flow between them.

Figure 13 Rocky server network interface (192.168.200.4)



```
[moha@server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:70:5b:d0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.200.4/24 brd 192.168.200.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe70:5bd0/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
[moha@server ~]$
```

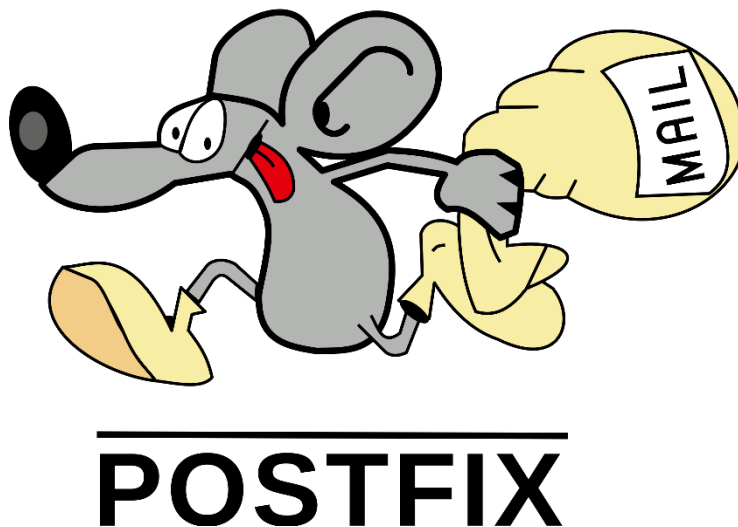
Here we verify that the Rocky Linux server is using IP address 192.168.200.4. This is the address that the Ubuntu client will later use to connect to the Postfix and Dovecot services.

Hostnames were configured to establish a local domain for email addressing. The server was named **server.distinction.club**, and the client was named **client.distinction.club**.

4.4 Introduction to Mail Server

Currently, to create a safe and entirely functioning mail server environment, three fundamental elements need to be introduced and adjusted appropriately: Postfix, dovecot, and tools of SSL. Postfix is the SMTP server of the system as it sends, accepts, and routes the email on the network. Dovecot offers IMAP and POP3 services which enable users to access their mailboxes and manage them in a secure manner. Cryptographic keys can be generated and managed by the server using SSL tools like OpenSSL. This ensures that all communication between email clients and the server is encrypted and secure. These components are the backbone of a safe and sound mail delivery system and before going ahead with configuration it is important to ensure that they are installed, enabled, and functioning properly before configuration can be done.

4.5 Introduction to Postfix



Postfix is the SMTP server that is used to relay and accept emails in the system. It handles routing of messages, mail queue control, domestic delivery, and inter-communication with other mail servers. In an email infrastructure, the responsibility of Postfix is the acceptance of outgoing mail, the reception of incoming mail on port 25 and the further routing of the messages to the individual mailboxes of the users. Postfix also ensures that the server can serve all SMTP functions in a competent manner on activation and continued use.

4.6 Introduction to Dovecot



Postfix delivers the messages; Dovecot allows clients such as Thunderbird to read and manage those messages securely. It supports IMAP (port 143/993) and POP3 (port 110/995), manages user authentication, and Dovecot is the IMAP/POP3 server that provides users access to their mailboxes. While ensures fast and secure mailbox handling. A running Dovecot service is essential for letting users log in and retrieve their mail.

4.7 Introduction to OpenSSL



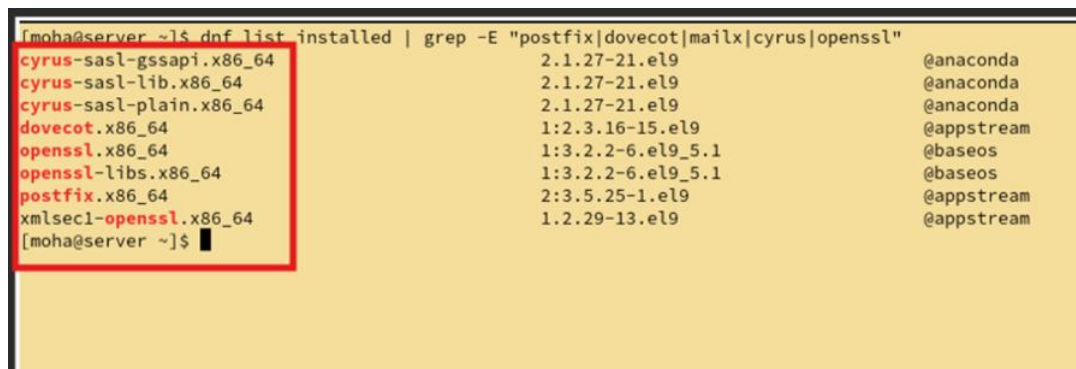
OpenSSL is a premier open-source toolkit implementing SSL/TLS protocols, serving as the foundation for secure network communications. It is pivotal for generating SSL certificates and managing cryptographic keys. This setup enables the server to provide encrypted versions of SMTP, IMAP, and POP3. When users connect via secure ports—specifically 465 or 587 for sending mail and 993 for retrieving it—OpenSSL encrypts the entire session. This effectively safeguards passwords and email content from potential eavesdropping, ensuring total confidentiality for all user data.

4.8 Installation of Postfix, Dovecot and OpenSSL

After confirming network connectivity, Postfix, Dovecot, and SSL tools were installed using the DNF package manager. The services were then enabled and verified to ensure proper operation.

- o `dnf list installed | grep -E "postfix | dovecot | mailx | cyrus | openssl"`

Figure 14 Installed mail and SSL packages

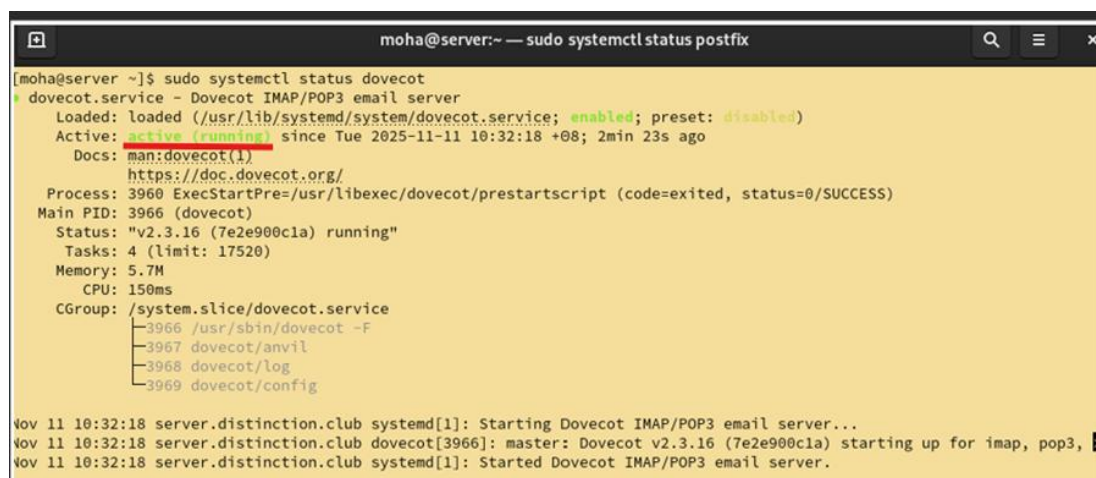


```
[moha@server ~]$ dnf list installed | grep -E "postfix|dovecot|mailx|cyrus|openssl"
cyrus-sasl-gssapi.x86_64      2.1.27-21.el9      @anaconda
cyrus-sasl-lib.x86_64        2.1.27-21.el9      @anaconda
cyrus-sasl-plain.x86_64      2.1.27-21.el9      @anaconda
dovecot.x86_64               1:2.3.16-15.el9    @appstream
openssl.x86_64               1:3.2.2-6.el9_5.1  @baseos
openssl-libs.x86_64         1:3.2.2-6.el9_5.1  @baseos
postfix.x86_64               2:3.5.25-1.el9     @appstream
xmlsec1-openssl.x86_64      1.2.29-13.el9      @appstream
[moha@server ~]$
```

This list shows that the required packages such as postfix, dovecot and OpenSSL are installed. At this point you should ensure that all mail and SSL packages are present before proceeding to configuration.

- o `sudo systemctl status dovecot`

Figure 15 Dovecot active and running



```
[moha@server ~]$ sudo systemctl status dovecot
dovecot.service - Dovecot IMAP/POP3 email server
Loaded: loaded (/usr/lib/systemd/system/dovecot.service; enabled; preset: disabled)
Active: active (running) since Tue 2025-11-11 10:32:18 +08; 2min 23s ago
Docs: man:dovecot(1)
      https://doc.dovecot.org/
Process: 3960 ExecStartPre=/usr/libexec/dovecot/prestartscript (code=exited, status=0/SUCCESS)
Main PID: 3966 (dovecot)
Status: "v2.3.16 (7e2e90c1a) running"
Tasks: 4 (limit: 17520)
Memory: 5.7M
CPU: 150ms
CGroup: /system.slice/dovecot.service
        └─3966 /usr/sbin/dovecot -F
           └─3967 dovecot/anvil
              └─3968 dovecot/log
                 └─3969 dovecot/config

Nov 11 10:32:18 server.distinction.club systemd[1]: Starting Dovecot IMAP/POP3 email server...
Nov 11 10:32:18 server.distinction.club dovecot[3966]: master: Dovecot v2.3.16 (7e2e90c1a) starting up for imap, pop3,
Nov 11 10:32:18 server.distinction.club systemd[1]: Started Dovecot IMAP/POP3 email server.
```

The systemctl status output indicates that the Dovecot IMAP/POP3 service is active and running without errors. This is essential because Dovecot will be responsible for serving user mailboxes securely over IMAP.

- `sudo systemctl status postfix`

Figure 16 Postfix active and running

```
[moha@server ~]$ sudo systemctl status postfix.service
postfix.service - Postfix Mail Transport Agent
Loaded: loaded (/usr/lib/systemd/system/postfix.service; enabled; preset: disabled)
Active: active (running) since Tue 2025-11-11 10:32:06 +08; 2min 47s ago
Process: 3870 ExecStartPre=/usr/sbin/restorecon -R /var/spool/postfix/pid (code=exited, status=0/SUCCESS)
Process: 3872 ExecStartPre=/usr/libexec/postfix/aliasesdb (code=exited, status=0/SUCCESS)
Process: 3874 ExecStartPre=/usr/libexec/postfix/chroot-update (code=exited, status=0/SUCCESS)
Process: 3875 ExecStart=/usr/sbin/postfix start (code=exited, status=0/SUCCESS)
Main PID: 3943 (master)
Tasks: 3 (limit: 17520)
Memory: 4.1M
CPU: 699ms
CGroup: /system.slice/postfix.service
├─3943 /usr/libexec/postfix/master -w
├─3944 pickup -l -t unix -u
└─3945 qmgr -l -t unix -u

Nov 11 10:32:06 server.distinction.club postfix[3915]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3915]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3915]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3920]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3920]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3920]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3920]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix[3920]: /usr/sbin/postconf: warning: /etc/postfix/main.cf: unused parameter
Nov 11 10:32:06 server.distinction.club postfix/postfix-script[3941]: starting the Postfix mail system
Nov 11 10:32:06 server.distinction.club postfix/master[3943]: daemon started -- version 3.5.25, configuration /etc/postfix
Nov 11 10:32:06 server.distinction.club systemd[1]: Started Postfix Mail Transport Agent.

lines 1-26/26 (END)
```

Here we see Postfix running correctly as the system's SMTP server. Once Postfix is active, it can accept messages, route them locally to Maildir, and later support secure submission from Thunderbird.

- Sudo systemctl restart dovecot postfix

Figure 17 Open ports for SMTP, IMAP, and secure mail protocols

```
[moha@server ~]$ sudo systemctl restart dovecot postfix
[moha@server ~]$ sudo ss -tuln | grep -E "25|110|143|465|587|993|995"
tcp        LISTEN     0          100        0.0.0.0:465      0.0.0.0:*
tcp        LISTEN     0          100        0.0.0.0:110     0.0.0.0:*
tcp        LISTEN     0          100        0.0.0.0:25      0.0.0.0:*
tcp        LISTEN     0          100        0.0.0.0:143     0.0.0.0:*
tcp        LISTEN     0          100        0.0.0.0:995     0.0.0.0:*
tcp        LISTEN     0          100        0.0.0.0:993     0.0.0.0:*
tcp        LISTEN     0          100        0.0.0.0:587     0.0.0.0:*
tcp        LISTEN     0          100        [::]:465        [::]:*
tcp        LISTEN     0          100        [::]:110        [::]:*
tcp        LISTEN     0          100        [::]:25         [::]:*
tcp        LISTEN     0          100        [::]:143        [::]:*
tcp        LISTEN     0          100        [::]:995        [::]:*
tcp        LISTEN     0          100        [::]:993        [::]:*
tcp        LISTEN     0          100        [::]:587        [::]:*
[moha@server ~]$
```

The open ports list confirms that the expected services are listening—for example, port 25/465 for SMTP/SMTPS, and port 143/993 for IMAP/IMAPS.

4.9 SSL/TLS Certificate Configuration

A self-signed SSL certificate was generated using OpenSSL to enable encrypted connections for both Postfix and Dovecot. The certificate and private key were stored under `/etc/ssl/certs` and `/etc/ssl/private`, respectively.

Figure 18 Certificate generation process

[illegible]

This screenshot shows the **OpenSSL** command used to generate a self-signed **SSL certificate** and private key. During this step you typically provide details such as country, organization, and **Common Name (CN)**, which should match the mail server hostname.

Figure 19 Certificate fingerprint verification on Rocky

```

[moha@server ~]$ sudo openssl x509 -in /etc/ssl/certs/mailcert.pem -noout \
-subject -issuer -serial -fingerprint -dates
subject=C=MY, ST=KualaLumpur, L=APU, O=DistinctionClub, OU=IT, CN=server.distinction.club, emailAddress=admin@distinction.club
issuer=C=MY, ST=KualaLumpur, L=APU, O=DistinctionClub, OU=IT, CN=server.distinction.club, emailAddress=admin@distinction.club
serial=12E0F273E03F8B2CA05D110B0B10BC60C0004
SHA1 Fingerprint=E6:41:24:AC:3D:B8:5A:A2:EA:F7:C2:65:44:75:37:9B:DB:84:9B:6B
notBefore=Nov 11 03:31:04 2025 GMT
notAfter=Nov 11 03:31:04 2026 GMT
[moha@server ~]$

```

Figure 20 Certificate Chain Verification on Ubuntu

```

moha@client:~$ openssl s_client -connect server.distinction.club:993 -quiet
depth=0 C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.distinction.club
verify error:num=18:self-signed certificate
verify return:1
depth=0 C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.distinction.club
verify return:1
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ AUTH=PLAIN AUTH=LOGIN] Dovecot ready.
^C
moha@client:~$ openssl s_client -connect server.distinction.club:993
CONNECTED(00000003)
depth=0 C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.distinction.club
verify error:num=18:self-signed certificate
verify return:1
depth=0 C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.distinction.club
verify return:1
Certificate chain
0 s:C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.distinction.club
i:C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.distinction.club
v:NotBefore: Nov 10 17:29:01 2025 GMT; NotAfter: Nov 10 17:29:01 2026 GMT

```

Here we inspect the generated certificate to confirm its validity period and fingerprint. Verifying the certificate ensures that Postfix and Dovecot will be using the exact certificate you created for secure connections.

4.10 Introduction to Thunderbird

Figure 21 Thunderbird Logo

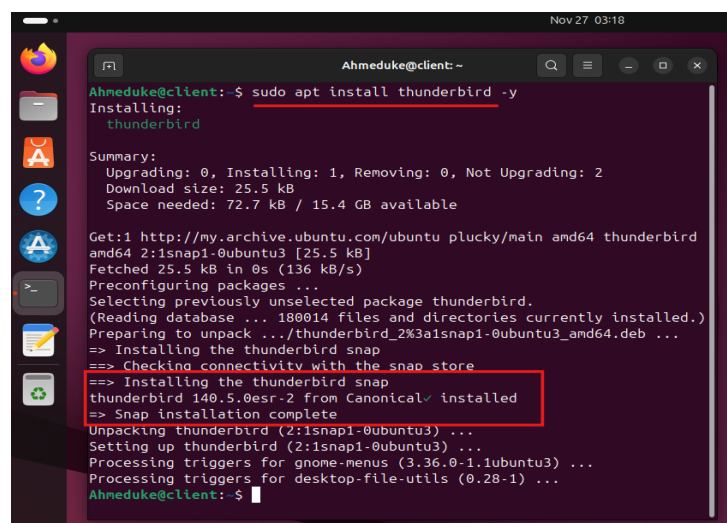


Thunderbird is a free and open-source email client developed by Mozilla Foundation. Designed to manage email, news feeds, chat, and calendaring, Thunderbird is renowned for its intuitive layout and wealth of customization possibilities. With its support for IMAP, POP3 and SMTP email protocols, users may access and manage their email from a variety of providers (Pal, 2022).

After completing the server-side configuration of Postfix and Dovecot on the Rocky Linux system, the next step was to configure an email client to interact with the mail server. For this purpose, Thunderbird was installed on the Ubuntu client machine.

○ `sudo apt install thunderbird -y`

Figure 22 Installing Thunderbird on Ubuntu Client

A screenshot of a terminal window on an Ubuntu system. The terminal shows the command `sudo apt install thunderbird -y` being executed. The output displays the installation progress, including downloading the package, preconfiguring packages, and installing the snap. A red box highlights the line `thunderbird 140.5.0esr-2 from Canonical✓ installed`. The terminal window has a dark background and a title bar that reads "Ahmeduke@client: ~". The system clock in the top right corner shows "Nov 27 03:18".

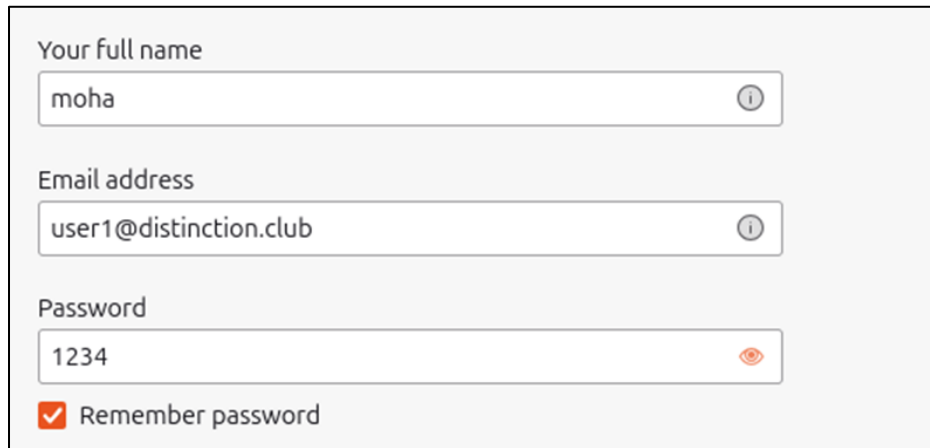
```
Ahmeduke@client: ~  
Ahmeduke@client:~$ sudo apt install thunderbird -y  
Installing:  
thunderbird  
  
Summary:  
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2  
Download size: 25.5 kB  
Space needed: 72.7 kB / 15.4 GB available  
  
Get:1 http://my.archive.ubuntu.com/ubuntu plucky/main amd64 thunderbird  
amd64 2:1snap1-0ubuntu3 [25.5 kB]  
Fetched 25.5 kB in 0s (136 kB/s)  
Preconfiguring packages ...  
Selecting previously unselected package thunderbird.  
(Reading database ... 180014 files and directories currently installed.)  
Preparing to unpack .../thunderbird_2%3a1snap1-0ubuntu3_amd64.deb ...  
=> Installing the thunderbird snap  
==> Checking connectivity with the snap store  
==> Installing the thunderbird snap  
thunderbird 140.5.0esr-2 from Canonical✓ installed  
=> Snap installation complete  
Unpacking thunderbird (2:1snap1-0ubuntu3) ...  
Setting up thunderbird (2:1snap1-0ubuntu3) ...  
Processing triggers for gnome-menus (3.36.0-1.1ubuntu3) ...  
Processing triggers for desktop-file-utils (0.28-1) ...  
Ahmeduke@client:~$
```

4.11 Setting up Thunderbird

Account Details Entry

Once Thunderbird was launched, the user was prompted to enter their account information, as shown in the screenshot below:

Figure 23 Thunderbird account login credentials

A screenshot of the Thunderbird account setup window. It contains three input fields: 'Your full name' with the value 'moha', 'Email address' with the value 'user1@distinction.club', and 'Password' with the value '1234'. Each field has an information icon (i) to its right. Below the password field is a checkbox labeled 'Remember password' which is checked. The entire form is enclosed in a light gray border.

- **Full Name:** moha
- **Email Address:** user1@distinction.club
- **Password:** 1234

This information corresponds to the local Linux user created earlier on the Rocky server. The password is the same Linux user password because Postfix/Dovecot uses system-based authentication. Selecting “Remember password” allows Thunderbird to store the credentials securely and prevents repeated login prompts during testing.

This step ensures that Thunderbird knows which mailbox it should attempt to access on the Rocky server.

Manual Server Configuration

Since this is a private server hosted in a virtual environment without DNS auto-configuration, **Thunderbird cannot automatically detect the IMAP and SMTP settings**. After entering the email address, Thunderbird displays a yellow warning message:

Your full name
moha

Email address
user1@distinction.club

Password
1234

☒ Remember password

Thunderbird failed to find the settings for your email account

Manual configuration

INCOMING SERVER

Protocol: IMAP

Hostname: .distinction.club

Port: 993

Connection security: SSL/TLS

Authentication method: Normal password

Username: user1@distinction.club

OUTGOING SERVER

Hostname: .distinction.club

Port: 465

Connection security: SSL/TLS

Authentication method: Normal password

Username: user1@distinction.club

[Advanced config](#)

Re-test Cancel Done

Done

“Thunderbird failed to find the settings for your email account.”

To proceed, the configuration must be done manually.

The following server settings were entered:

Incoming Mail Server (IMAP)

- **Protocol:** IMAP
- **Hostname:** server.distinction.club
- **Port:** 993 (IMAPS)
- **Connection Security:** SSL/TLS
- **Authentication Method:** Normal password
- **Username:** user1@distinction.club

This configuration allows Thunderbird to retrieve emails securely using Dovecot over IMAPS (encrypted IMAP). Port 993 ensures that communication is protected with SSL/TLS encryption.

Outgoing Mail Server (SMTP)

- **Hostname:** server.distinction.club
- **Port:** 465 (SMTPS)
- **Connection Security:** SSL/TLS
- **Authentication Method:** Normal password
- **Username:** user1@distinction.club

Manual configuration

INCOMING SERVER

Protocol: IMAP

Hostname: server.distinction.club

Port: 993

Connection security: SSL/TLS

Authentication method: Normal password

Username: user1

OUTGOING SERVER

Hostname: server.distinction.club

Port: 465

Connection security: SSL/TLS

Authentication method: Normal password

Username: user1

[Advanced config](#)

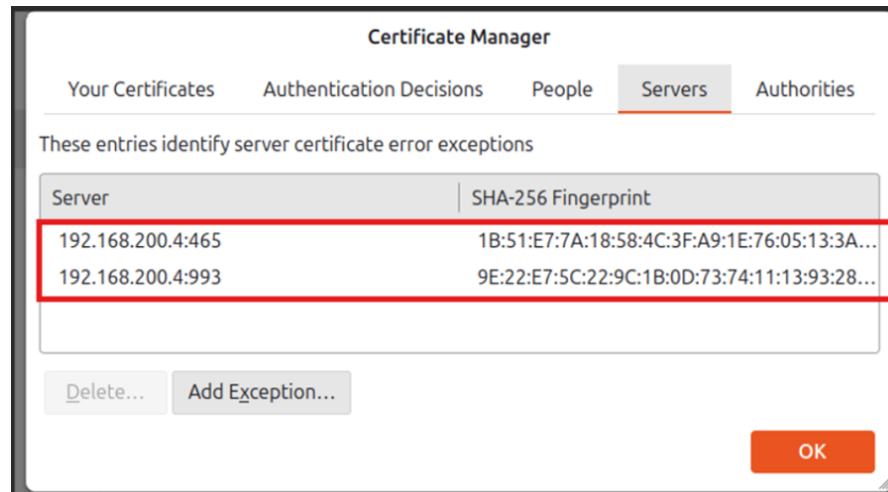
Re-test Cancel Done

Thunderbird will attempt to auto-detect fields that are left blank.

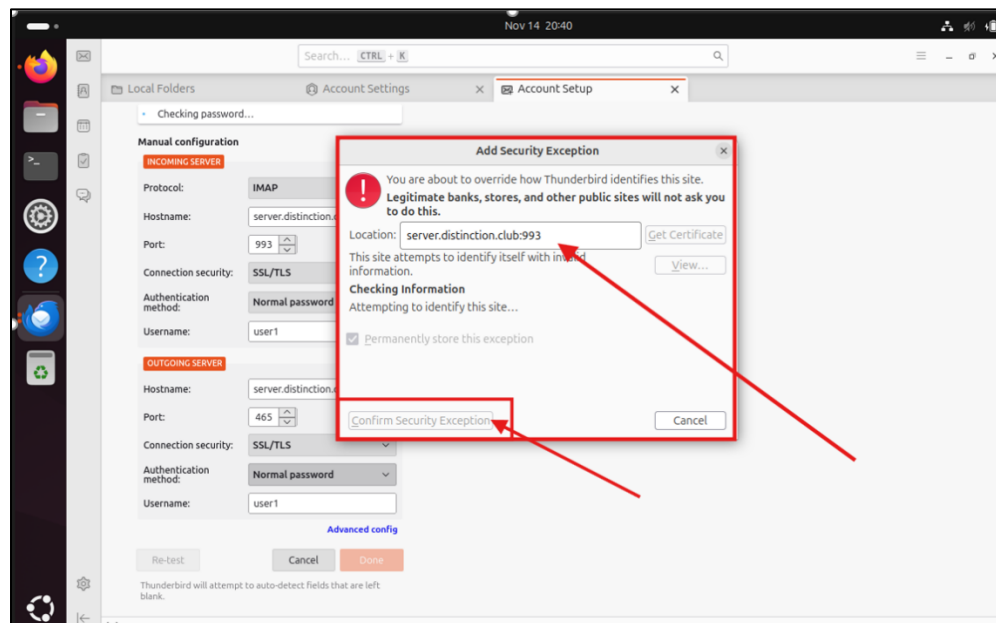
These settings allow Thunderbird to send emails through the Postfix SMTP server over an encrypted connection. Port 465 is used for SMTPS (SMTP over SSL/TLS), providing full encryption of outgoing messages.

4.12 Verification and Certificate Trust

NOTE: During the setup, Thunderbird detects that the server uses a self-signed SSL certificate. This is normal in a lab environment. Thunderbird displays a certificate warning and asks the user whether to trust the certificate.



By reviewing the certificate details and selecting “**Confirm Security Exception**”, Thunderbird is allowed to proceed with secure communication.



Once accepted, both IMAP and SMTP connections work over SSL/TLS, and Thunderbird can send and receive emails without security warnings.

4.13 Finalizing the Setup

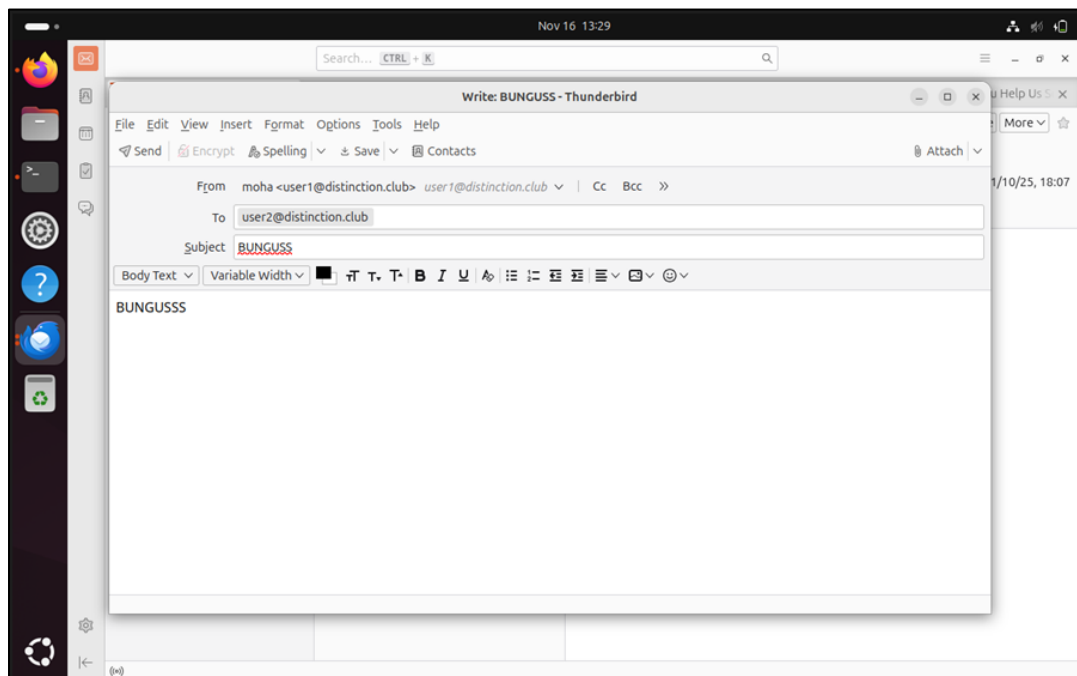
After entering the correct IMAP and SMTP settings and trusting the SSL certificate, Thunderbird successfully connects to the mail server. The **“Done”** button becomes active, confirming that the configuration is valid, and the account is ready to use.

This completes the client-side setup. Thunderbird is now fully synchronized with the **Postfix/Dovecot** server, supporting:

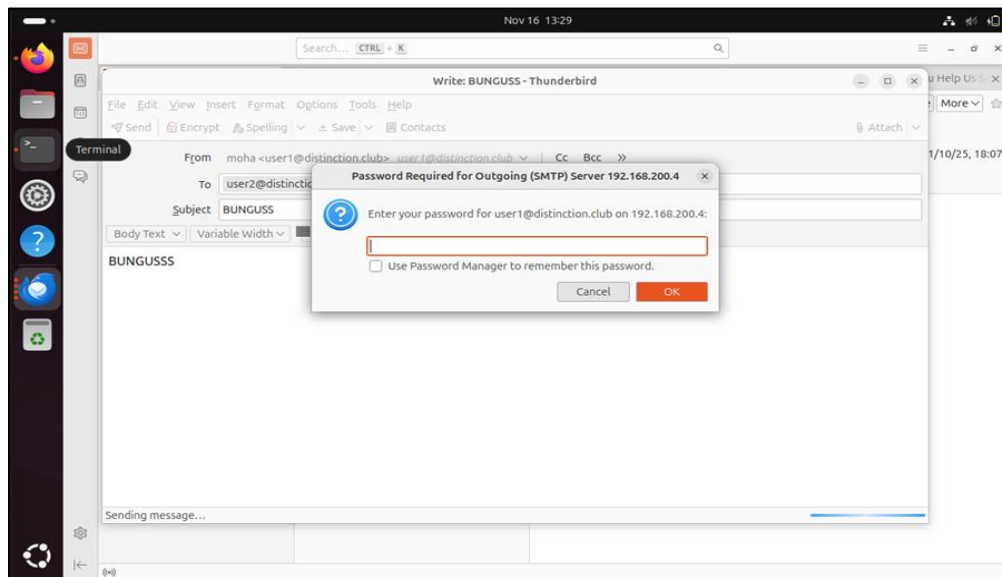
- Secure incoming mail (IMAPS port 993)
- Secure outgoing mail (SMTPS port 465)
- Password authentication
- Encrypted communication via SSL/TLS

4.14 Local Mail Delivery Testing

The user can now compose, send, and receive emails as part of the end-to-end testing of the mail server.



REMEMBER YOUR USER PASSWORD when sending email in THUNDERBIRD



After configuring **IMAP & SMTP** accounts in Thunderbird, a test email was sent from user1 to user2.

o Sudo -iu user2

```

Activities Terminal Nov 16 13:19
user1@server:~$ sudo -iu user2
[sudo] password for moha:
[user2@server ~]$ echo "Hello user1 BUNGUS , test email" | mail -s "Test to user1" user1
[user2@server ~]$ exit
logout
[moha@server ~]$ sudo -iu user1
[user1@server ~]$ ls -l ~/Maildir/new
-rw-rw-r-- 1 user1 user1 686 Nov 13 14:18 1763014710.Vfd00I116059SM625023.server.distinction.club
-rw-rw-r-- 1 user1 user1 468 Nov 14 18:06 1763114809.Vfd00I11649f6M136411.server.distinction.club
-rw-rw-r-- 1 user1 user1 437 Nov 14 18:10 1763115041.Vfd00I11649f7M775776.server.distinction.club
-rw-rw-r-- 1 user1 user1 468 Nov 14 18:16 1763115414.Vfd00I11649f8M820829.server.distinction.club
-rw-rw-r-- 1 user1 user1 461 Nov 16 13:15 1763270110.Vfd00I116059aM84823.server.distinction.club
[user1@server ~]$ ls -ltr ~/Maildir/new
total 20
-rw-rw-r-- 1 user1 user1 686 Nov 13 14:18 1763014710.Vfd00I116059SM625023.server.distinction.club
-rw-rw-r-- 1 user1 user1 468 Nov 14 18:06 1763114809.Vfd00I11649f6M136411.server.distinction.club
-rw-rw-r-- 1 user1 user1 437 Nov 14 18:10 1763115041.Vfd00I11649f7M775776.server.distinction.club
-rw-rw-r-- 1 user1 user1 468 Nov 14 18:16 1763115414.Vfd00I11649f8M820829.server.distinction.club
-rw-rw-r-- 1 user1 user1 461 Nov 16 13:15 1763270110.Vfd00I116059aM84823.server.distinction.club
[user1@server ~]$ cat ~/Maildir/new/1763270110.Vfd00I116059aM84823.server.distinction.club
Return-Path: <user2@distinction.club>
X-Original-To: user1
Delivered-To: user1@distinction.club
Received: by server.distinction.club (Postfix, from userid 5002)
        id 067FD21CD20A; Sun, 16 Nov 2025 13:15:09 +0800 (+08)
Date: Sun, 16 Nov 2025 13:15:09 +0800
To: user1@distinction.club
Subject: Test to user1
User-Agent: s-naïl v14.9.22
Message-Id: <20251116051510.067FD21CD20A@server.distinction.club>
From: user2@distinction.club

Hello user1 BUNGUS , test email
[user1@server ~]$
  
```

This confirms that Thunderbird can send messages through the secure SMTP connection. AND that's how we configure and test a secure email server using Postfix and Dovecot on Rocky Linux, with Thunderbird on Ubuntu as the client.

4.15 Personal Reflection

The culmination of this mail server set up and configuration project was considered a very useful learning experience, as it not only provided me with technical skills but also made me more confident in managing the actual Linux service administration.

Throughout the project, I was in a position to understand the interaction between the different elements namely Postfix, dovecot, and the use of the SSL/TLS certificates, Maildir storage, and Thunderbird to create a secure and reliable email system.

This troubleshooting phase and especially the fixing of permission issues, Maildir paths settings and certificate warnings of Thunderbird helped to develop a more analytical and methodical way of solving problems.

Instead of using only the trial & error method, I embraced the practices of inspecting logs, checking of settings, and reasoning the actions of every service.

In general, the project taught me not just to implement a working secure mail environment, but also helped me to sharpen my skills in diagnosing problems and fixing them in a professional systematic way.

CHAPTER 5: INSTALLING & CONFIGURING A WEB SERVER USING APACHE

5.1 Introduction to Apache

Figure 24 Apache Web Server Logo



Apache is more than just software that delivers web pages it's a foundational piece of internet infrastructure that quietly runs behind countless websites we use every day. What makes Apache interesting isn't only its popularity, but its design philosophy. Instead of acting like a simple "on/off" web server, Apache gives full control to the administrator, letting them shape how a website behaves, responds, protects itself, and interacts with users.

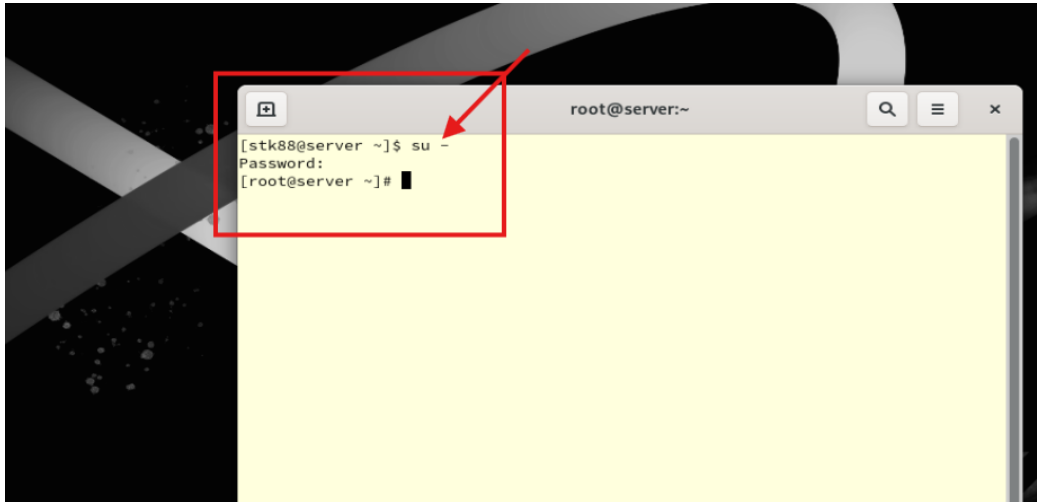
The strength of Apache comes from its modular architecture. Every feature -whether it's encryption, authentication, redirection, or error handling is treated like a building block that can be customized, rearranged, or combined depending on what the website actually needs. This makes Apache not just a tool, but a platform for creating secure and reliable web environments.

Exploring Apache from a configuration level gave me a different appreciation for how websites work behind the scenes. It showed me that things we take for granted, like secure connections, login prompts, or polished error pages, don't happen by accident -they happen because someone configured the server intelligently. Apache makes that possible, and understanding it feels like learning the "backstage controls" of the modern web.

5.2 Switching to Root in Rocky

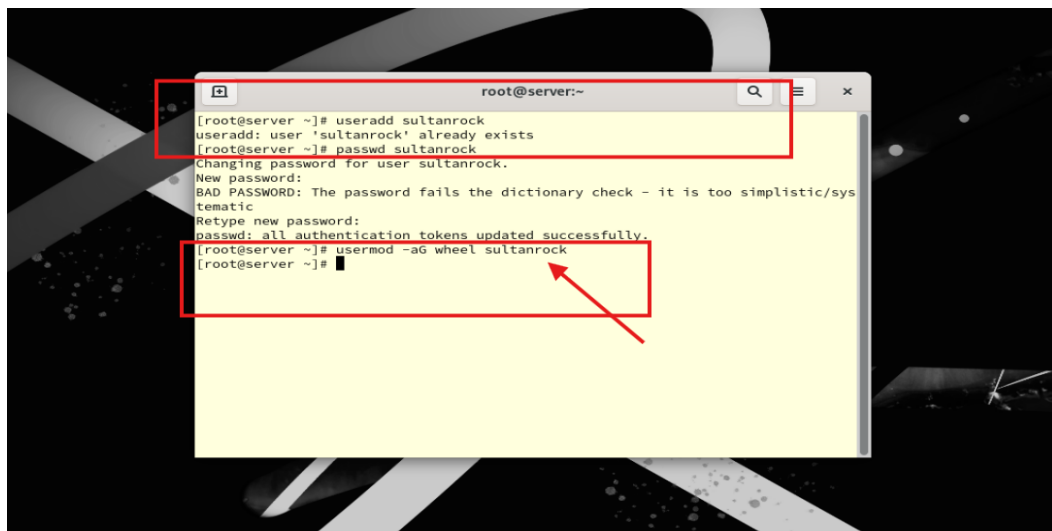
This gives you root access, so you can install and configure things

- `$ su -`



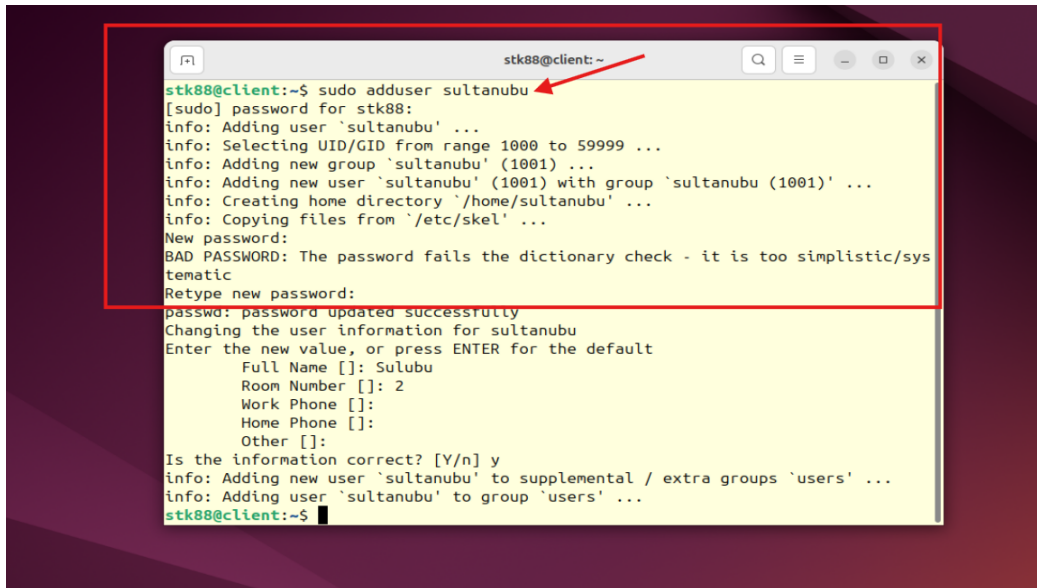
5.3 Creating Users on Rocky Server

- `# useradd sultanrock`
- `# passwd sultanrock`
- `# usermod -aG wheel sultanrock`



We create users to separate server responsibilities. The Rocky user (sultanrock) is the admin-level user who manages the server.

- \$ sudo useradd sultanubu

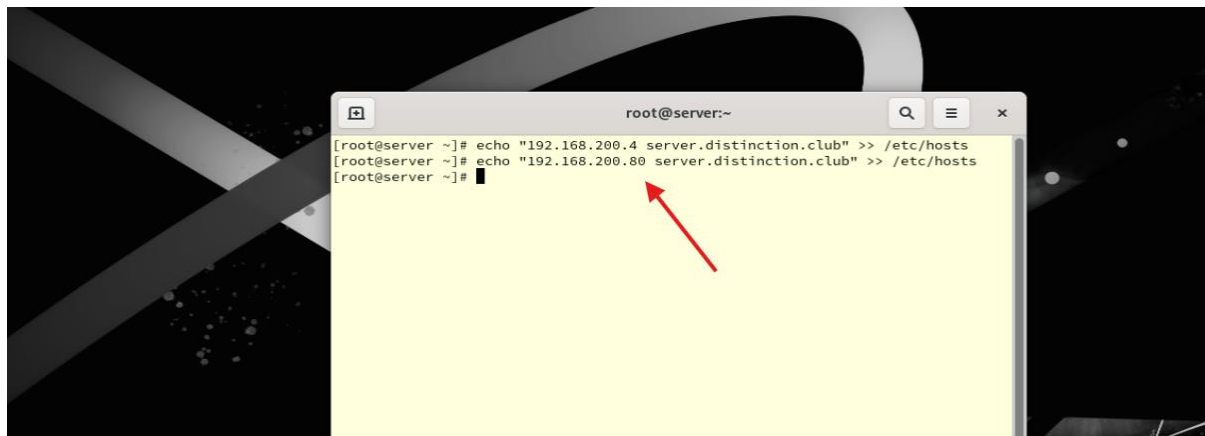


```
stk88@client:~$ sudo adduser sultanubu
[sudo] password for stk88:
info: Adding user `sultanubu' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `sultanubu' (1001) ...
info: Adding new user `sultanubu' (1001) with group `sultanubu (1001)' ...
info: Creating home directory `/home/sultanubu' ...
info: Copying files from `/etc/skel' ...
New password:
BAD PASSWORD: The password fails the dictionary check - it is too simplistic/systematic
Retype new password:
passwd: password updated successfully
Changing the user information for sultanubu
Enter the new value, or press ENTER for the default
  Full Name []: Sulubu
   Room Number []: 2
   Work Phone []:
   Home Phone []:
    Other []:
Is the information correct? [Y/n] y
info: Adding new user `sultanubu' to supplemental / extra groups `users' ...
info: Adding user `sultanubu' to group `users' ...
stk88@client:~$
```

While the Ubuntu user (sultanubu) is used for accessing the website as a normal client. This helps simulate a real environment where the server and client are two different systems.

5.4 Setting Hostnames in /etc/hosts directory

- # echo "192.168.200.4 server.distinction.club" >> /etc/hosts
- # echo "192.168.200.80 server.distinction.club" >> /etc/hosts



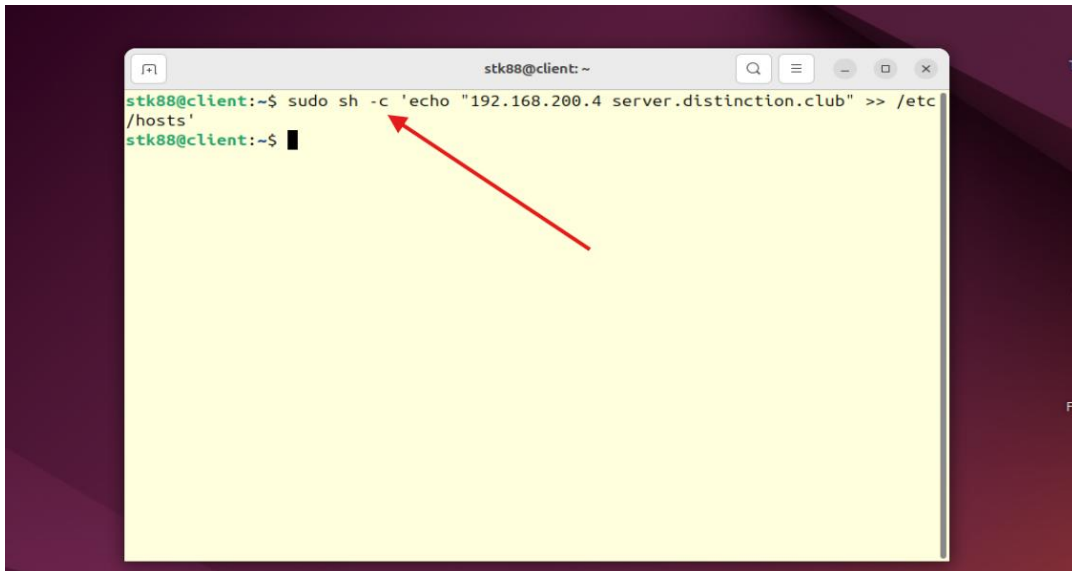
```
root@server:~# echo "192.168.200.4 server.distinction.club" >> /etc/hosts
root@server:~# echo "192.168.200.80 server.distinction.club" >> /etc/hosts
root@server:~#
```

Explanation:

/etc/hosts is like a mini address book inside the computer.

When you type server.distinction.club in a browser, the system will look in this file first to know it points to 192.168.200.4.

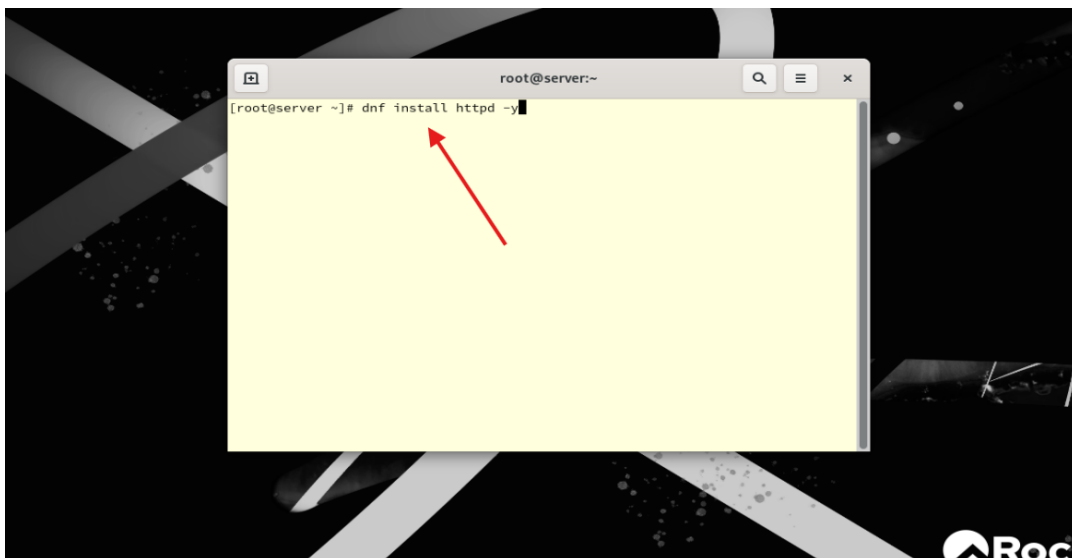
- `$ sudo sh -c 'echo "192.168.200.4 server.distinction.club" >> /etc/hosts'`



We make sure Ubuntu knows that **server.distinction.club** is the Rocky server using IP **192.168.200.4**. This is important so we can type the hostname instead of the IP in browser.

5.5 Installing Apache on Rocky

- `# dnf install httpd -y`



Installs Apache web server.

dnf is the package manager in Rocky (like an app store for Linux).

We're installing Apache (**httpd**) which is the program that runs websites.

```

root@server:~
Running scriptlet: httpd-2.4.62-4.el9_6.4.x86_64 11/11
Running scriptlet: mod_http2-2.0.26-4.el9_6.1.x86_64 11/11
Verifying : apr-util-bdb-1.6.1-23.el9.x86_64 1/11
Verifying : httpd-tools-2.4.62-4.el9_6.4.x86_64 2/11
Verifying : httpd-2.4.62-4.el9_6.4.x86_64 3/11
Verifying : apr-util-1.6.1-23.el9.x86_64 4/11
Verifying : rocky-logos-httpd-90.16-1.el9.noarch 5/11
Verifying : httpd-core-2.4.62-4.el9_6.4.x86_64 6/11
Verifying : httpd-filesystem-2.4.62-4.el9_6.4.noarch 7/11
Verifying : mod_lua-2.4.62-4.el9_6.4.x86_64 8/11
Verifying : mod_http2-2.0.26-4.el9_6.1.x86_64 9/11
Verifying : apr-util-openssl-1.6.1-23.el9.x86_64 10/11
Verifying : apr-1.7.0-12.el9_3.x86_64 11/11

Installed:
apr-1.7.0-12.el9_3.x86_64          apr-util-1.6.1-23.el9.x86_64
apr-util-bdb-1.6.1-23.el9.x86_64 apr-util-openssl-1.6.1-23.el9.x86_64
httpd-2.4.62-4.el9_6.4.x86_64    httpd-core-2.4.62-4.el9_6.4.x86_64
httpd-filesystem-2.4.62-4.el9_6.4.noarch httpd-tools-2.4.62-4.el9_6.4.x86_64
mod_http2-2.0.26-4.el9_6.1.x86_64 mod_lua-2.4.62-4.el9_6.4.x86_64
rocky-logos-httpd-90.16-1.el9.noarch

Complete!
[root@server ~]#

```

Then start and enable it:

- # systemctl enable httpd
- # systemctl start httpd
- # systemctl status httpd

```

root@server:~
[root@server ~]# systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@server ~]# systemctl start httpd
[root@server ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2025-11-11 21:24:08 +08; 15s ago
     Docs: man:httpd.service(8)
   Main PID: 3370 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served: 0"
   Tasks: 177 (limit: 10931)
   Memory: 29.8M
     CPU: 54ms
   CGroup: /system.slice/httpd.service
           └─3370 /usr/sbin/httpd -DFOREGROUND
             └─3371 /usr/sbin/httpd -DFOREGROUND
               └─3372 /usr/sbin/httpd -DFOREGROUND
                 └─3373 /usr/sbin/httpd -DFOREGROUND
                   └─3377 /usr/sbin/httpd -DFOREGROUND

Nov 11 21:24:08 server.distinction.club systemd[1]: Starting The Apache HTTP Server: listening on port 80.
Nov 11 21:24:08 server.distinction.club systemd[1]: Started The Apache HTTP Server: listening on port 80.
Nov 11 21:24:08 server.distinction.club httpd[3370]: Server configured, listening on port 80

```

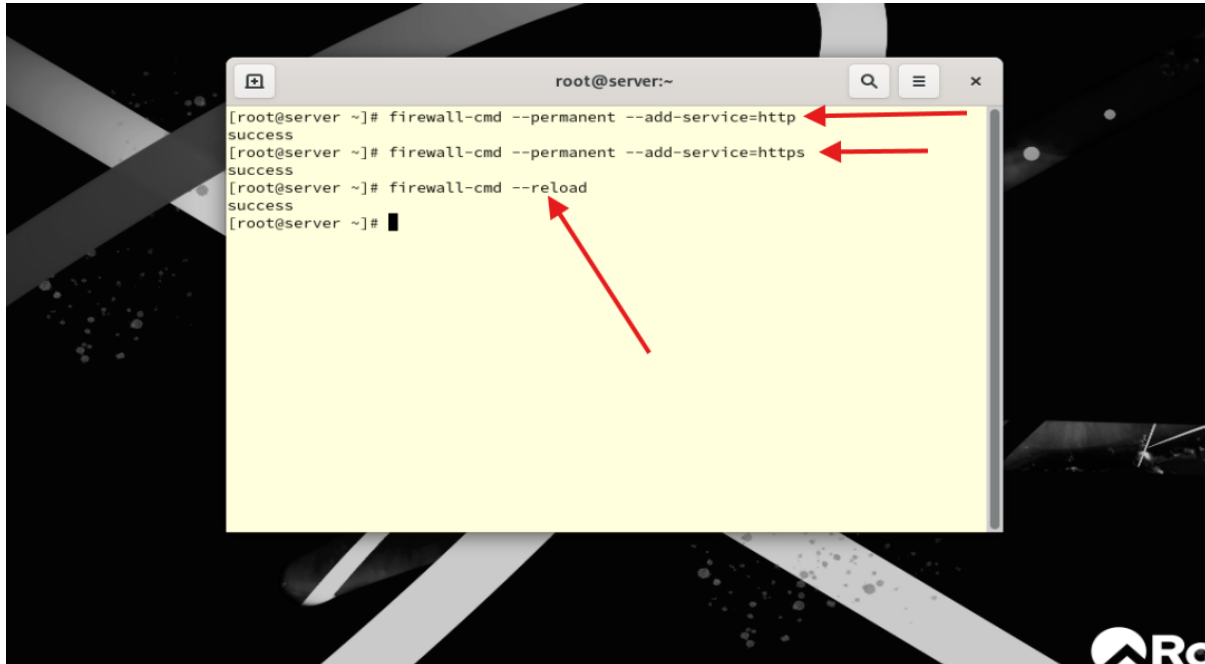
enable → makes Apache start automatically every time you boot.

start → starts the web server now.

status → checks if it's running. You should see "active (running)".

5.6 Allowing Web Traffic Through Firewall

- # firewall-cmd --permanent --add-service=http
- # firewall-cmd --permanent --add-service=https
- # firewall-cmd --reload

A terminal window titled 'root@server:~' with a yellow background. It displays three commands and their outputs: 'firewall-cmd --permanent --add-service=http' followed by 'success', 'firewall-cmd --permanent --add-service=https' followed by 'success', and 'firewall-cmd --reload' followed by 'success'. Three red arrows point from the right side of the terminal to the 'http', 'https', and 'reload' parts of the commands respectively.

```
root@server:~  
[root@server ~]# firewall-cmd --permanent --add-service=http  
success  
[root@server ~]# firewall-cmd --permanent --add-service=https  
success  
[root@server ~]# firewall-cmd --reload  
success  
[root@server ~]#
```

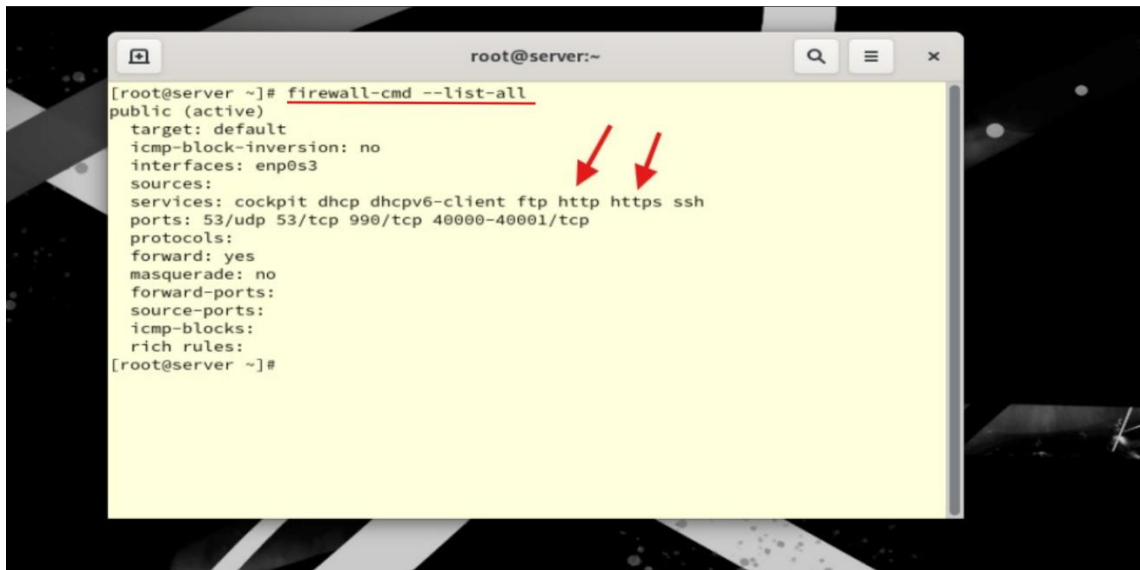
Note: If firewall is blocking the website, we need to open ports 80 (HTTP) and 443 (HTTPS).

1. The first two commands tell the firewall to allow traffic on HTTP and HTTPS.
2. “**--permanent**” means the rule will stay even after reboot.
3. **reload** applies the changes.

This step is like unlocking your door so others can visit your website.

5.7 Verifying Through Firewall service-lists

○ # firewall-cmd --list-all

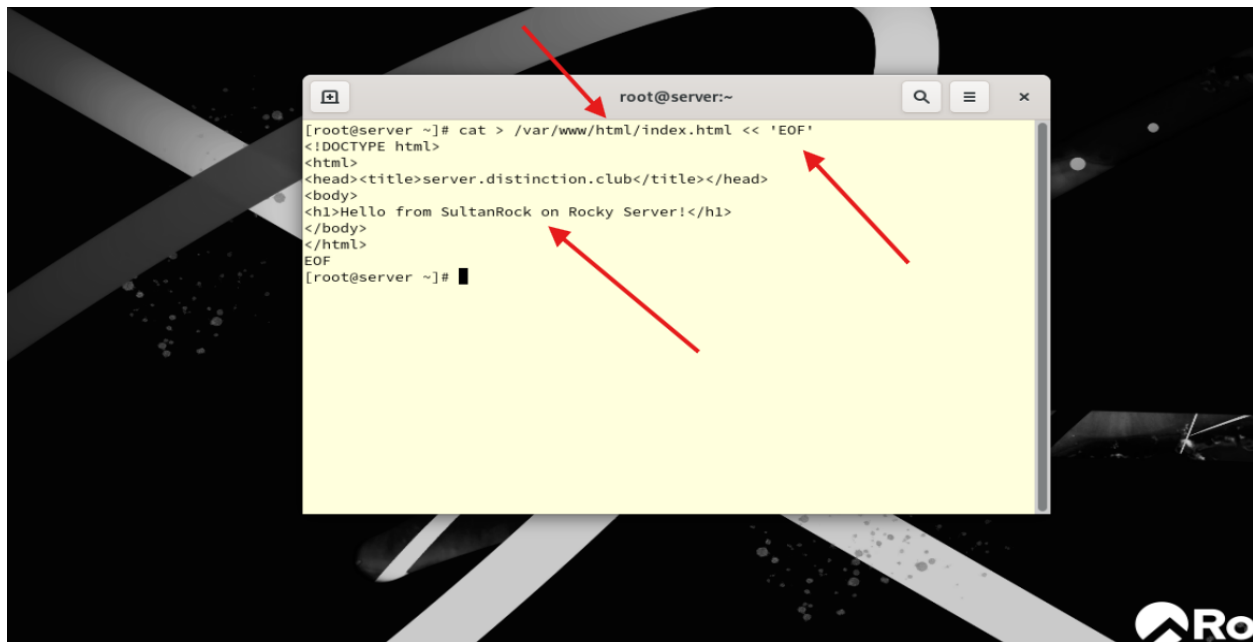


```
root@server:~  
[root@server ~]# firewall-cmd --list-all  
public (active)  
target: default  
icmp-block-inversion: no  
interfaces: enp0s3  
sources:  
services: cockpit dhcp dhcpv6-client ftp http https ssh  
ports: 53/udp 53/tcp 990/tcp 40000-40001/tcp  
protocols:  
forward: yes  
masquerade: no  
forward-ports:  
source-ports:  
icmp-blocks:  
rich rules:  
[root@server ~]#
```

The output above confirms that both HTTP and HTTPS are open.

5.8 Creating a Test Webpage

○ # cat > /var/www/html/index.html << 'EOF'



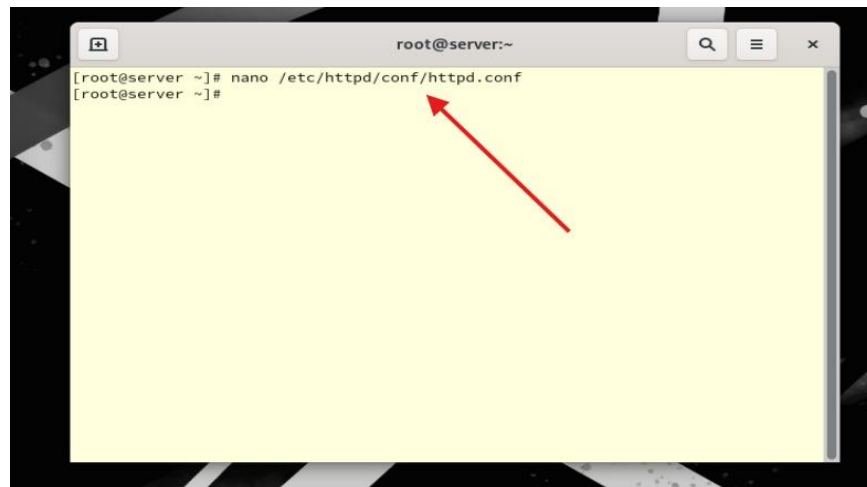
```
root@server:~  
[root@server ~]# cat > /var/www/html/index.html << 'EOF'  
<!DOCTYPE html>  
<html>  
<head><title>server.distinction.club</title></head>  
<body>  
<h1>Hello from SultanRock on Rocky Server!</h1>  
</body>  
</html>  
EOF  
[root@server ~]#
```

Creating a basic `index.html` allows us to confirm that Apache is serving pages correctly before we start adding advanced features like HTTPS or authentication. If the simple page works, it means Apache is installed correctly and the network connection between Rocky and Ubuntu is working.

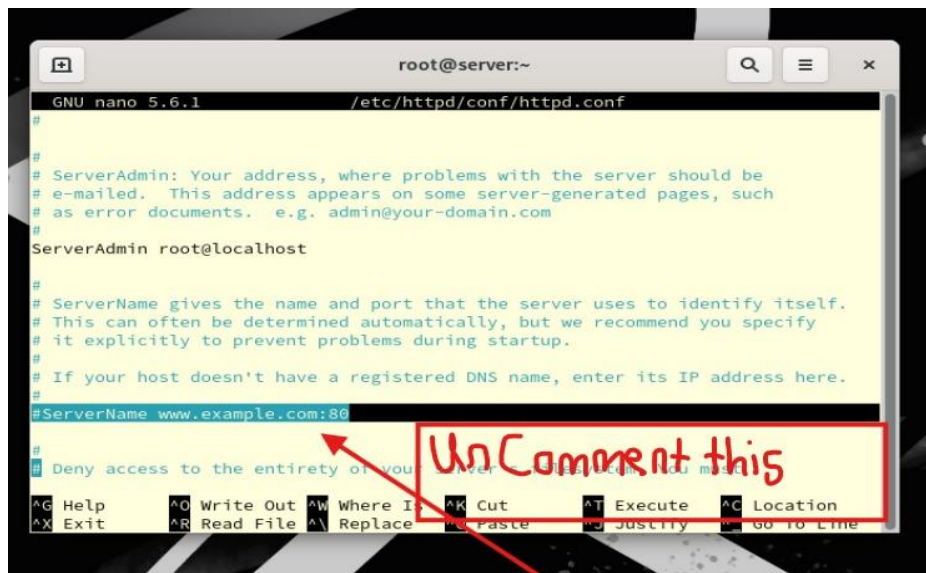
5.9 Setting Apache Server Name

Open the file with nano editor

o `# nano /etc/httpd/conf/httpd.conf`



Scroll down until you find this line, then Remove the ‘#’ and change it to your hostname.



Note: The # means the line is disabled. Removing it activates the ServerName directive.



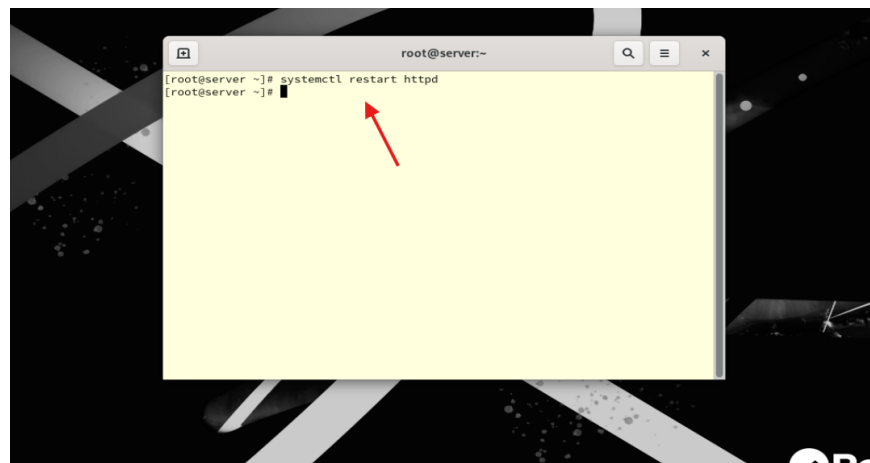
ServerName tells Apache what hostname the server answers to.

Without this, Apache might show a warning like “Could not reliably determine the server's fully qualified domain name.”

Setting the hostname to **server.distinction.club:80** means:

“When someone visits server.distinction.club on port 80 (HTTP), this Apache instance should handle it.” Then we simply restart Apache:

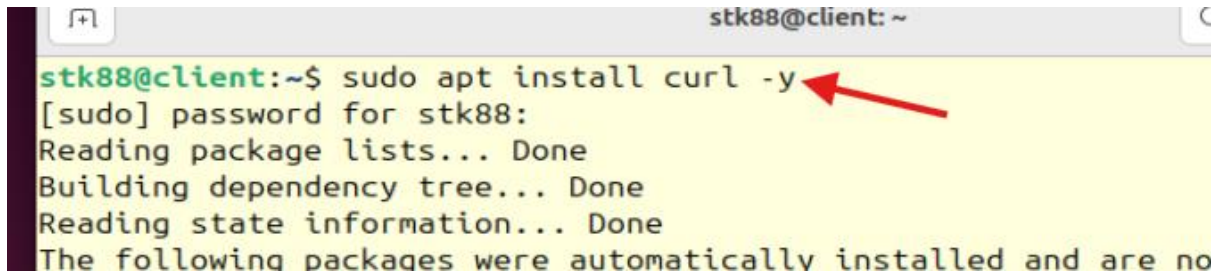
○ # systemctl restart httpd



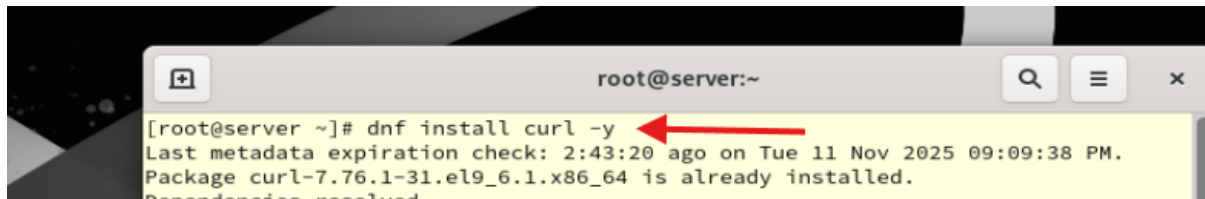
5.10 Installing Curl on Rocky and Ubuntu

Note: If you don't have Curl on your Rocky or your Ubuntu, you can download them using this command:

- `$ sudo apt install curl -y`
- `# dnf install curl -y`



```
stk88@client: ~  
stk88@client:~$ sudo apt install curl -y  
[sudo] password for stk88:  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following packages were automatically installed and are no
```

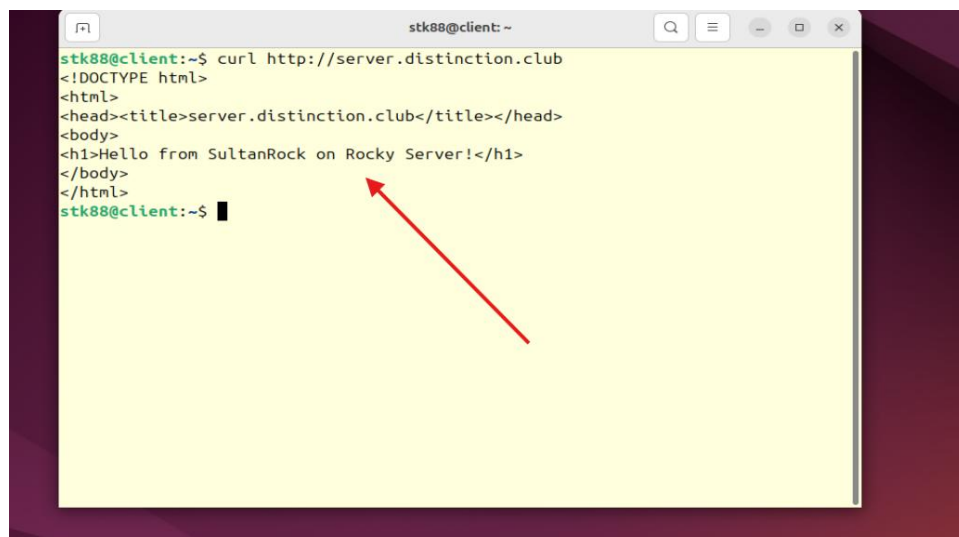


```
root@server:~  
[root@server ~]# dnf install curl -y  
Last metadata expiration check: 2:43:20 ago on Tue 11 Nov 2025 09:09:38 PM.  
Package curl-7.76.1-31.el9_6.1.x86_64 is already installed.  
Dependencies resolved
```

5.11 Testing HTTP Connection

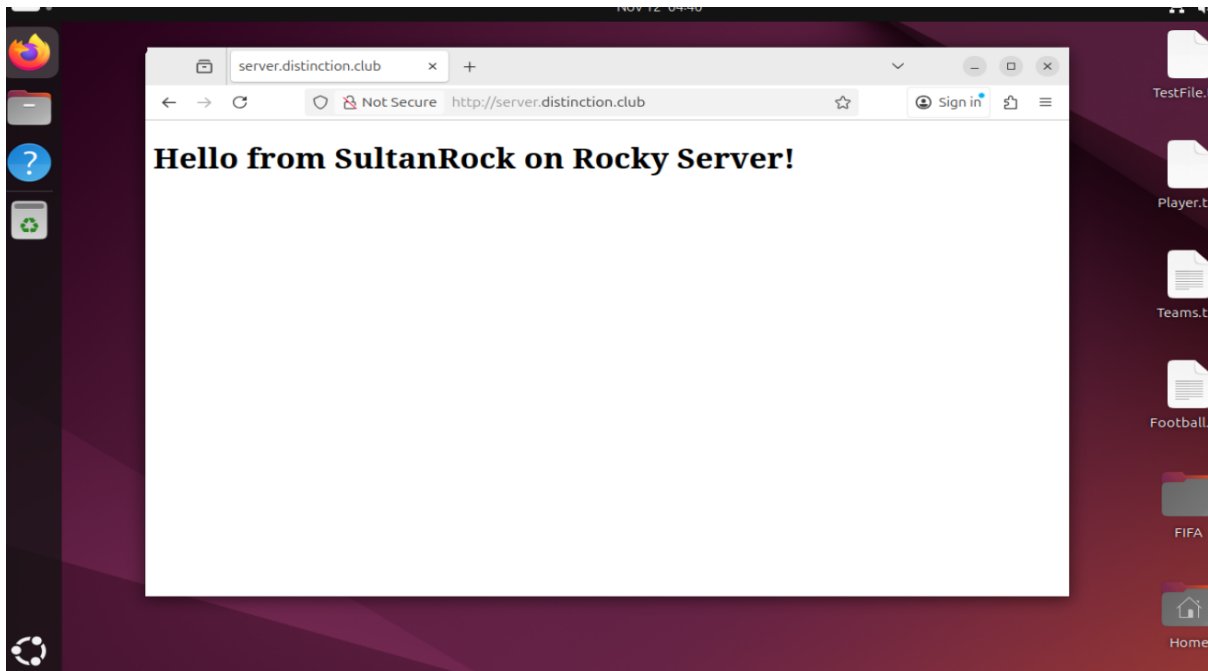
5.11.1 Using Curl on Rocky Terminal

- `$ curl http://server.distinction.club`



```
stk88@client:~$ curl http://server.distinction.club  
<!DOCTYPE html>  
<html>  
<head><title>server.distinction.club</title></head>  
<body>  
<h1>Hello from SultanRock on Rocky Server!</h1>  
</body>  
</html>  
stk88@client:~$
```

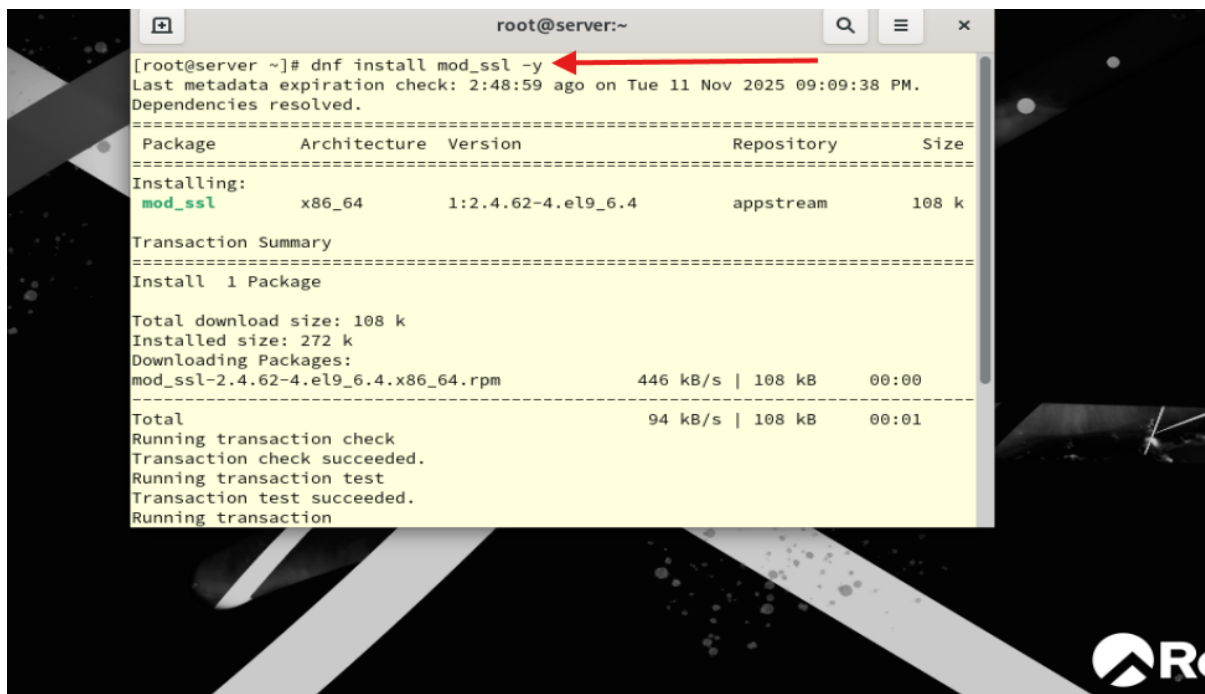
5.11.2 Using the Mozilla Firefox Browser



Let's go, its Working!!!

5.12 Adding SSL / TLS on Rocky

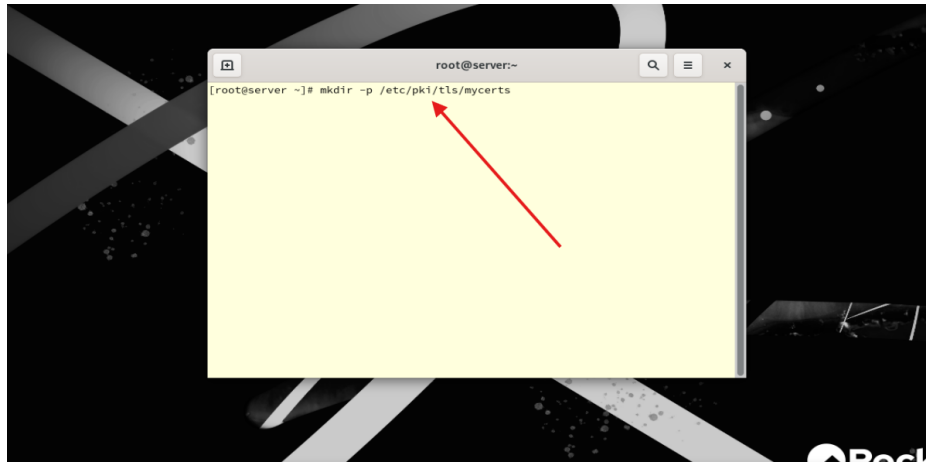
o # dnf install mod_ssl -y



This adds SSL support to Apache — it's what allows HTTPS (secure connection).

When SSL is installed, it also creates a file `/etc/httpd/conf.d/ssl.conf`

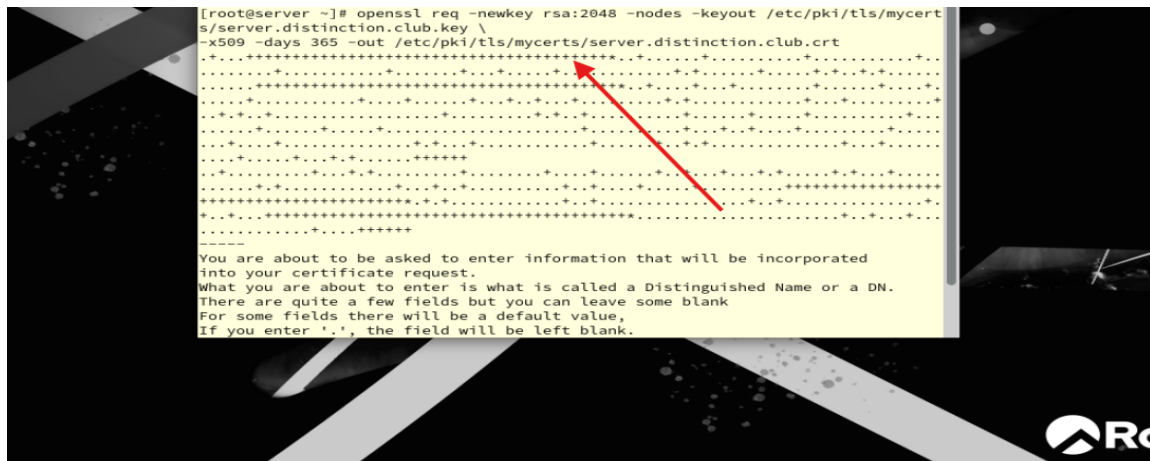
- `# mkdir -p /etc/pki/tls/mycerts`



The SSL certificate and key files are stored in `/etc/pki/tls/mycerts/`, which is the standard directory for Apache's security files. The main website files are stored under `/var/www/html/`. Apache's configuration file `/etc/httpd/conf.d/ssl.conf` links these two directories together — the certificate provides HTTPS security, while the HTML files provide the website content.

Make sure to type it right:

- `# openssl req -newkey rsa:2048 -nodes -keyout /etc/pki/tls/mycerts/server.distinction.club.key \`
`-x509 -days 365 -out /etc/pki/tls/mycerts/server.distinction.club.crt`



This creates 2 files in `/etc/pki/tls/mycerts/`:

- The key (private file)
- The certificate (public file)

Then,

Write your own hostname:

○ `server.distinction.club`

Explanation:

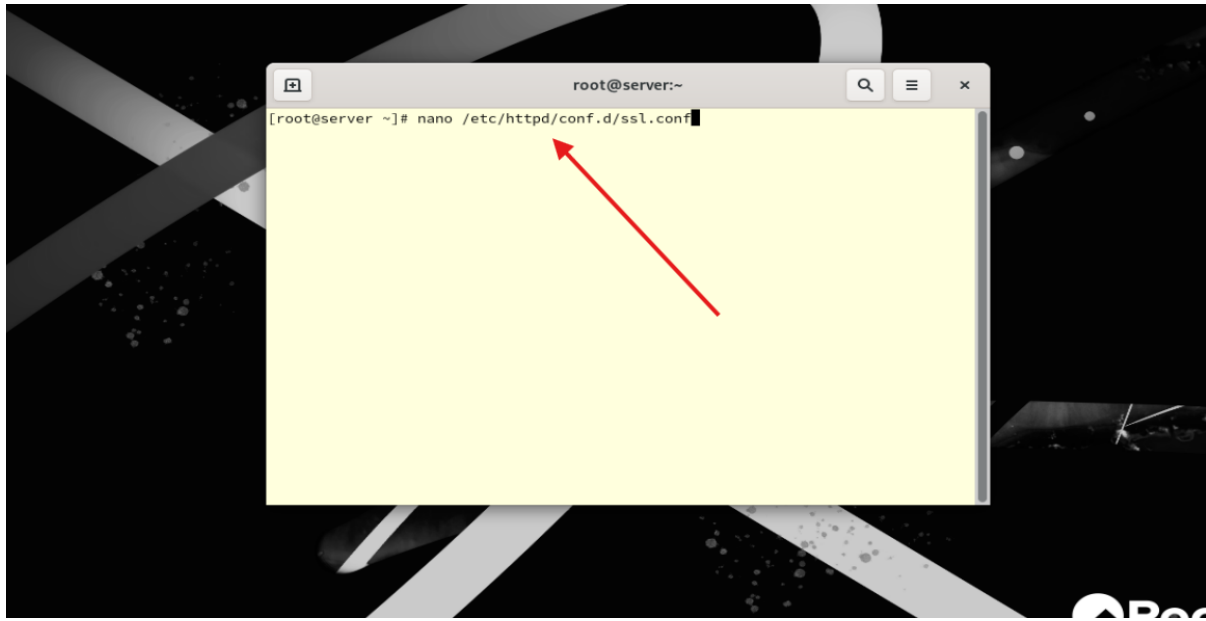
This makes your own mini “security certificate” so you can use HTTPS.

It’s self-signed, which means it’s not from a company like Google or DigiCert ,but good enough for testing.

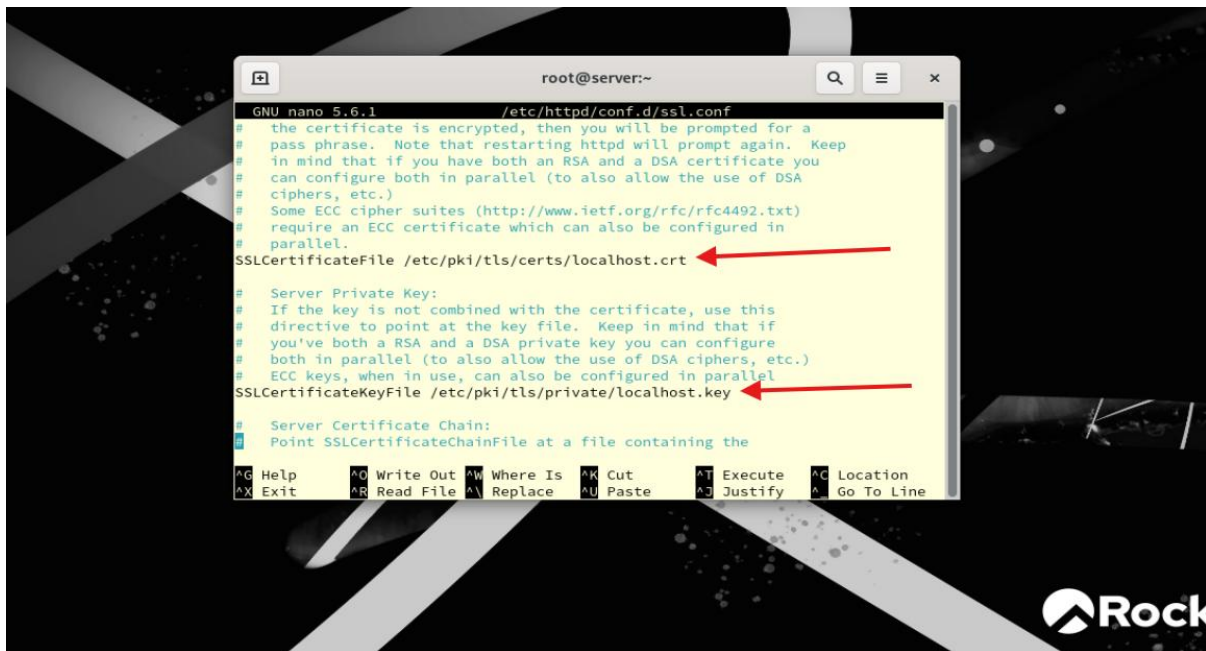
You’ll get browser warnings, but it still works.

5.13 Edit SSL Config to use it

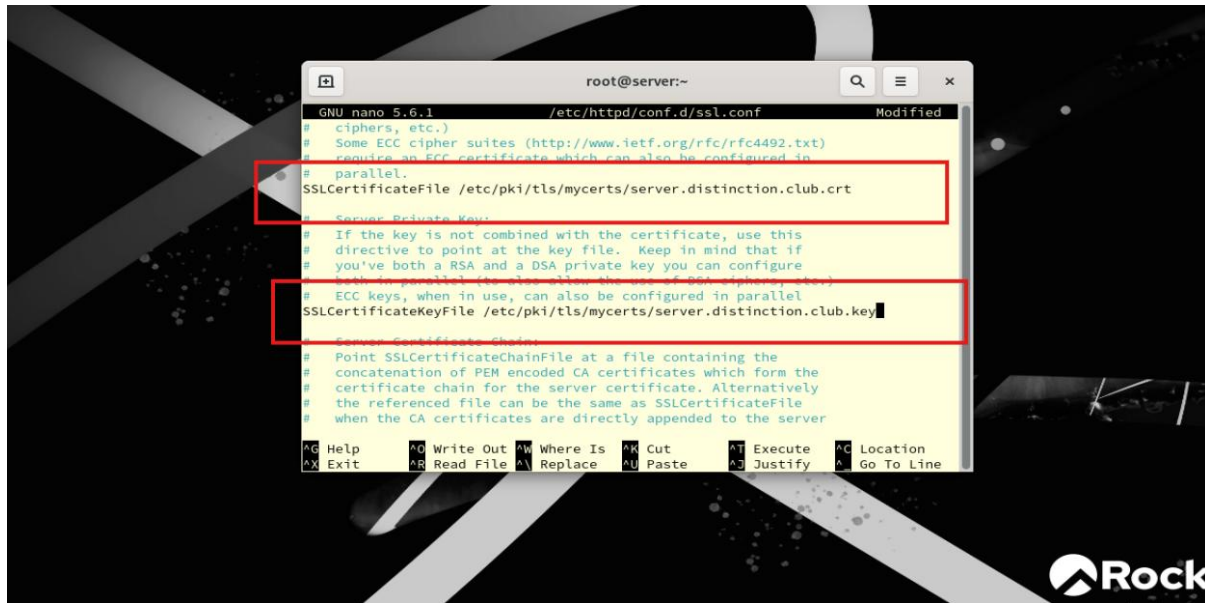
- o # nano /etc/httpd/conf.d/ssl.conf



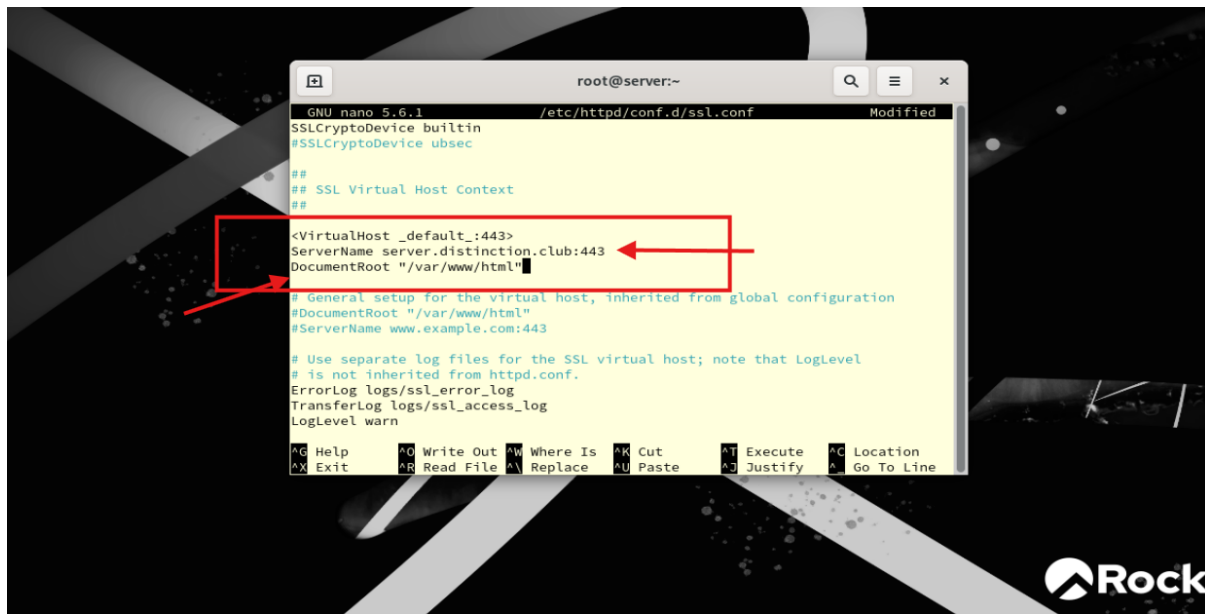
Open the file and find these:



Replace them with the paths that match your hostname:



Also make sure this line exists inside <VirtualHost *:443>:(Very Important)



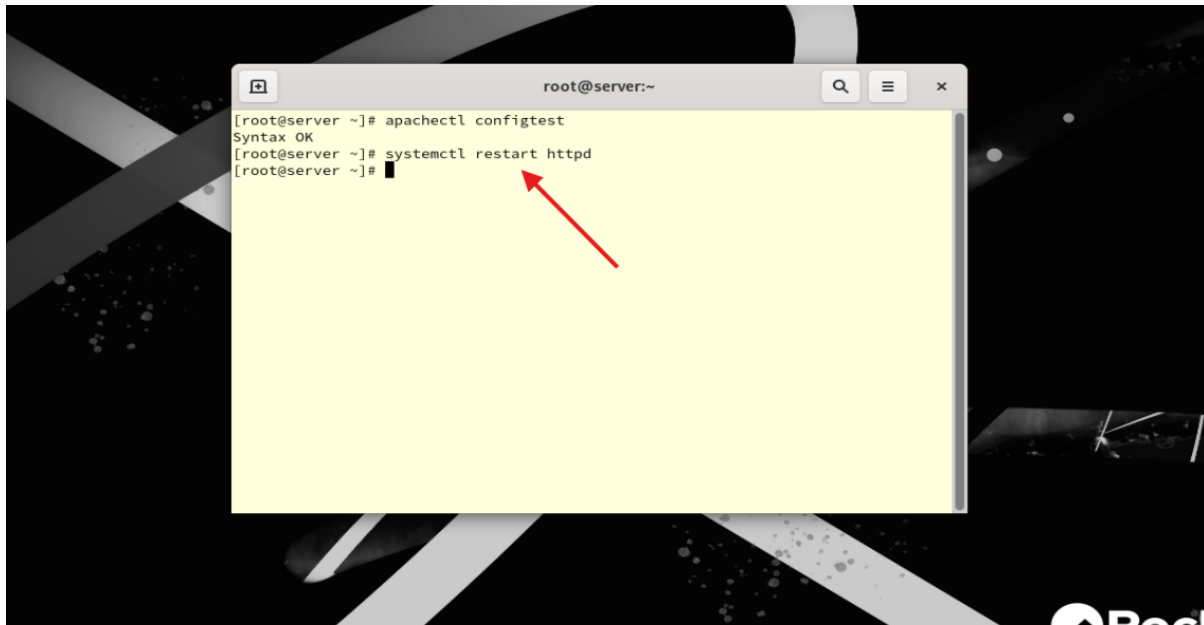
Here we are telling Apache where to find the new SSL certificate and key.
443 is the port number for HTTPS websites.

The SSL configuration file controls how Apache handles secure connections on port 443. Apache cannot use HTTPS unless it knows exactly where your certificate and private key are stored. This is why we edit the SSLCertificateFile and SSLCertificateKeyFile paths. If these paths do not match the files you created, HTTPS will fail. Adding the <VirtualHost *:443> block ensures Apache listens for secure connections on port 443.

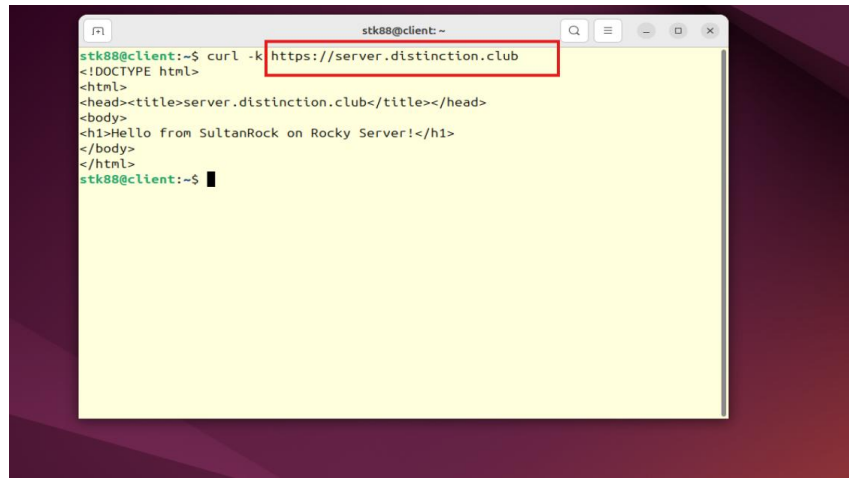
5.14 Testing HTTPS from Ubuntu

Note: Restart it from Rocky and test it, like this:

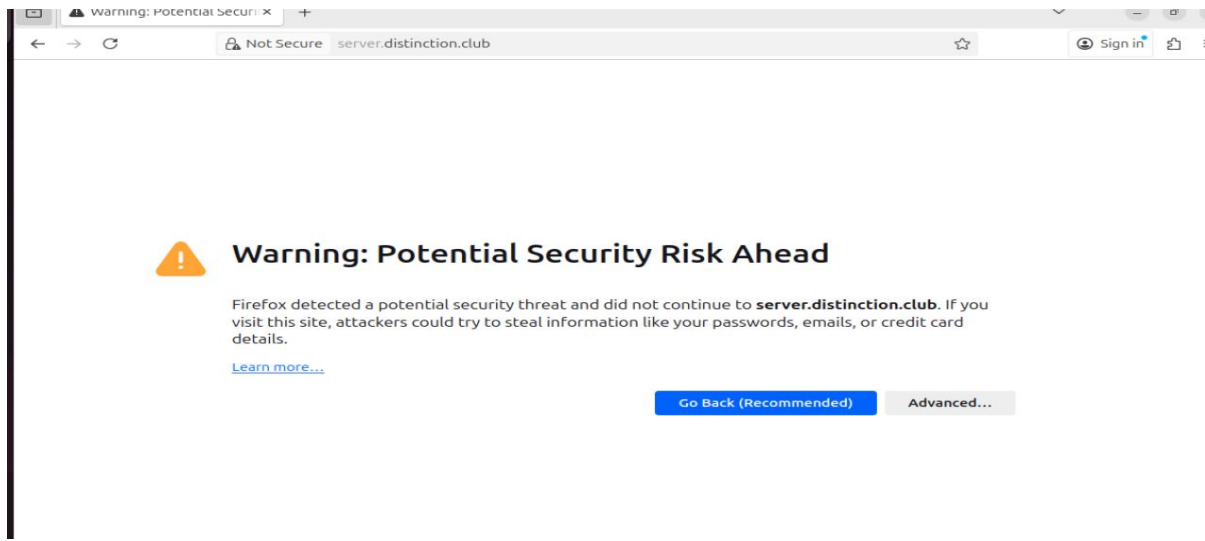
- # apachectl configtest
- # systemctl restart httpd

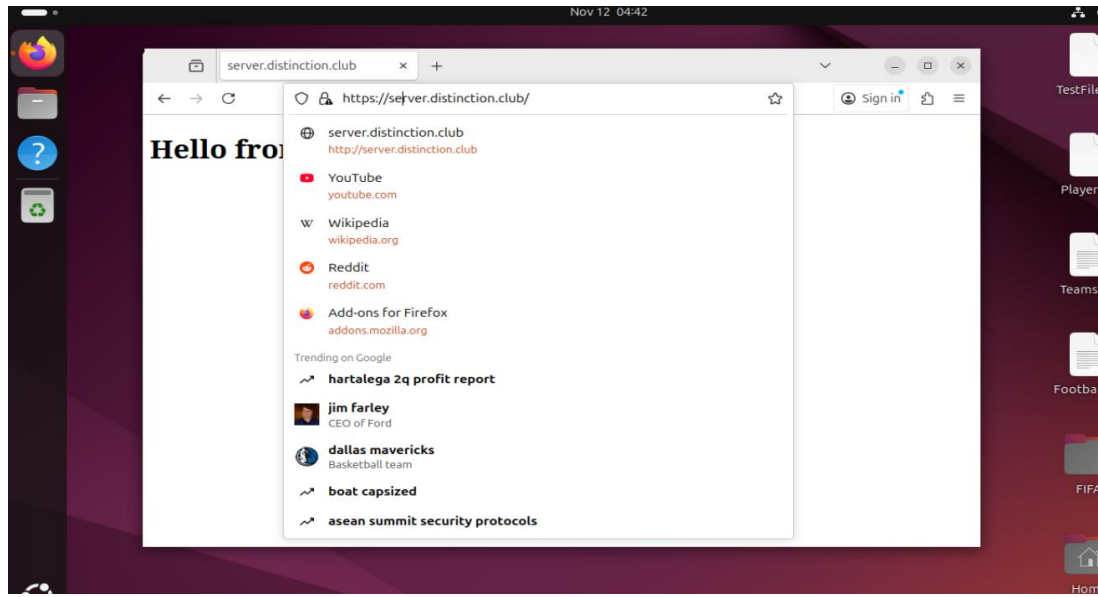
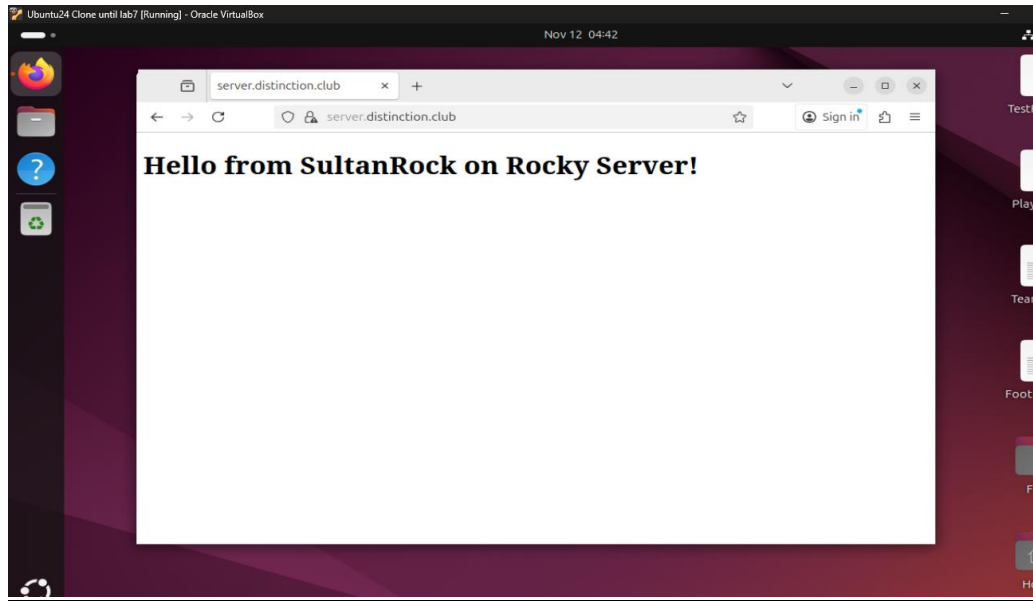


Let's TRY:



```
stk88@client:~$ curl -k https://server.distinction.club
<!DOCTYPE html>
<html>
<head><title>server.distinction.club</title></head>
<body>
<h1>Hello from SultanRock on Rocky Server!</h1>
</body>
</html>
stk88@client:~$
```





It's working lets goo!!!

Important Note:

Even though the browser doesn't "trust" the certificate (because it's not from a big company like google), the connection is still encrypted.

So everything between your Ubuntu browser and the Rocky server is safely protected:)

5.15 Personal Reflection

In this project, I successfully configured an Apache web server on Rocky Linux with HTTPS enabled using a self-signed SSL certificate. The Ubuntu client accessed the secure site successfully. This setup demonstrates how to implement basic encryption and hostname configuration in a LAN environment.

CHAPTER 6: ADDITIONAL SERVICE CONFIGURATION

6.1 Samba File Server

6.1.1 Introduction to Samba



Samba is an open-source implementation of the SMB/CIFS protocol which permits the Linux systems to share files and directories over a network. It is commonly used in enterprise setups to facilitate smoother sharing of files between Linux, Windows and macOS systems, In this additional feature, Samba was installed and configured in the Rocky Linux server to have a shared centralized directory that is shared and accessed by the Ubuntu client.

6.1.2 Installation of Samba

o `sudo dnf install -y samba samba-common samba-client`

Figure 25 Installing Samba File Server Packages

A screenshot of a terminal window on a Rocky Linux 9 system. The terminal shows the command `sudo dnf install -y samba samba-common samba-client` being executed. The output displays the progress of installing the packages from various repositories (BaseOS, AppStream, Extras) and confirms that the packages are already installed. The terminal output is as follows:

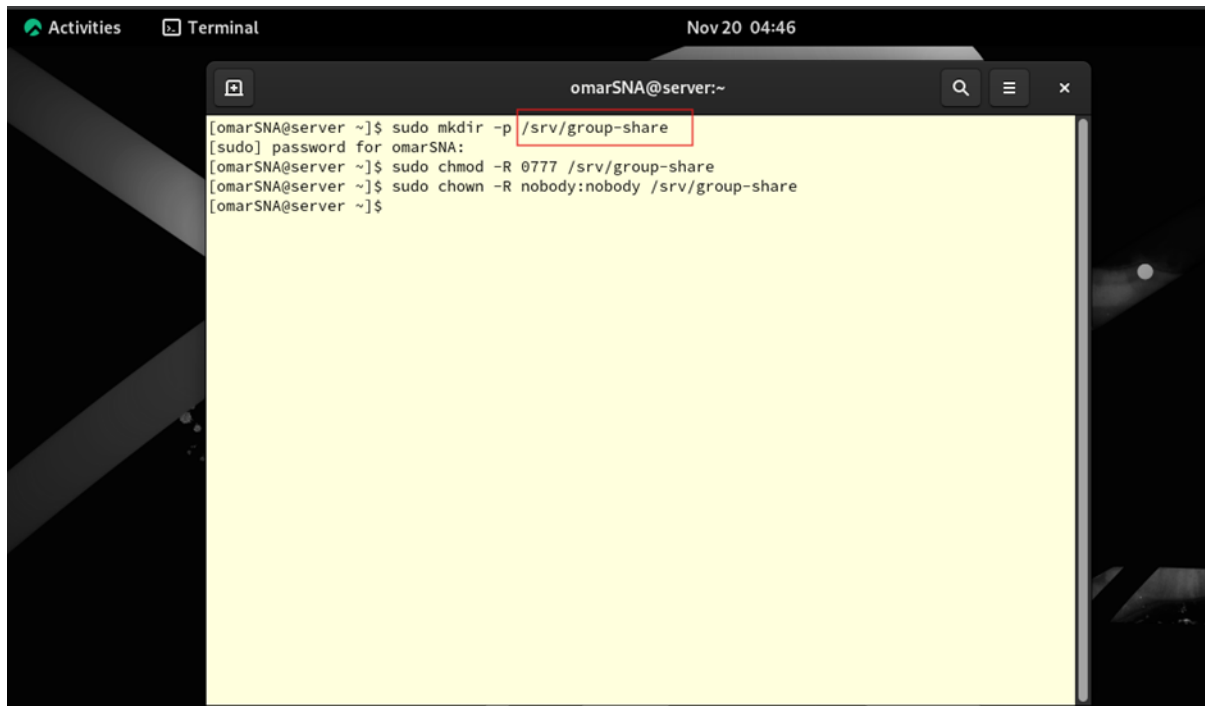
```
[omarSNA@server ~]$ sudo dnf install -y samba samba-common samba-client
[sudo] password for omarSNA:
Rocky Linux 9 - BaseOS                               1.3 kB/s | 4.1 kB | 00:03
Rocky Linux 9 - BaseOS                               2.3 MB/s | 2.5 MB | 00:01
Rocky Linux 9 - AppStream                             4.6 kB/s | 4.5 kB | 00:00
Rocky Linux 9 - Extras                               2.0 kB/s | 2.9 kB | 00:01
Package samba-4.21.3-14.el9_6.x86_64 is already installed.
Package samba-common-4.21.3-14.el9_6.noarch is already installed.
Package samba-client-4.21.3-14.el9_6.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[omarSNA@server ~]$
```

We install the required samba packages on our rocky server, These packages offer the smb and nmb daemons necessary to offer the SMB file shares, which the network client uses to discover the server.

6.1.3 Creating Shared Directory

- o `sudo mkdir -p /srv/group-share`

Figure 26 changing permissions & ownership of 'group-share' file under srv directory

A terminal window titled 'omarSNA@server:~' with a search icon, menu icon, and close icon. The terminal shows the following commands and output:

```
[omarSNA@server ~]$ sudo mkdir -p /srv/group-share
[sudo] password for omarSNA:
[omarSNA@server ~]$ sudo chmod -R 0777 /srv/group-share
[omarSNA@server ~]$ sudo chown -R nobody:nobody /srv/group-share
[omarSNA@server ~]$
```

The path `/srv/group-share` in the first command is highlighted with a red box.

A directory named `/srv/group-share` was created as the shared folder, The permissions and ownership were set in such a way so that the folder can be accessed and modified under the use of Samba.

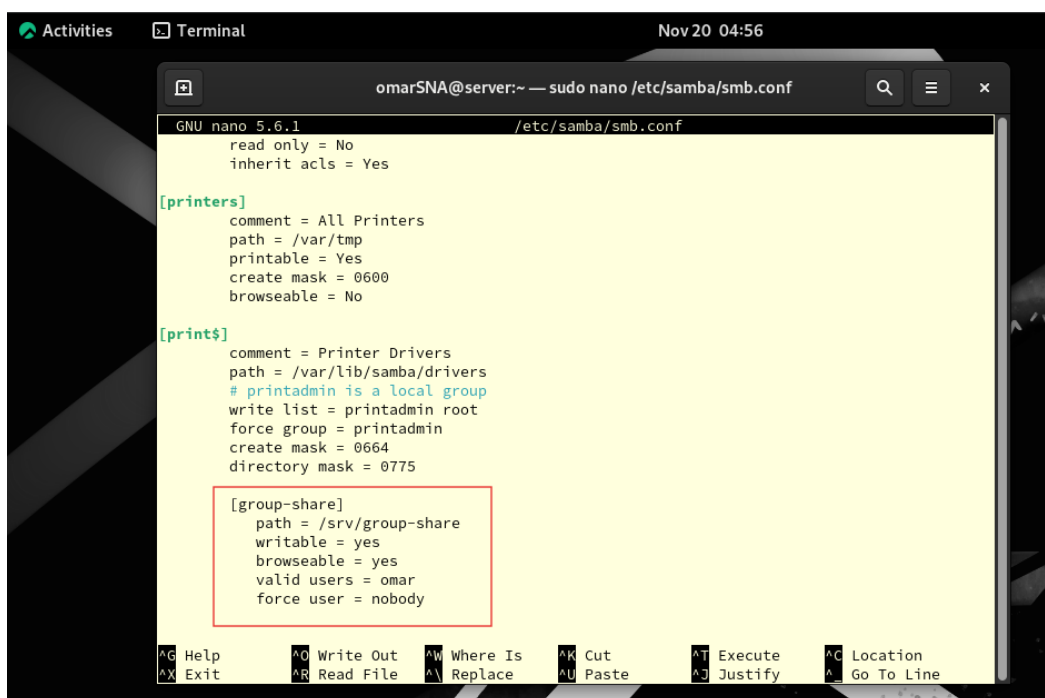
6.1.4 Configuring Samba Shared Directory

o `sudo nano /etc/samba/smb.conf`

Figure 27 command to open smb.conf configuration file



Figure 28 contents of smb.conf file

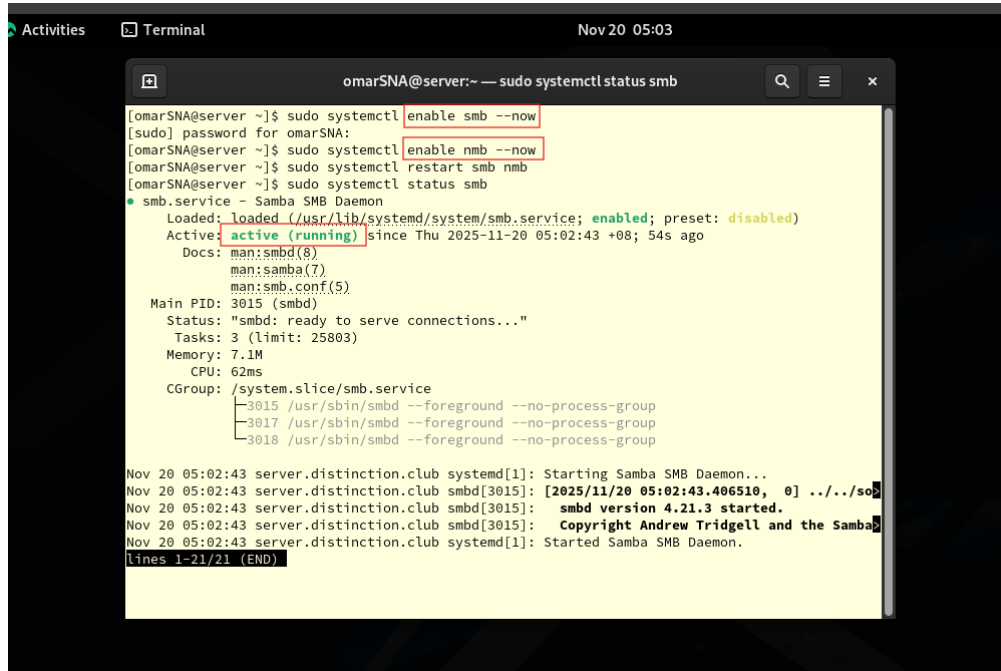


A new Samba share called group-share was specified in the configuration file. For now, the user Omar is the only user who can authenticate, and the share is writable which means that the client can upload and download files.

6.1.5 Restarting Samba File services

- `sudo systemctl enable smb --now`
- `sudo systemctl enable nmb --now`

Figure 29 Samba SMB Daemon in Active (running) status



```

[omarSNA@server ~]$ sudo systemctl enable smb --now
[sudo] password for omarSNA:
[omarSNA@server ~]$ sudo systemctl enable nmb --now
[omarSNA@server ~]$ sudo systemctl restart smb nmb
[omarSNA@server ~]$ sudo systemctl status smb
● smb.service - Samba SMB Daemon
   Loaded: loaded (/usr/lib/systemd/system/smb.service; enabled; preset: disabled)
   Active: active (running) since Thu 2025-11-20 05:02:43 +08; 54s ago
     Docs: man:smbd(8)
           man:samba(7)
           man:smb.conf(5)
    Main PID: 3015 (smbd)
      Status: "smbd: ready to serve connections..."
        Tasks: 3 (limit: 25803)
       Memory: 7.1M
          CPU: 62ms
    CGroup: /system.slice/smb.service
            └─3015 /usr/sbin/smbd --foreground --no-process-group
              └─3017 /usr/sbin/smbd --foreground --no-process-group
                └─3018 /usr/sbin/smbd --foreground --no-process-group

Nov 20 05:02:43 server.distinction.club systemd[1]: Starting Samba SMB Daemon...
Nov 20 05:02:43 server.distinction.club smbd[3015]: [2025/11/20 05:02:43.406510, 0] ../../so
Nov 20 05:02:43 server.distinction.club smbd[3015]: smbd version 4.21.3 started.
Nov 20 05:02:43 server.distinction.club smbd[3015]: Copyright Andrew Tridgell and the Samba
Nov 20 05:02:43 server.distinction.club systemd[1]: Started Samba SMB Daemon.
lines 1-21/21 (END)

```

Both Samba services were enabled and started. Restarting them ensures that the new configuration that we just did is applied successfully.

6.1.6 Allowing Samba through Firewall

- `sudo firewall-cmd --permanent --add-service=samba`
- `sudo firewall-cmd --reload`

Figure 30 Commands to whitelist samba file service in firewall



The screenshot shows a terminal window titled "Terminal" with the date and time "Nov 20 05:19". The user is logged in as "omarSNA@server". The terminal displays the following commands and output:

```
[omarSNA@server ~]$ sudo firewall-cmd --permanent --add-service=samba
warning: ALREADY_ENABLED: samba
success
[omarSNA@server ~]$ sudo firewall-cmd --reload
success
[omarSNA@server ~]$
```

Samba needs network connection via ports 137, 138, 139 and 445, The firewall rules were updated to allow connections from the Ubuntu client.

6.1.7 Connecting to Samba share

We first open files application on Rocky, then type the following path directory:

○ **smb://192.168.200.4/group-share**

Figure 31 Graphical User Interface (GUI) of group-share directory contents

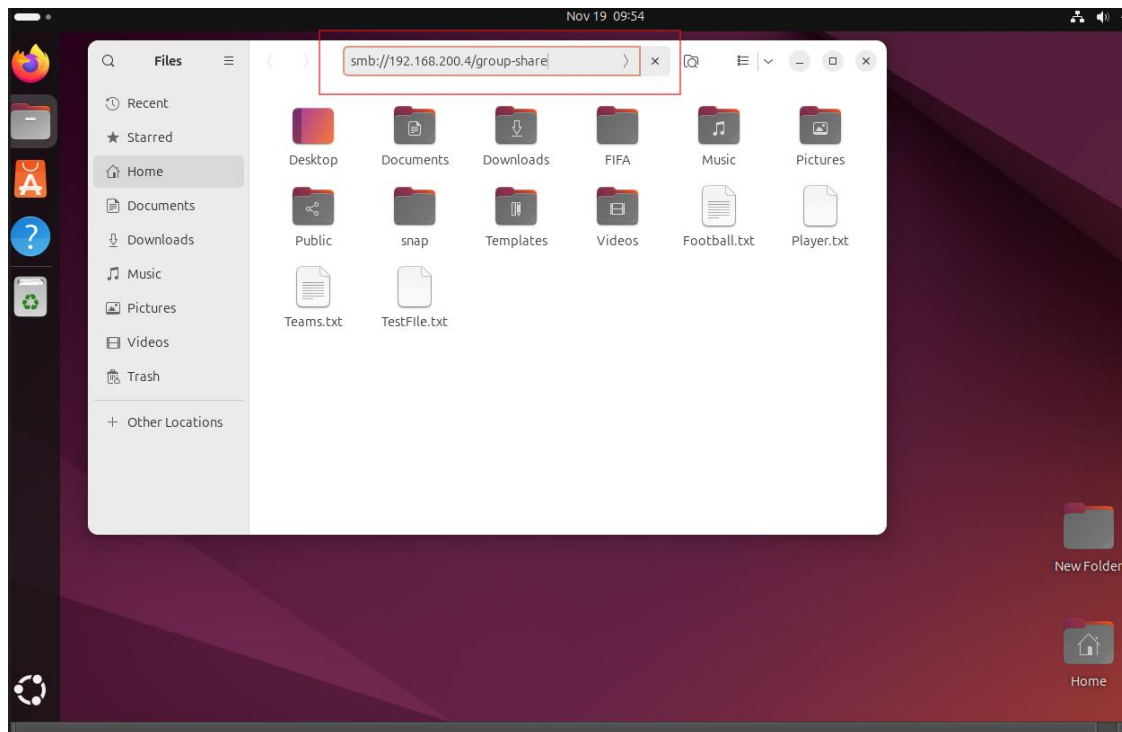
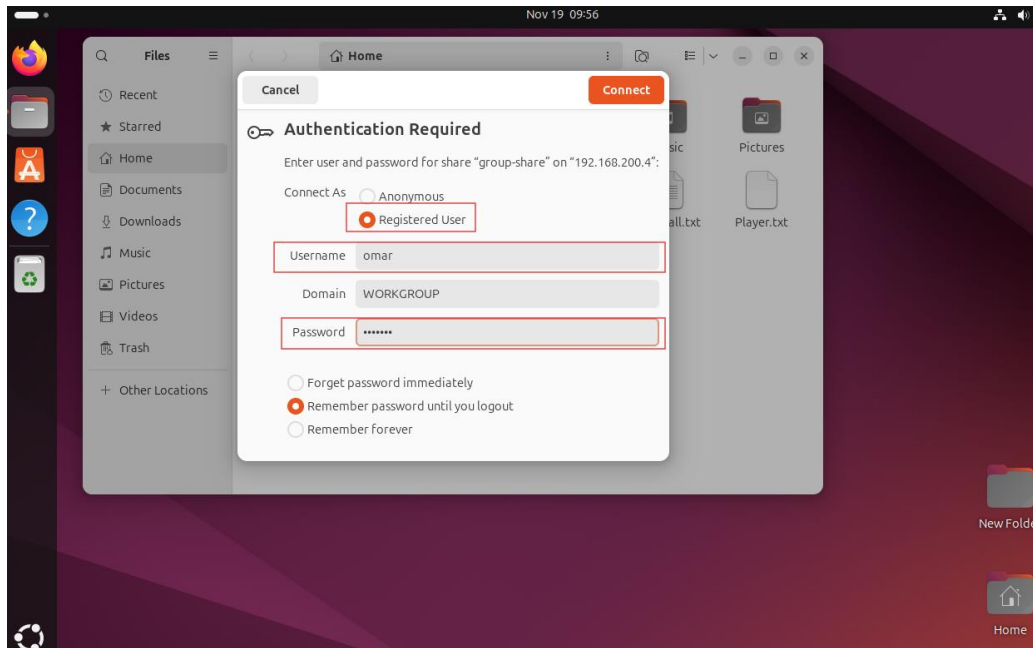
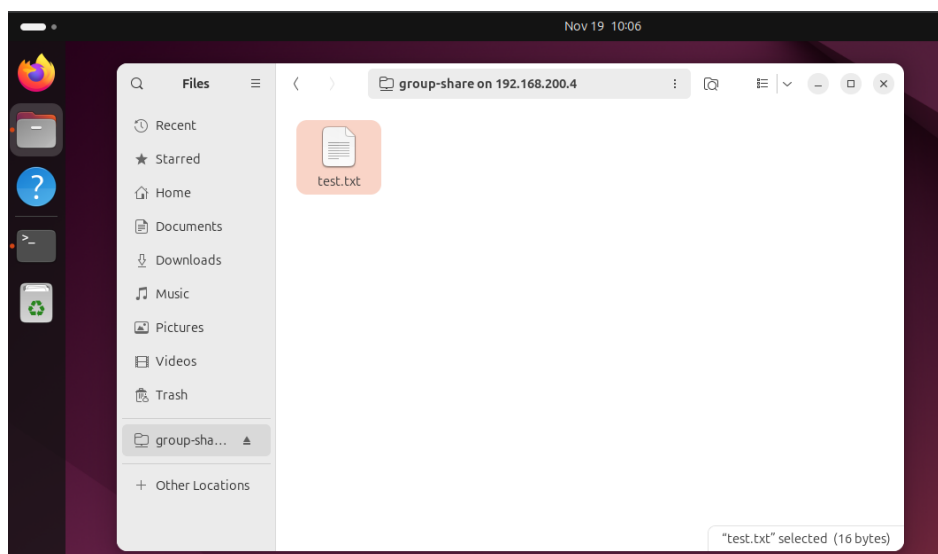
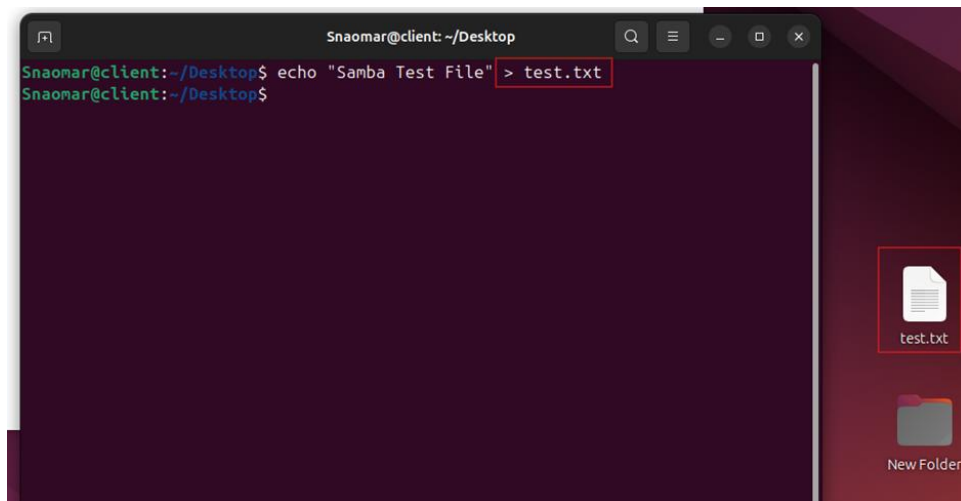


Figure 32 Connecting to samba share using samba user credentials

The Ubuntu client was able to connect to Samba share using Samba user credentials. This proves that authentication and access control are working.

6.1.8 Uploading a file from Ubuntu Client

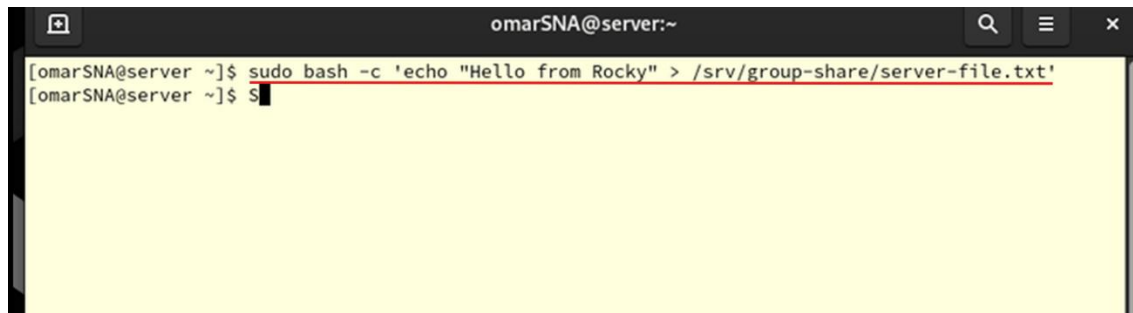
- `echo "Samba Test File" > test.txt`



We create a test file in the client ubuntu and successfully uploaded to the shared folder Which verifies that the Samba share is writable and working.

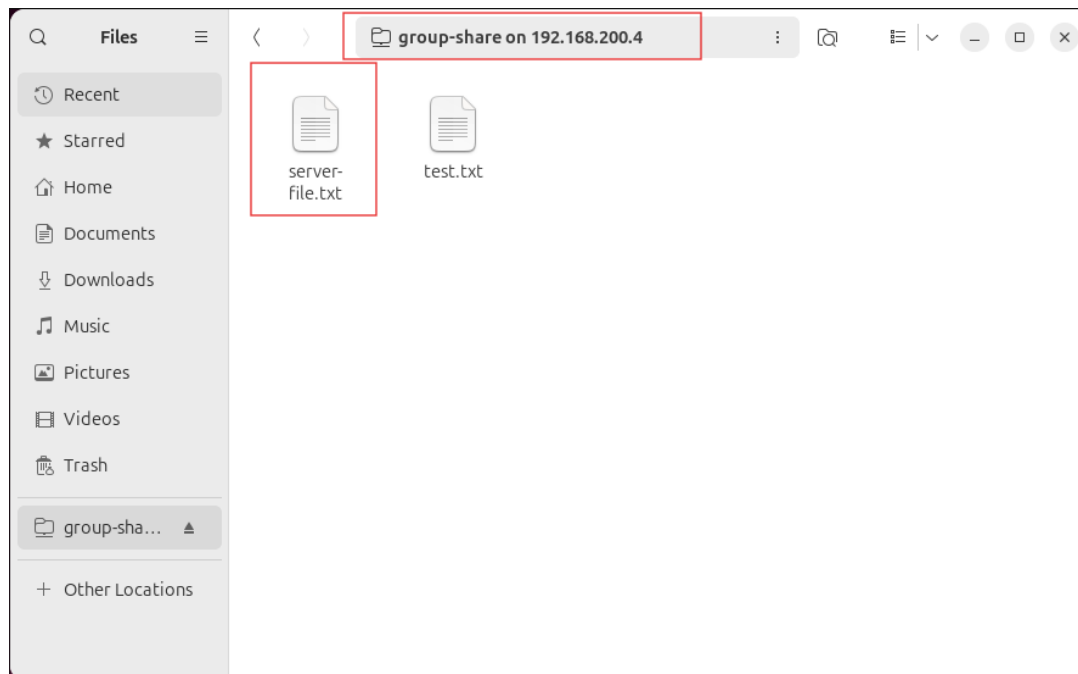
6.1.9 Downloading a file from Samba

- o `sudo bash -c 'echo "Hello from Rocky" > /srv/group-share/server-file.txt'`

A terminal window titled 'omarSNA@server:~' with a yellow background. The command `sudo bash -c 'echo "Hello from Rocky" > /srv/group-share/server-file.txt'` is entered and executed. The prompt changes to `[omarSNA@server ~]$` after the command runs.

```
omarSNA@server:~  
[omarSNA@server ~]$ sudo bash -c 'echo "Hello from Rocky" > /srv/group-share/server-file.txt'  
[omarSNA@server ~]$
```

Then we refresh the shared folder on Ubuntu and download the file.



Downloading the file back from the server confirms that the two-way communication and full functionality of the Samba server.

6.1.10 Personal Reflection

My experience of working on the Samba file server taught me how Linux systems use shared folders, access, and permissions of users. I managed to learn the difference between Linux users and Samba users, testing the files uploads and downloads from the Ubuntu client helped me to understand how the SMB feature works in practice. This feature provided me with practical knowledge on a real-world file-sharing service and enhanced my capabilities in handling other network services.

6.2 MariaDB

6.2.1 Introduction to MariaDB

Figure 33 MariaDB logo



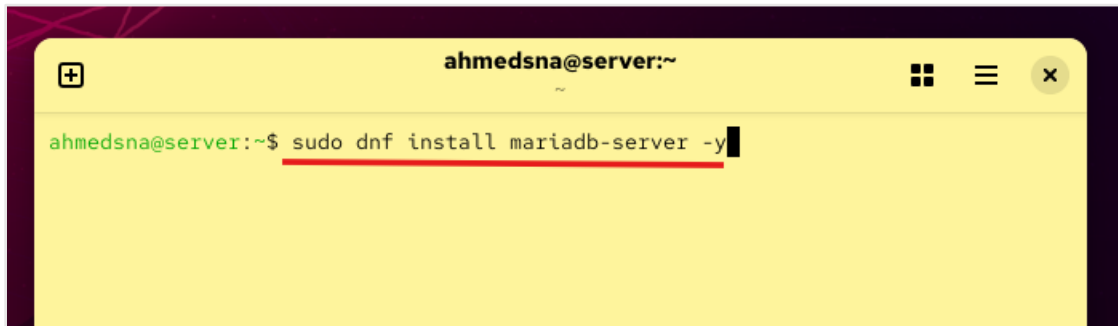
MariaDB is an open-source, high-performance relational database management system (RDBMS) that started out as a fork of MySQL. While introducing additional enhancements and optimizations, it has full compatibility with MySQL. It is Known for its reliability, scalability, and strong security features and is commonly used in web applications while having a wide range of software environments.

Key features of MariaDB include:

- Optimized for speed and efficiency.
- Supports several storage engines.
- Advanced Features like JSON data type, spatial extensions, and encryption.
- Provides security features to protect sensitive data.
- Community Driven.

6.2.2 MariaDB Installation

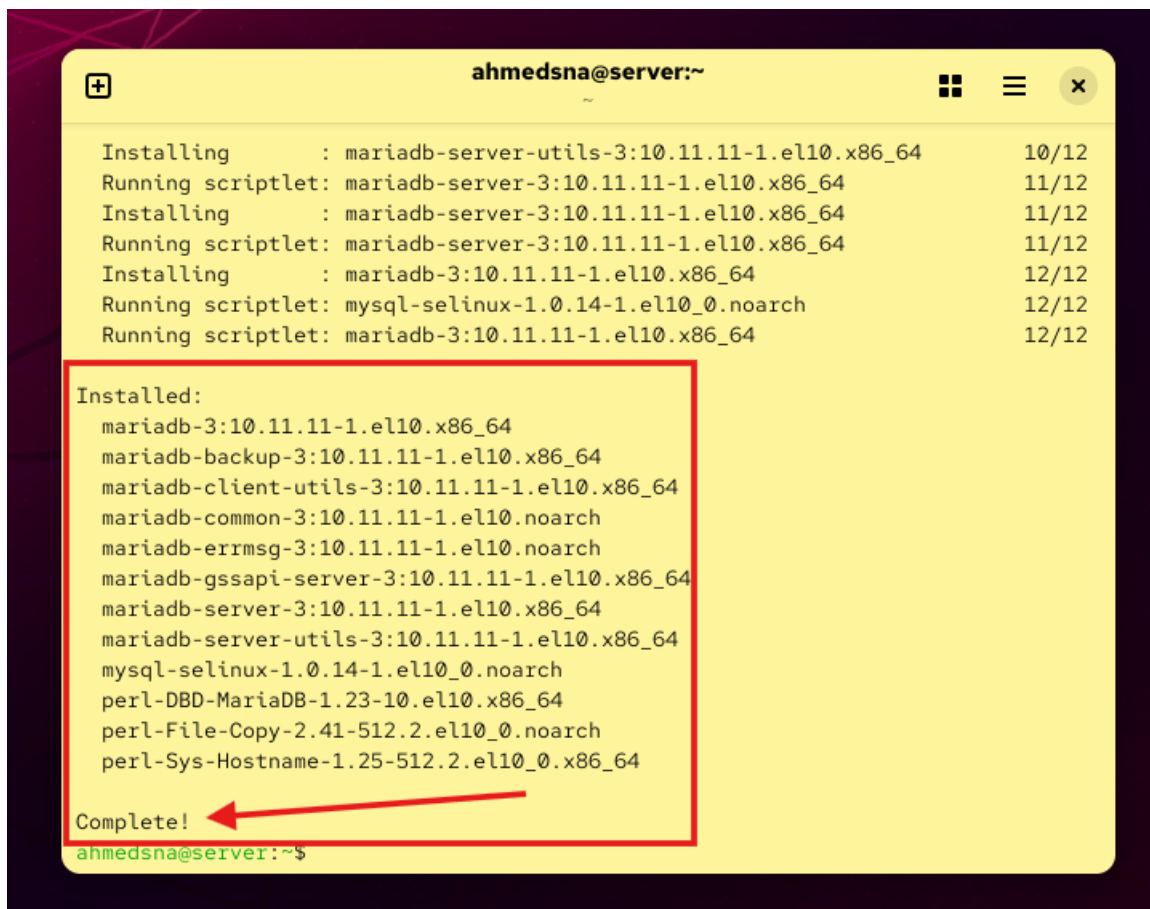
Figure 34 Installing MariaDB on Rocky Server



```
ahmedsna@server:~$ sudo dnf install mariadb-server -y
```

Before starting any configuration, we install the required packages on the server using the command “**sudo dnf install mariadb-server -y**”.

Figure 35 verification of mariadb server installation



```

Installing      : mariadb-server-utils-3:10.11.11-1.el10.x86_64      10/12
Running scriptlet: mariadb-server-3:10.11.11-1.el10.x86_64        11/12
Installing      : mariadb-server-3:10.11.11-1.el10.x86_64        11/12
Running scriptlet: mariadb-server-3:10.11.11-1.el10.x86_64        11/12
Installing      : mariadb-3:10.11.11-1.el10.x86_64                12/12
Running scriptlet: mysql-selinux-1.0.14-1.el10_0.noarch            12/12
Running scriptlet: mariadb-3:10.11.11-1.el10.x86_64                12/12

Installed:
 mariadb-3:10.11.11-1.el10.x86_64
 mariadb-backup-3:10.11.11-1.el10.x86_64
 mariadb-client-utils-3:10.11.11-1.el10.x86_64
 mariadb-common-3:10.11.11-1.el10.noarch
 mariadb-errmsg-3:10.11.11-1.el10.noarch
 mariadb-gssapi-server-3:10.11.11-1.el10.x86_64
 mariadb-server-3:10.11.11-1.el10.x86_64
 mariadb-server-utils-3:10.11.11-1.el10.x86_64
 mysql-selinux-1.0.14-1.el10_0.noarch
 perl-DBD-MariaDB-1.23-10.el10.x86_64
 perl-File-Copy-2.41-512.2.el10_0.noarch
 perl-Sys-Hostname-1.25-512.2.el10_0.x86_64

Complete!
ahmedsna@server:~$
```

We confirm that the MariaDB server is installed successfully

6.2.3 Adding support for all Unicode Characters

Next, we want to configure the `charset.cnf` file to ensure compatibility with all Unicode characters

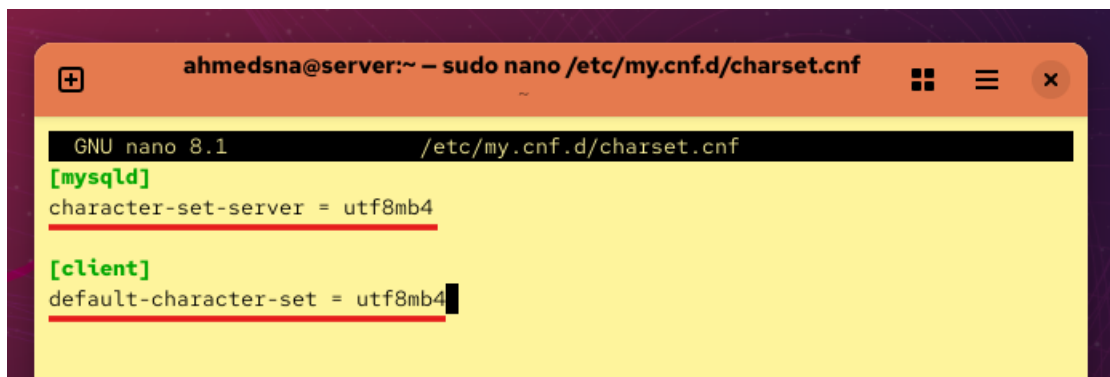
Figure 36 command to create `charset.cnf` file inside `my.cnf.d` directory



- `$ sudo nano /etc/my.cnf.d/charset.cnf`

Note: the `charset.cnf` file will be empty, we need to carefully type the lines as seen in the below image

Figure 37 Configuring inside `charset.cnf` file



In the figure above, we edit the `/my.cnf.d/charset.cnf` file and update its settings to use `utf8mb4` encoding, which is the modern standard and supports emojis and other special characters.

- Ctrl + S to save
- Ctrl + X to exit

6.2.4 Enabling MariaDB permanently

Figure 38 Enabling mariadb-server permanently

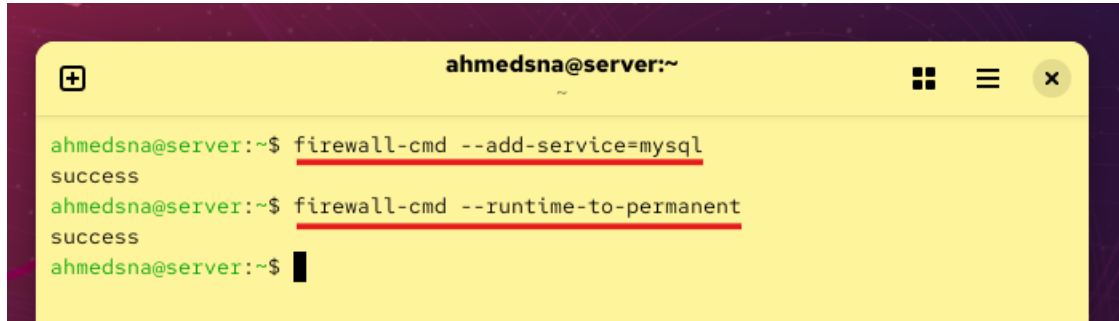
A terminal window titled 'ahmedsna@server:~' with a yellow background. The user enters the command 'systemctl enable --now mariadb'. The output shows three symlinks being created: '/etc/systemd/system/mysql.service' pointing to '/usr/lib/systemd/system/mariadb.service', '/etc/systemd/system/mysqld.service' pointing to '/usr/lib/systemd/system/mariadb.service', and '/etc/systemd/system/multi-user.target.wants/mariadb.service' pointing to '/usr/lib/systemd/system/mariadb.service'. The prompt returns to 'ahmedsna@server:~\$' with a cursor.

```
ahmedsna@server:~$ systemctl enable --now mariadb
Created symlink '/etc/systemd/system/mysql.service' → '/usr/lib/systemd/system/mariadb.service'.
Created symlink '/etc/systemd/system/mysqld.service' → '/usr/lib/systemd/system/mariadb.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/mariadb.service' → '/usr/lib/systemd/system/mariadb.service'.
ahmedsna@server:~$
```

- \$ Systemctl enable --now mariadb

In this step, after installing and configuring everything, we simply enable the mariadb service, so it stays on permanently.

6.2.5 Configuring Firewall to whitelist MariaDB

A terminal window titled 'ahmedsna@server:~' with a yellow background. The user enters two commands: 'firewall-cmd --add-service=mysql' and 'firewall-cmd --runtime-to-permanent'. Both commands return 'success'. The prompt returns to 'ahmedsna@server:~\$' with a cursor.

```
ahmedsna@server:~$ firewall-cmd --add-service=mysql
success
ahmedsna@server:~$ firewall-cmd --runtime-to-permanent
success
ahmedsna@server:~$
```

- \$ firewall-cmd --add-service=mysql
- \$ firewall-cmd --runtime-to-permanent

In this step, we configure the firewall. The command “`firewall-cmd --add-service=mysql`” allows the MariaDB service through the firewall, and “`firewall-cmd --runtime-to-permanent`” saves this rule so it stays active permanently.

6.2.6 Securing MariaDB Installation

Figure 39 Installing and Configuring secure MariaDB

```

root@server:~ - sudo su -

ahmedsna@server:~$ sudo su -
Last login: Wed Nov  5 17:52:20 +08 2025 on pts/0
root@server:~# mysql_secure_installation

NOTE: RUNNING ALL PARTS OF THIS SCRIPT IS RECOMMENDED FOR ALL MariaDB
SERVERS IN PRODUCTION USE! PLEASE READ EACH STEP CAREFULLY!

In order to log into MariaDB to secure it, we'll need the current
password for the root user. If you've just installed MariaDB, and
haven't set the root password yet, you should just press enter here.

Enter current password for root (enter for none):
OK, successfully used password, moving on...

Setting the root password or using the unix_socket ensures that nobody
can log into the MariaDB root user without the proper authorisation.

You already have your root account protected, so you can safely answer 'n'.

Switch to unix_socket authentication [Y/n] n
... skipping.
  
```

- \$ sudo su -
- # mysql_secure_installation

Note: The command “mysql_secure_installation” is executed with root user privileges

In this step, we install MySQL by running “mysql_secure_installation” with root privileges. This process strengthens the security of MariaDB through several prompts. It first asks us to set a root password if one has not already been created.

Figure 40 Configuring Secure MariaDB (Continued)

```
root@server:~ - sudo su -

Remove anonymous users? [Y/n] Y
... Success!

Normally, root should only be allowed to connect from 'localhost'. This
ensures that someone cannot guess at the root password from the network.

Disallow root login remotely? [Y/n] Y
... Success!

By default, MariaDB comes with a database named 'test' that anyone can
access. This is also intended only for testing, and should be removed
before moving into a production environment.

Remove test database and access to it? [Y/n] Y
- Dropping test database...
... Success!
- Removing privileges on test database...
... Success!

Reloading the privilege tables will ensure that all changes made so far
will take effect immediately.

Reload privilege tables now? [Y/n] Y
... Success!

Cleaning up...

All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.

Thanks for using MariaDB!
root@server:~#
```

Note: Press Y for all questions asked except unit_socket authentication

Continuing from the previous step, the system removes anonymous users, blocks remote root logins, and deletes the test database along with its access. These changes help secure MariaDB.

Type “Exit” to logout of root user

6.2.6 Verifying MariaDB Service

Now that we have MariaDB enabled at boot and running securely on our Rocky server, we need to confirm that the service is operating correctly.

To check the status of MariaDB Server we run the command:

- `$ sudo systemctl status mariadb`

Figure 41 Status of MariaDB database server

```

ahmedsna@server:~ – sudo systemctl status mariadb

● mariadb.service - MariaDB 10.11 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Fri 2025-11-28 10:03:31 +08; 1h 6min ago
 Invocation: 4d78962ea64444b7880668a49cb17639
    Docs: man:mariabdb(8)
          https://mariadb.com/kb/en/library/systemd/
 Process: 4843 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
 Process: 4865 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
 Process: 4972 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
 Main PID: 4959 (mariabdb)
   Status: "Taking your SQL requests now..."
    Tasks: 8 (limit: 67504)
  Memory: 223.6M (peak: 250.2M)
     CPU: 4.414s
   CGroup: /system.slice/mariadb.service
           └─4959 /usr/libexec/mariabdb --basedir=/usr

Nov 28 10:03:30 server.distinction.club mariadb-prepare-db-dir[4906]: you need to be the syst
Nov 28 10:03:30 server.distinction.club mariadb-prepare-db-dir[4906]: After connecting you ca
Nov 28 10:03:30 server.distinction.club mariadb-prepare-db-dir[4906]: able to connect as any
lines 1-20

```

This verifies that the MariaDB service is running properly.

Output shows:

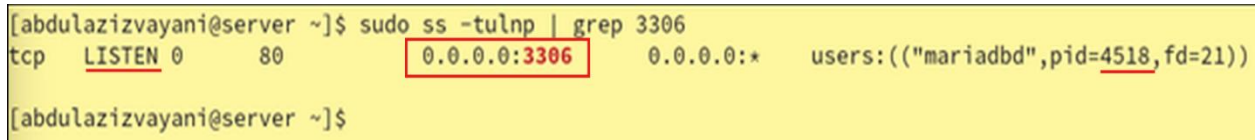
- **Active: active (running)**
- MariaDB 10.11 is started successfully
- No errors during startup

6.2.7 Verifying Port Connections

To check if the MariaDB database service is running and listening for connections, we run the command:

- `$ sudo ss -tulnp | grep 3306`
- `ss` is the tool used to display socket information.
- `-t` shows TCP connections, which are reliable.
- `-u` shows UDP connections, which are faster and connectionless.
- `-l` shows only services that are listening for incoming connections.
- `-n` shows ports as numbers instead of names to make the output clearer.
- `-p` shows the process and its ID that is using the port, and it needs sudo for access.
- `|` takes output of the first command and passes it to the next command
- `grep` filters the list to show only the lines containing “3306”

Figure 42 Confirms that MariaDB is listening on port 3306



```
[abdulazizvayani@server ~]$ sudo ss -tulnp | grep 3306
tcp LISTEN 0      80      0.0.0.0:3306  0.0.0.0:*    users:((\"mariadb\",pid=4518,fd=21))
[abdulazizvayani@server ~]$
```

This confirms that MariaDB is listening on the default MySQL port 3306.

LISTEN indicates that the service is active and waiting for a connection.

0.0.0.0:3306 means the server is open to connections from any IP address on port 3306.

pid=4518: shows the unique ID number assigned to the running database process.

6.2.8 Configuring MariaDB to listen on all interfaces

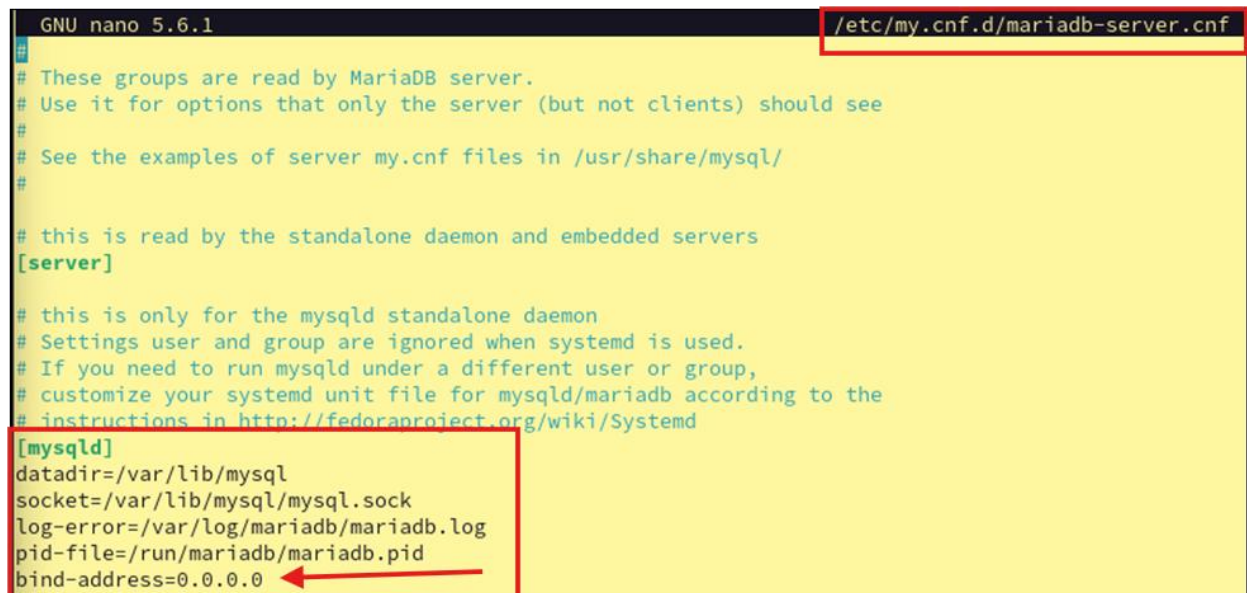
Next we configure MariaDB to listen on all interfaces by setting the bind address to 0.0.0.0

- `$ sudo nano /etc/my.cnf.d/mariadb-server.cnf`

Figure 43 command to open mariadb-server.cnf file for configuration



Scroll down in the file and enter “`bind-address=0.0.0.0`” in the same spot shown in the image below



This allows MariaDB to accept connections from any IP Address, which is needed for remote access to the database.

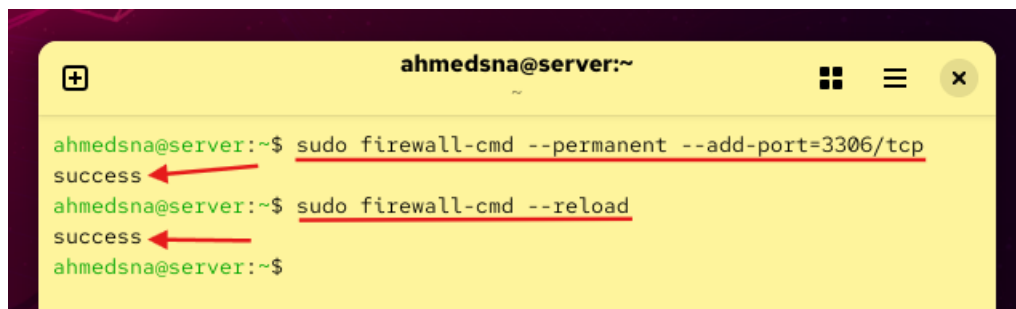
- Ctrl + S to Save
- Ctrl + X to exit

6.2.9 Whitelisting Port 3306

To allow port 3306/TCP (MariaDB/MySQL) through the firewall using firewalld, we need to run the following two commands:

- `$ sudo firewall-cmd --permanent --add-port=3306/tcp`
 - `--permanent`: Ensures the rule survives a system reboot.
 - `--add-port=3306/tcp`: Specifically opens TCP port 3306.
- `$ sudo firewall-cmd --reload`
 - This activates the new rule immediately without dropping current connections.

Figure 44 commands to whitelist port 3306 and reload firewalld



```

ahmedsna@server:~$ sudo firewall-cmd --permanent --add-port=3306/tcp
success
ahmedsna@server:~$ sudo firewall-cmd --reload
success
ahmedsna@server:~$
  
```

Make sure you receive a success message like the one shown in the image above.

To display the complete current configuration of our active firewall zone, we need to run the command:

- `$ sudo firewall-cmd --list-all`

Figure 45 shows complete configuration of firewall



```

[abdulazizvayani@server ~]$ sudo firewall-cmd --list-all
public (active)
target: default
icmp-block-inversion: no
interfaces: enp0s3 enp0s8
sources:
services: cockpit dhcp dhcpv6-client ftp http https imap imaps pop3 pop3s smtp smtps ssh
ports: 53/udp 53/tcp 990/tcp 40000-40001/tcp 3306/tcp
protocols:
forward: yes
masquerade: no
forward-ports:
source-ports:
icmp-blocks:
rich rules:
[abdulazizvayani@server ~]$
  
```

The output shows that port **3306/TCP** (MariaDB) is allowed through the firewall, confirming that remote clients can reach the database.

6.2.10 Logging into MariaDB

To log into the MariaDB database server with the highest level of administrative privileges:

- `$ sudo mysql -u root -p`
 - `mysql` is the name of the client program. It tells the terminal, “I want to start the tool that talks to the database”
 - `-u` stands for User. It tells the database, "I am logging in as..."
 - `root` refers to the main database administrator account.
 - `-p` tells the system to ask you for the password after you press Enter.

Note: This is the "root" account *inside* the database, which allows you to do anything (delete databases, create users, etc.).

We check the list of existing database users and their allowed hosts:

- `> SELECT host, user FROM mysql.user`

Figure 46 list of existing database users and their allowed hosts

```
[abdulazizvayani@server ~]$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SELECT host, user FROM mysql.user;
+-----+-----+
| Host          | User          |
+-----+-----+
| %             | rockyuser     |
| 10.0.200.80   | rockyuser     |
| localhost     | mariadb.sys   |
| localhost     | mysql         |
| localhost     | root          |
+-----+-----+
5 rows in set (0.002 sec)

MariaDB [(none)]>
```

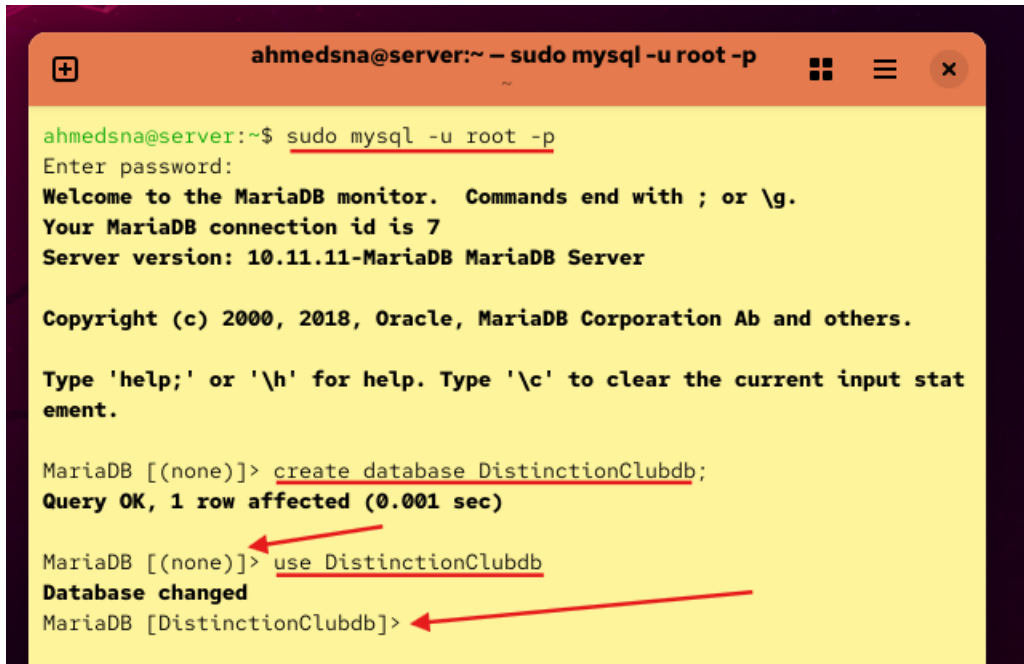
- rockyuser is allowed from % (any IP) — enabling remote connection.
- rockyuser is also allowed specifically from 10.0.200.80 (Ubuntu client).
- root and mysql.sys accounts are local-only.
- This verifies that the correct MariaDB users are configured for both local and remote access.

6.2.11 Creating and using the database

To create and use the database DistinctionClubdb, we run the following two SQL queries after logging into the MariaDB terminal:

- > create database DistinctionClubdb
- > use DistinctionClubdb

Figure 47 creation of DistinctionClubdb database



```
ahmedsna@server:~ -- sudo mysql -u root -p

ahmedsna@server:~$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 10.11.11-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> create database DistinctionClubdb;
Query OK, 1 row affected (0.001 sec)

MariaDB [(none)]> use DistinctionClubdb
Database changed
MariaDB [DistinctionClubdb]>
```

Note: Notice that the prompt has changed to [DistinctionClubdb] > , this shows that we are now inside the DistinctionClubdb database.

6.2.12 Verifying Database Creation

To List Existing Databases, we run the query “Show databases;”

Figure 48 lists existing databases in MariaDB

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| DistinctionClubdb |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.002 sec)

MariaDB [(none)]> █
```

- The database **DistinctionClubdb** is successfully listed
- The below are default system databases that were automatically created when we installed MariaDB:
 - Information_schema,
 - Mysql,
 - Performance_schema
 - sys

Note: The default System databases store user accounts, privileges, and performance data. Do not delete them, as they are essential for the server to run correctly.

6.2.13 Creating and Inserting Data into Table

Note: Before creating a table or adding records, we need to make sure we are currently inside the MariaDB terminal.

Type the following commands to create a table users and insert 3 records into it:

- > use DistinctionClubdb
- > CREATE TABLE users (id int, name varchar(25), address varchar(25), primary key(id));
- > INSERT INTO users (id, name, address) VALUES(1, 'Aziz', 'Malaysia'), (2, 'John', 'Singapore'), (3, 'Sarah', 'Jakarta');

```
MariaDB [(none)]> use DistinctionClubdb
Database changed
MariaDB [DistinctionClubdb]> CREATE TABLE users (id int, name varchar(25), address
varchar(25), primary key(id) );
Query OK, 0 rows affected (0.034 sec)

MariaDB [DistinctionClubdb]> INSERT INTO users (id, name, address) VALUES(1, 'Aziz'
,'Malaysia'), (2, 'John', 'Singapore'), (3, 'Sarah', 'Jakarta');
Query OK, 3 rows affected (0.014 sec)
Records: 3 Duplicates: 0 Warnings: 0

MariaDB [DistinctionClubdb]> 
```

The messages “Query OK” and “Query OK, 3 rows affected” show that the users table was created successfully and the records were inserted without any issues.

6.2.14 Verifying Table Creation

To list existing tables in DistinctionClubdb database, type:

- > Show tables;

Figure 49 Confirms creation of users table in DistinctionClubdb

```
MariaDB [DistinctionClubdb]> show tables;
+-----+
| Tables_in_DistinctionClubdb |
+-----+
| users                        |
+-----+
1 row in set (0.004 sec)

MariaDB [DistinctionClubdb]> █
```

This confirms that the users table has been created in the DistinctionClubdb database

6.2.15 Querying the Table Content

To retrieve the records in the users table we run the query:

- > SELECT * FROM users;
- The SELECT query retrieves all records from the users table.

```
MariaDB [DistinctionClubdb]> SELECT * FROM users;
+----+-----+-----+
| id | name  | address |
+----+-----+-----+
| 1  | Aziz  | Malaysia |
| 2  | John  | Singapore |
| 3  | Sarah | Jakarta  |
+----+-----+-----+
3 rows in set (0.001 sec)

MariaDB [DistinctionClubdb]>
```

The query results confirm that all three records were added to the table successfully.

6.2.15 Installing MariaDB Client on Ubuntu

To allow the Ubuntu client machine to connect to the MariaDB server, the MariaDB client package was installed. This provides the mysql command-line tool required for remote access.



```
abdulazizvayani@client:~/Desktop$ sudo apt install mariadb-client -y
[sudo] password for abdulazizvayani:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mariadb-client is already the newest version (1:10.11.13-0ubuntu0.24.04.1).
The following packages were automatically installed and are no longer required:
  linux-headers-6.14.0-29-generic linux-hwe-6.14-headers-6.14.0-29 linux-hwe-6.14-tools-6.14.0-29
  linux-image-6.14.0-29-generic linux-modules-6.14.0-29-generic linux-modules-extra-6.14.0-29-generic
  linux-tools-6.14.0-29-generic
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 7 not upgraded.
abdulazizvayani@client:~/Desktop$
```

Output Explanation:

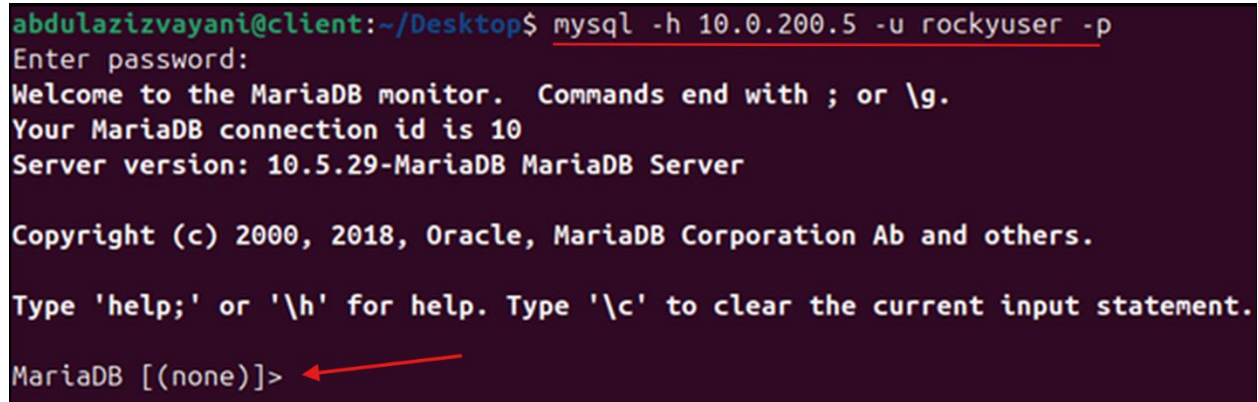
- The terminal output shows that mariadb-client is already the newest version, meaning it was successfully installed previously.
- No additional packages were required.
- The system confirms:
0 upgraded, 0 newly installed, 0 to remove and 7 not upgraded.

Screenshot shows:

- Successful installation (or confirmation) of the MariaDB client.
- The client is now ready to connect remotely to the Rocky Linux server.

6.2.16 Remote Connection to MariaDB from Ubuntu Client

After installing the MariaDB client, we test remote access by connecting from the Ubuntu machine to the MariaDB server running on Rocky Linux.



```
abdulazizvayani@client:~/Desktop$ mysql -h 10.0.200.5 -u rockyuser -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 10
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

Explanation:

- **-h 10.0.200.5** → Connect to the Rocky server's IP address
- **-u rockyuser** → Authenticate using the remote MariaDB user
- **-p** → Prompt for password securely

Result:

The connection was successful, and the MariaDB welcome message was displayed — confirming:

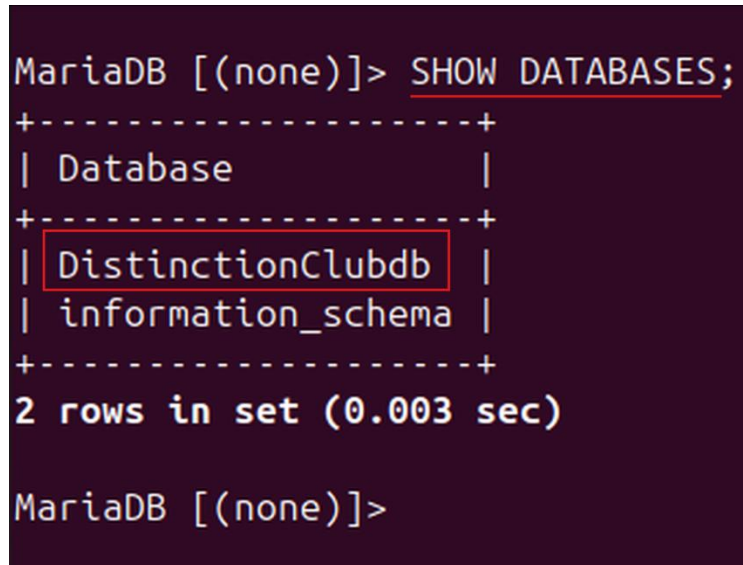
- The server is reachable
- The user rockyuser has remote privileges
- Port 3306 is open and accepting connections

Screenshot shows:

- Successful login from Ubuntu to Rocky's MariaDB server
- Server version: 10.5.29-MariaDB

6.2.17 Verifying Databases from the Ubuntu Client

Once connected to the MariaDB server remotely, we checked which databases were accessible to the rockyuser account using the “SHOW DATABASES;” command.



```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| DistinctionClubdb |
| information_schema |
+-----+
2 rows in set (0.003 sec)

MariaDB [(none)]>
```

Result:

The output shows:

- DistinctionClubdb — the custom database we created earlier
- information_schema — a system metadata database

This confirms that:

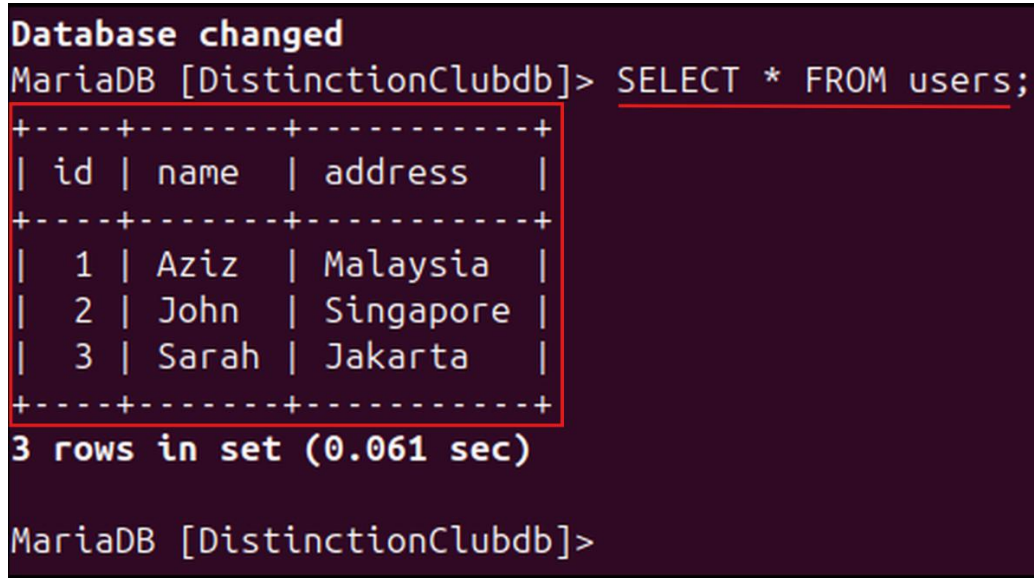
- The remote user has correct privileges
- The Ubuntu client can successfully view databases stored on the Rocky Linux MariaDB server

Screenshot shows:

- DistinctionClubdb is visible from the remote client
- Remote authentication and permissions are correctly configured

6.2.18 Querying Table Data from Ubuntu Client

To validate full read access from the Ubuntu client, we selected all records from the users table inside the DistinctionClubdb database.



```
Database changed
MariaDB [DistinctionClubdb]> SELECT * FROM users;
+----+-----+-----+
| id | name  | address |
+----+-----+-----+
|  1 | Aziz  | Malaysia |
|  2 | John  | Singapore |
|  3 | Sarah | Jakarta  |
+----+-----+-----+
3 rows in set (0.061 sec)

MariaDB [DistinctionClubdb]>
```

This proves that:

- The Ubuntu client can not only connect, but also query data
- Database configuration, permissions, and networking are correctly set
- Remote DB access is fully functional

Screenshot shows:

- Correct data retrieved remotely
- Full MariaDB functionality verified end-to-end

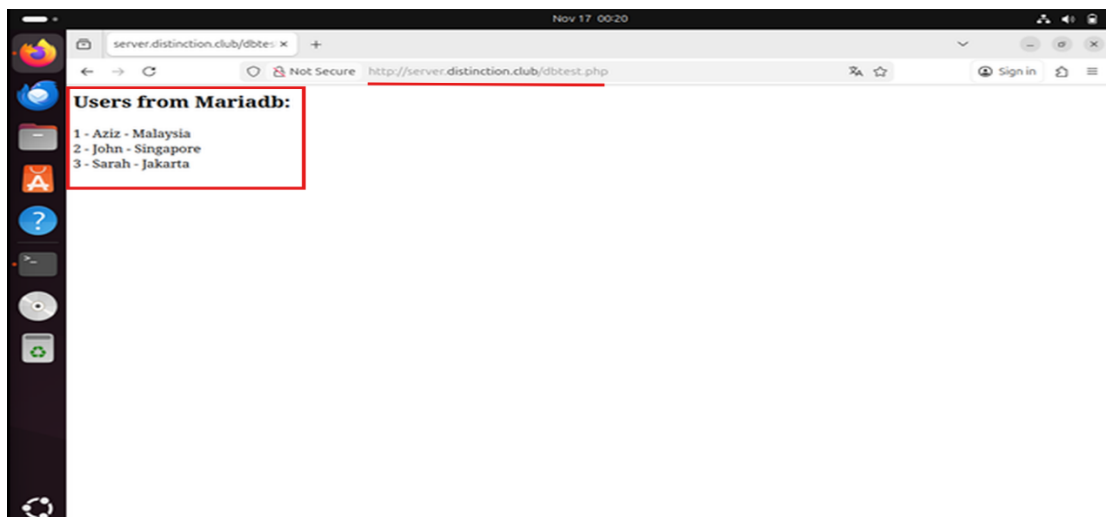
6.2.19 Web-based database output (PHP Integration)

To verify that MariaDB was successfully integrated with the web server, a PHP script (dbtest.php) was created inside the document root. This script connects to the MariaDB server and fetches user data from the users table.

Figure 50 Webpage running locally on Rocky Server Browser



Figure 51 webpage running remotely on Ubuntu Client Browser



This confirms:

- Apache is running correctly
- PHP is functioning and able to query MariaDB
- The database connection is working from both the server and client sides

First Image — Rocky Server Browser:

- Shows the webpage running locally on the server
- Confirms server-side PHP → MariaDB connectivity

Second Image — Ubuntu Client Browser:

- Shows the webpage running remotely on the Ubuntu Client Machine
- Confirms full network connectivity and database access

6.2.20 Personal Reflection

Through my project, I configured and installed a MariaDB database on Rocky Linux which permitted remote access to a MariaDB database server hosted on Ubuntu. In addition, my project provided the opportunity for me to establish database connectivity via a website developed with PHP, learn more about managing user permissions for database access, create firewall rules for server protection, set up network bindings for database connections, and finally, troubleshoot connectivity issues in a live environment. In addition to enhancing your understanding of Server - Client architecture related to Web Servers and Database Deployment within a Local Area Network

6.3 Advanced Apache Server Features

After I finished the HTTPS setup on Apache, I decided to add some extra features to make my web server more advanced, more secure, and more realistic exactly just like a real-world hosting environment. These features help me understand how Apache can handle multiple websites, redirect traffic securely, protect folders, monitor performance, and make websites more user-friendly.

6.3.1 Virtual Hosting (Hosting Custom Websites)

By default, Apache shows web files from the folder `/var/www/html`.

But if we want to host multiple websites on the same server, we use something called **Virtual Hosts**.

Note: Each Virtual Host has its own folder, domain name, and settings.

1. Create a new website folder

○ `# mkdir -p /var/www/distinction.club`



mkdir means “make directory.”

-p just makes sure the full path is created.

So here we are creating a brand-new folder called `/var/www/distinction.club` where our website files will live.

2. Create a simple Homepage file

A terminal window titled 'root@server:~' with search, menu, and close buttons. The command '[root@server ~]# echo '<h1>Welcome to Distinction Club!</h1>' > /var/www/distinction.club/index.html' is entered. A red arrow points from the text 'this symbol tells the system' to the '>' character in the command.

```
[root@server ~]# echo '<h1>Welcome to Distinction Club!</h1>' > /var/www/distinction.club/index.html
[root@server ~]#
```

`echo` prints the message inside the quotes.

`'>'` this symbol tells the system “Save this text into a file.”

So this creates a new file called `index.html` in that folder so it's like our website's home page.

Whenever someone visits the site, Apache will show this page first.

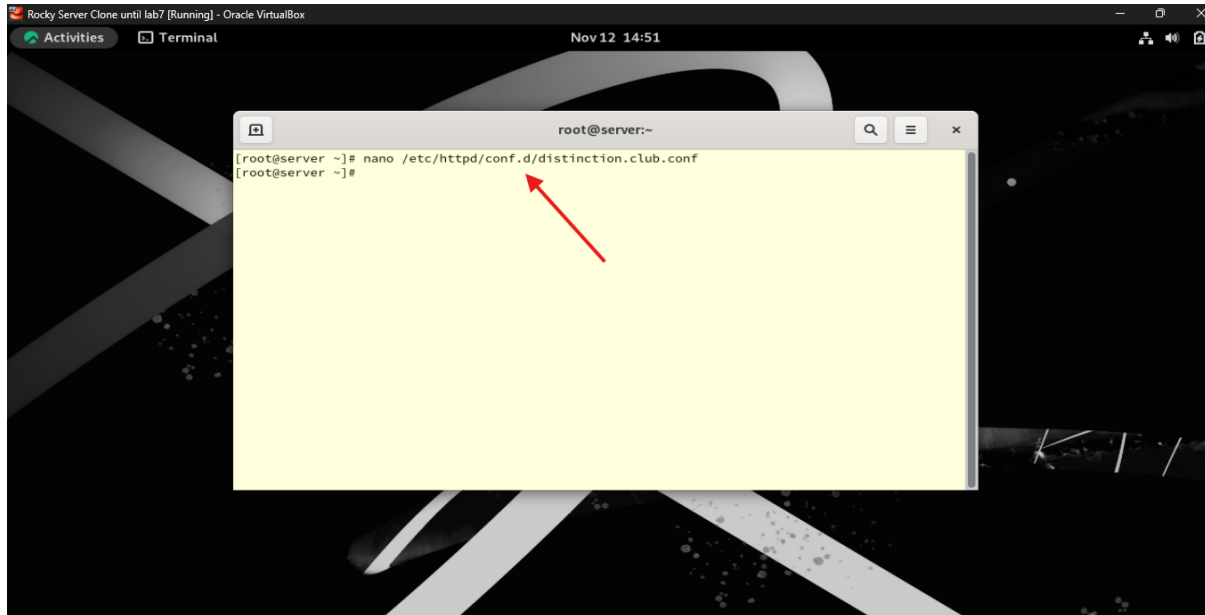
Note: Single quotes tell bash “everything inside is plain text — don't interpret symbols.”

This will safely create your HTML file.

3. Create a new configuration file

Open the file:

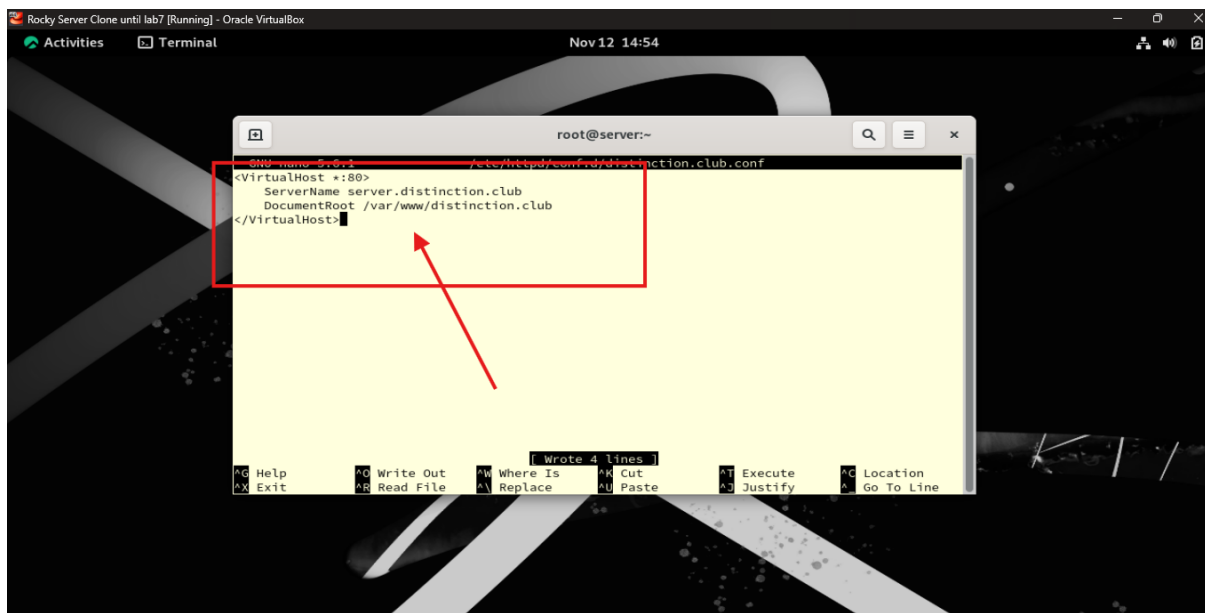
o `# nano /etc/httpd/conf.d/distinction.club.conf`



We're now opening a configuration file inside Apache's folder `/etc/httpd/conf.d/`.

Each `.conf` file here tells Apache how to handle a specific site.

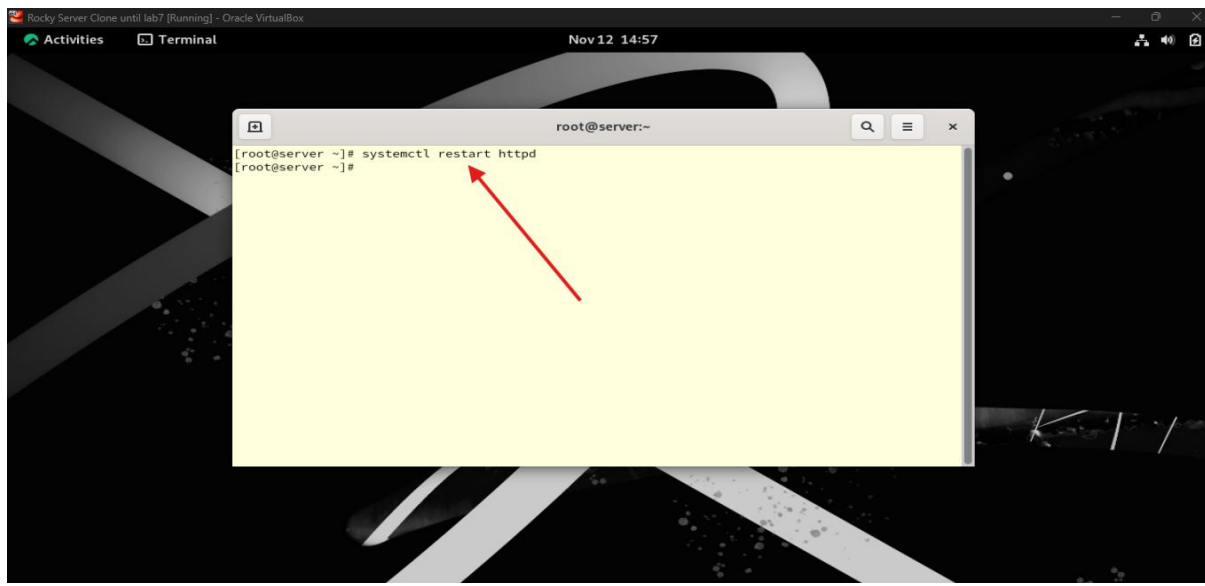
We'll now set our domain and its folder.



- <VirtualHost *:80> → tells Apache this site listens on port 80 (normal HTTP).
- ServerName → the domain name that belongs to this site.
- DocumentRoot → the folder where the website files are stored.
- </VirtualHost> → marks the end of this block.

4. Restart Apache

- # systemctl restart httpd



Note: Any time we change Apache configs, we restart it, so it loads the new settings.

Conclusion:

Virtual Hosts allow a single server to host multiple websites by separating each site's files and settings. This is how real hosting companies support many clients on one machine. Apache loads configuration files from **/etc/httpd/conf.d** so that each site can be managed independently without editing the main Apache file.

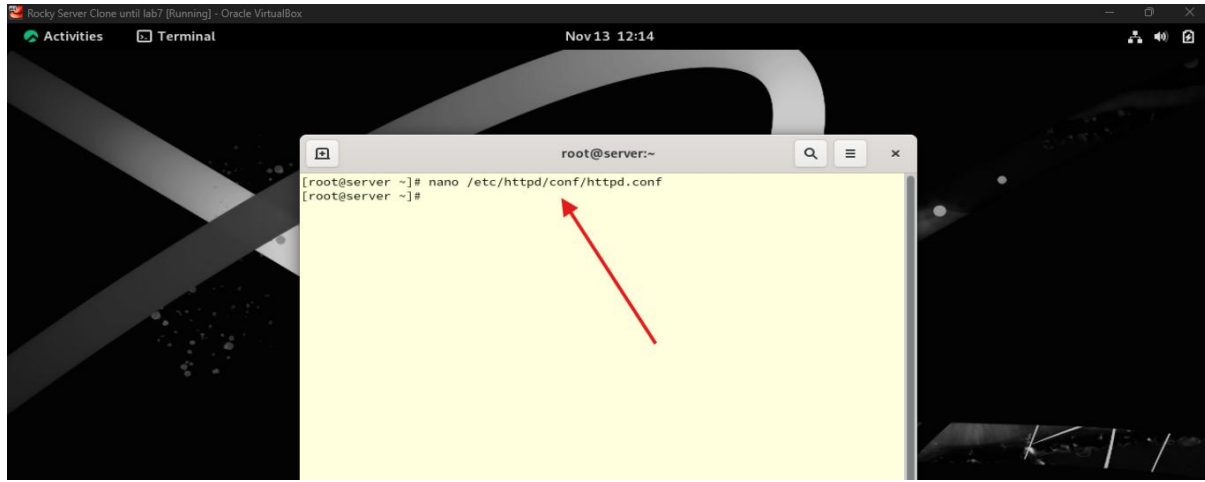
6.3.2 Force HTTPS (Redirect HTTP → HTTPS)

We already have HTTPS working, **but some users might still type http://**.

We'll make Apache automatically send them to https:// — this keeps all traffic encrypted.

1. Open Apache's main config

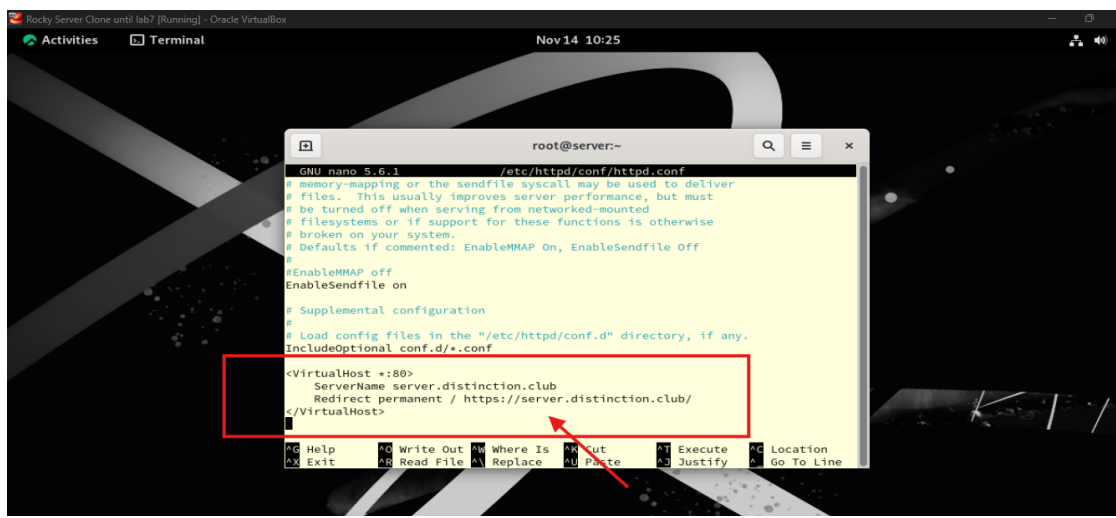
- # nano /etc/httpd/conf/httpd.conf



This file is the main brain of Apache.

We'll add a rule at the bottom so that any visitor who comes to HTTP gets redirected to HTTPS.

2. Adding the re-direct block



Put the re-direct block at the bottom of the file.

- `<VirtualHost *:80>` → This block handles **all HTTP (port 80)** traffic
- **ServerName** → The domain name we want Apache to match
- **Redirect permanent / https://server.distinction.club/** → This sends **every single HTTP request** to the **HTTPS version**

3. Disable the Conflicting Port 80 Config

- `mv /etc/httpd/conf.d/distinction.club.conf /etc/httpd/conf.d/distinction.club.conf.bak`



By renaming the file to .bak:

- Apache will **ignore** the file
- The conflicting `<VirtualHost *:80>` is **disabled**
- Only **one** VirtualHost (the redirect block inside `httpd.conf`) is now active
- This allows the redirect to function correctly.

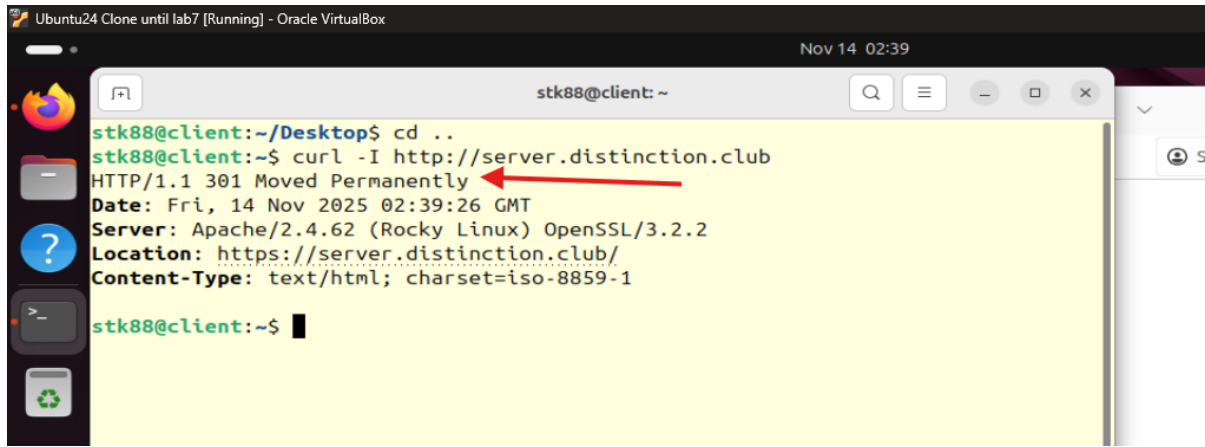
4. Restart Apache

- `# systemctl restart httpd`

This activates the redirect rule.

Now if anyone visits `http://server.distinction.club`, Apache will instantly redirect them to the HTTPS page.

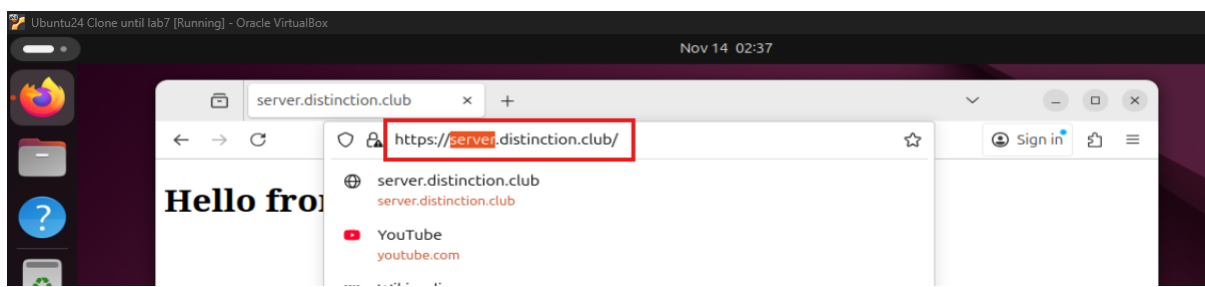
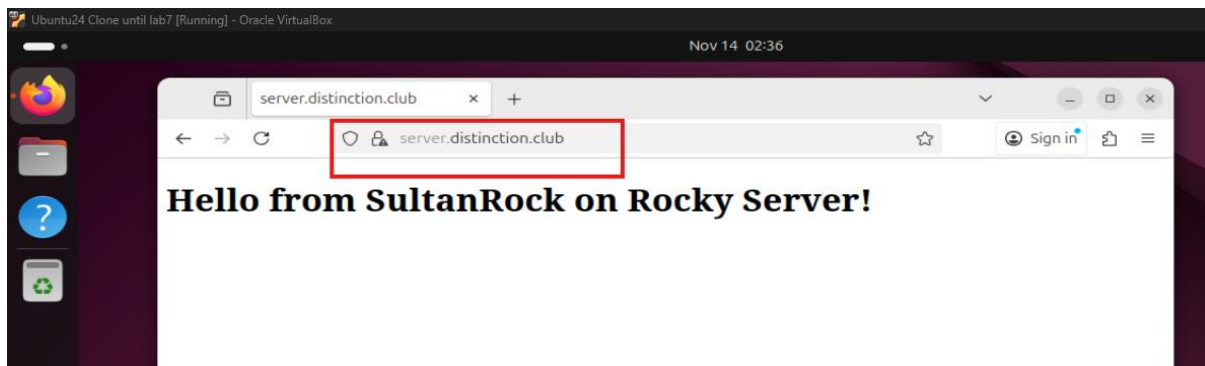
5. Testing it on Ubuntu using Firefox and Curl



```
stk88@client:~/Desktop$ cd ..
stk88@client:~$ curl -I http://server.distinction.club
HTTP/1.1 301 Moved Permanently
Date: Fri, 14 Nov 2025 02:39:26 GMT
Server: Apache/2.4.62 (Rocky Linux) OpenSSL/3.2.2
Location: https://server.distinction.club/
Content-Type: text/html; charset=iso-8859-1
stk88@client:~$
```

The screenshot shows a terminal window with the command `curl -I http://server.distinction.club` being executed. The output shows an HTTP 301 status code, indicating a permanent redirect to the HTTPS version of the site. A red arrow points to the `301 Moved Permanently` line.

Now Apache correctly returns **301 Moved Permanently**, which means the redirect is active. The browser or curl will now automatically switch from HTTP to HTTPS, ensuring a secure connection for all users.



6.3.3 Securing the website using Basic Authentication

Sometimes you don't want random people snooping on your site.

You want them to sign in like: "Who are you bro? Show me ID."

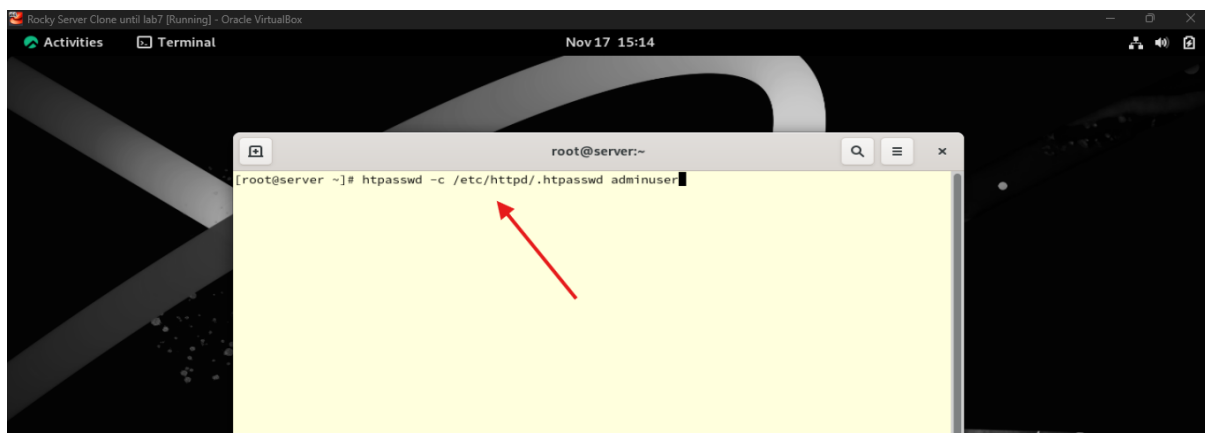
So we set up Basic Authentication , The simplest login system ever

1. Create the Authentication File

Note: You DO NOT need to create a new Rocky Linux user.

You can create a **login JUST for the website.**

○ # htpasswd -c /etc/httpd/.htpasswd adminuser



It will ask you for a password → **set ANY password.**(I put Bro9932sultan)

- This command creates the file that Apache will check when someone tries to access the site.
- **The username adminuser does not need to exist on Linux.** It only exists inside this authentication file.
- This is how real companies protect private areas.

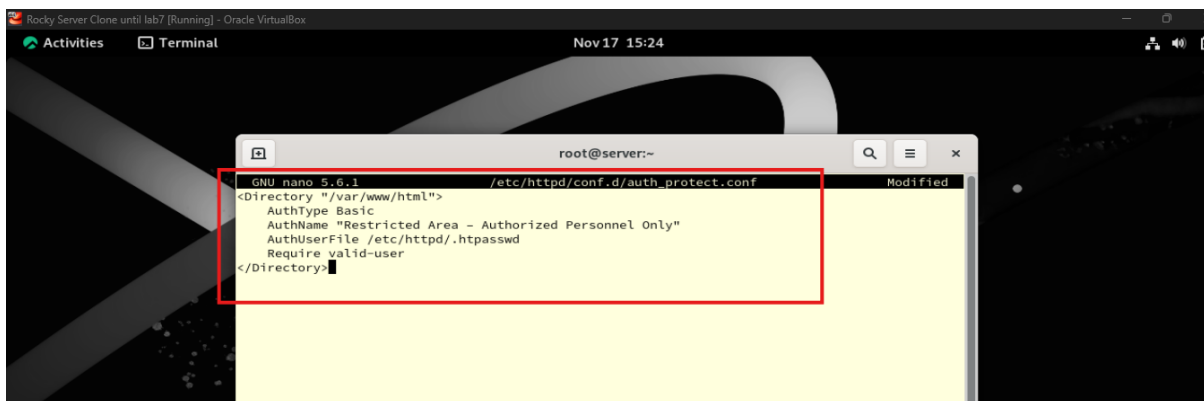
2. Telling Apache to Protect the Website Folder

Open a configuration file:

- # nano /etc/httpd/conf.d/auth_protect.conf



Write This (Very Important):

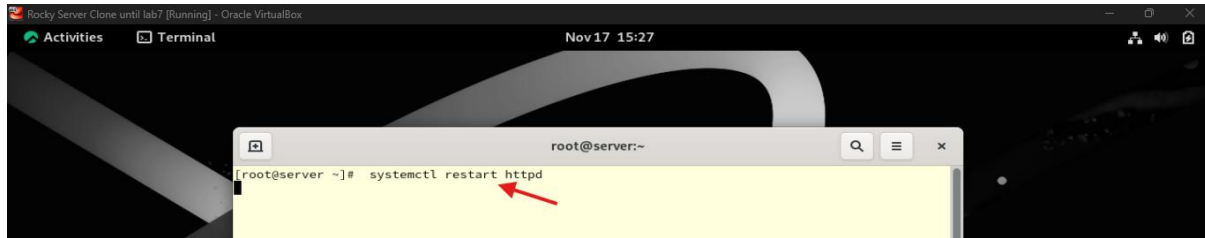


Why this step so important to this authentication process:

- **<Directory>** → tells Apache which folder to protect.
- **AuthType Basic** → uses built-in login popup.
- **AuthUserFile** → points to the file we created.
- **Require valid-user** → any user in the file is allowed.

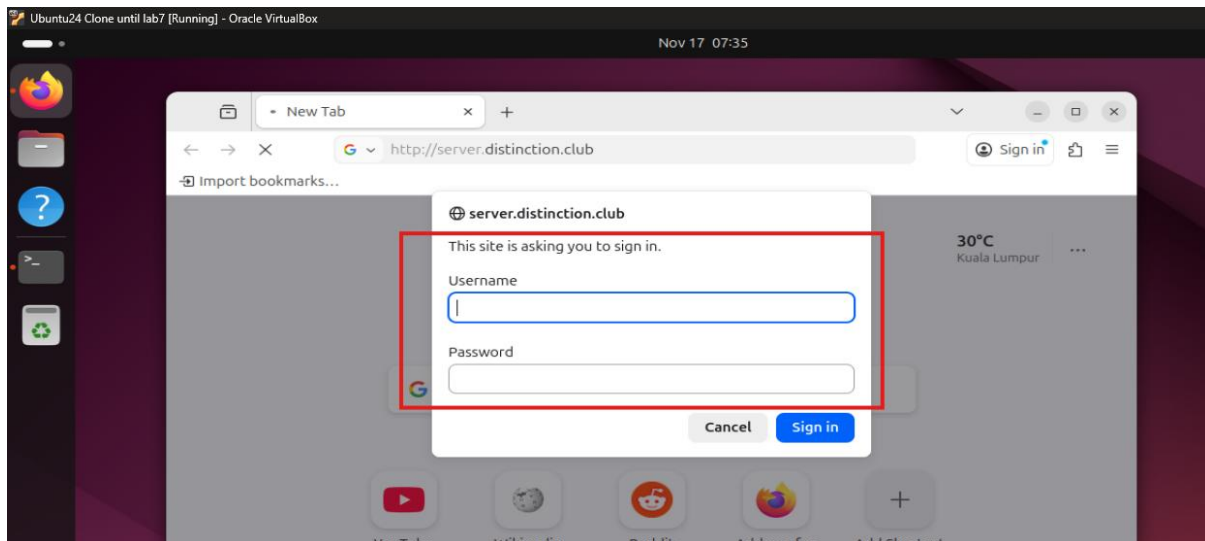
3. Restart Apache:

o # systemctl Restart httpd

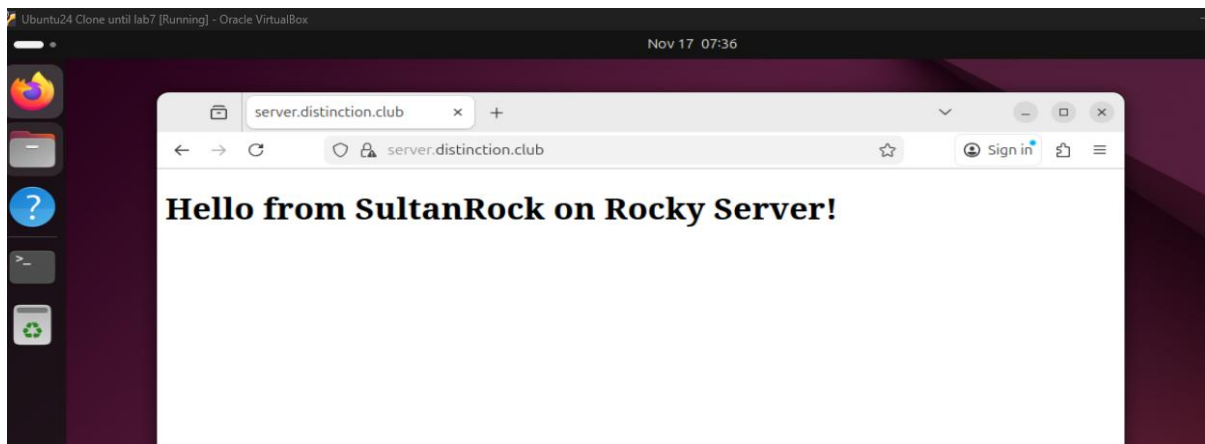


4. Testing from Ubuntu Client

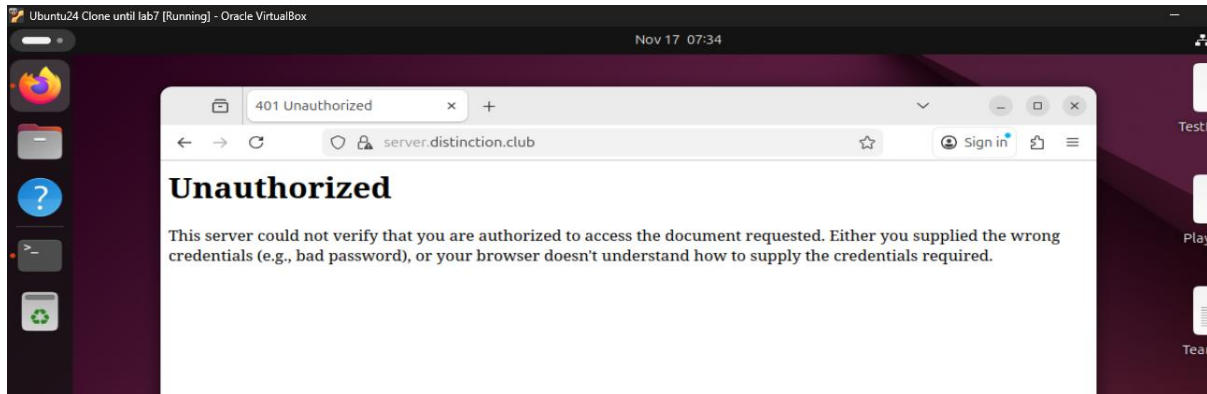
Open the Website on Ubuntu:



It works, ok let's login in:



If we try to just sign in without anything, let's see:



6.3.4 Custom 404 Error Page

Instead of showing a boring default “404 Not Found” page, we can make our own custom error page it improves professionalism, reduce user confusion, and help guide visitors back to the main site. It also prevents exposing Apache’s default error messages, which may reveal unnecessary system information.

Step 1: Create a Public Folder for Error Pages

By default, my website’s main folder `/var/www/html` is protected by a password.

If I put the error page inside that folder, Apache would require a password before showing the 404 page.

To avoid this, I created a separate public directory:

```
# mkdir -p /var/www/errorpages
```

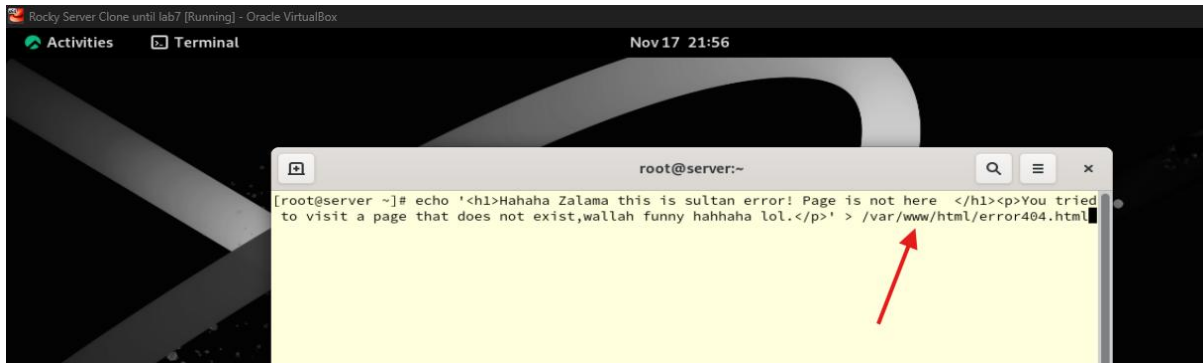


This folder is outside the protected area and will store my custom 404 file.

Step 2: Create the Custom 404 file

Inside the new folder, I created the custom error page:

(Put whatever you want , myself I put my own twist)



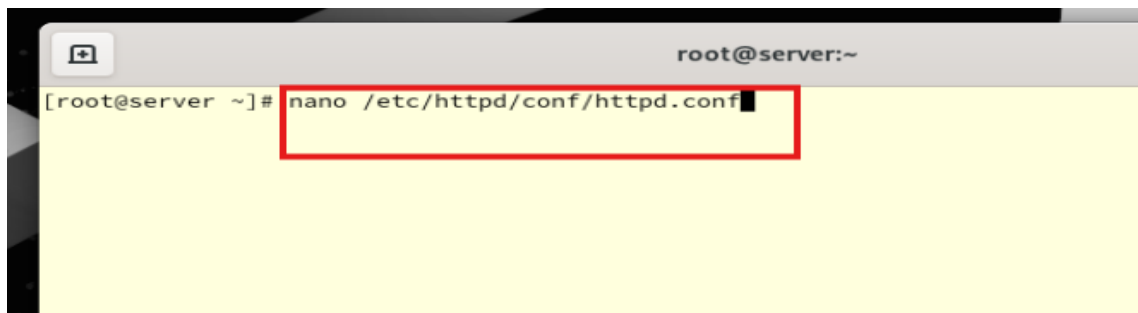
A screenshot of a terminal window titled "Rocky Server Clone until lab7 [Running] - Oracle VirtualBox". The terminal shows the command: `echo '<h1>Hahaha Zalama this is sultan error! Page is not here </h1><p>You tried to visit a page that does not exist,wallah funny hahaha lol.</p>' > /var/www/html/error404.html`. A red arrow points to the file path `/var/www/html/error404.html`.

This is the HTML message that users will see.

Step 3: Create an Alias for the Error Page URL

Open Apache's main config file:

o `# nano /etc/httpd/conf/httpd.conf`



A screenshot of a terminal window titled "root@server:~". The command `nano /etc/httpd/conf/httpd.conf` is entered and highlighted with a red box.

Add This:

A screenshot of the Apache configuration file `/etc/httpd/conf/httpd.conf` opened in the nano editor. The file shows various configuration directives. At the bottom, the line `Alias /errorpages/ "/var/www/errorpages/"` is added and highlighted with a red box. A red arrow points to this line.

This step creates a link between:

- The **URL path** → */errorpages/*
- The **actual folder on disk** → */var/www/errorpages/*

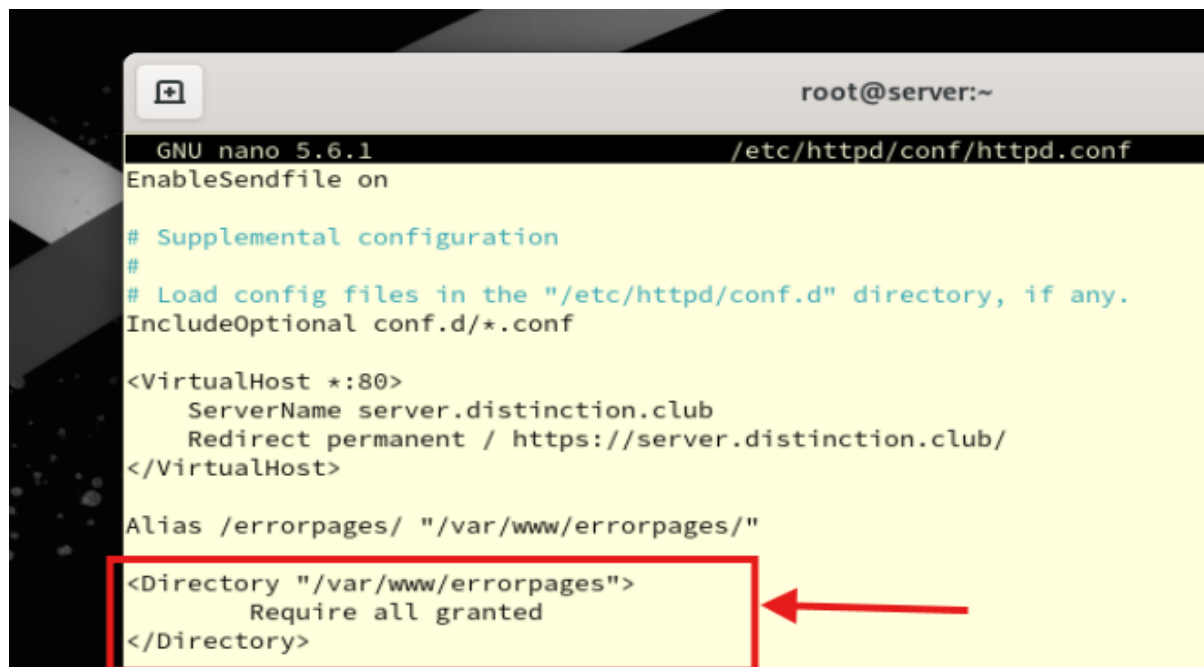
Think of it like this: When someone goes to *server.distinction.club/errorpages/meow* Apache knows to look inside */var/www/errorpages/*

Without this, Apache does not know how to serve this folder at all.

Step 4: Allow Apache to Access the Error Page Folder

Add This under:

Alias



```
root@server:~  
GNU nano 5.6.1 /etc/httpd/conf/httpd.conf  
EnableSendfile on  
  
# Supplemental configuration  
#  
# Load config files in the "/etc/httpd/conf.d" directory, if any.  
IncludeOptional conf.d/*.conf  
  
<VirtualHost *:80>  
    ServerName server.distinction.club  
    Redirect permanent / https://server.distinction.club/  
</VirtualHost>  
  
Alias /errorpages/ "/var/www/errorpages/"  
  
<Directory "/var/www/errorpages">  
    Require all granted  
</Directory>
```

This step tells Apache: “You’re allowed to read files from this folder. No password needed. No restrictions.”

Note: If we don’t do this, Apache will **block** the custom 404 page or ask for a password.

This is important because **the error page must always load**, even if the **main site is protected**.

STEP 5 :Tell Apache to Use Your Custom 404 Page (Very Important)

```
root@server:~  
GNU nano 5.6.1 /etc/httpd/conf/httpd.conf  
EnableSendfile on  
  
# Supplemental configuration  
# Load config files in the "/etc/httpd/conf.d" directory, if any.  
IncludeOptional conf.d/*.conf  
  
<VirtualHost *:80>  
    ServerName server.distinction.club  
    Redirect permanent / https://server.distinction.club/  
</VirtualHost>  
  
Alias /errorpages/ "/var/www/errorpages/"  
  
<Directory "/var/www/errorpages">  
    Require all granted  
</Directory>  
  
ErrorDocument 404 /errorpages/error404.html
```

This tells Apache: “Whenever someone enters a page that does not exist, show THIS custom file instead of the ugly default page.”

This is the final connection piece:

- Apache **knows how to access its URL** (because of Step 3)
- Apache **knows where the error page is** (because of Step 4)
- Apache **knows to use it for 404 errors** (this step 5)

Now the 404 system is fully linked and works like a real website.

STEP 6: Restart Apache

Restarting reloads all settings.

If you don't restart, Apache will NOT apply the new rules.

o # systemctl restart httpd

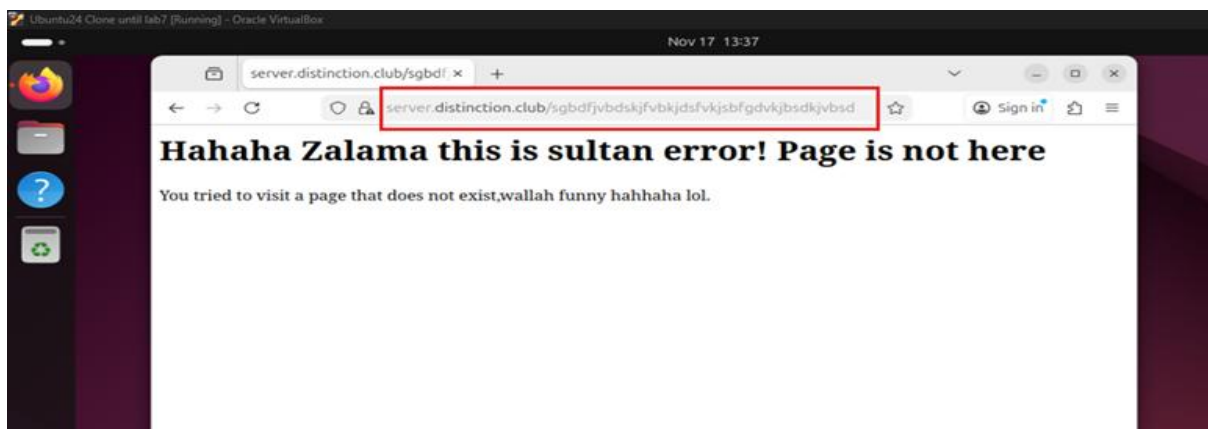
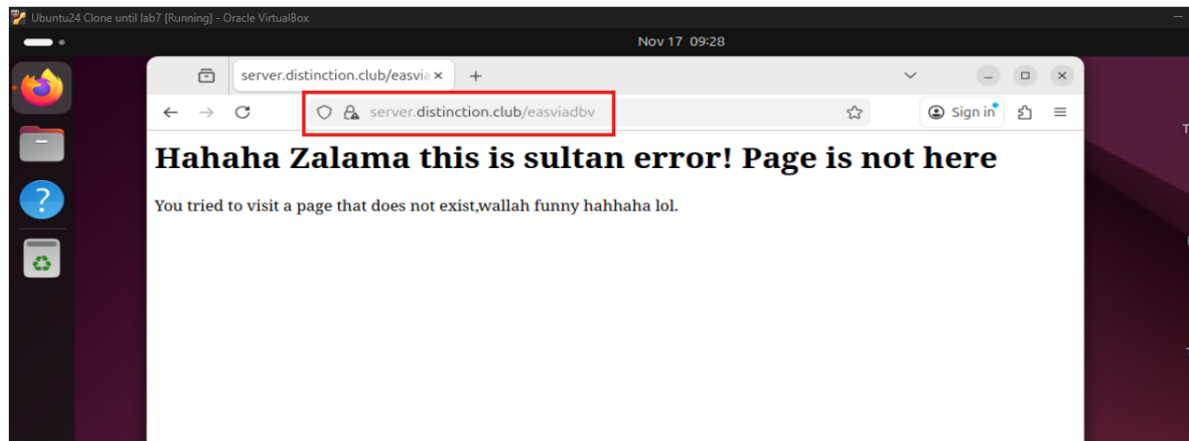


```
root@server:~  
[root@server ~]# systemctl restart httpd  
[root@server ~]#
```

A red arrow points to the command `systemctl restart httpd` in the terminal output.

STEP 7 :Test the Custom 404 Page

Try it on Ubuntu with putting random words after the slash of the Website:



6.3.5 Personal Reflection

My Own Reflection about what I did and learned from this:

Apache taught me that running a web server is more than just installing. Every small setting actually matters. When something broke, it wasn't random, I had to understand which file Apache was reading first, which rule was blocking something, or why a feature wasn't loading. Fixing those issues made me understand the server on a deeper level instead of just copying commands.

What I liked most is that I started thinking like someone who manages real servers. When HTTPS didn't work, I had to trace the certificate path. When the redirect failed, I had to check for conflicting VirtualHost files. When the 404-page kept asking for a password, I learned how Apache handles directories and aliases. Each problem forced me to figure out what Apache was actually doing behind the scenes.

6.3 Apache Additional Configurations

6.4 Samba File Server

CHAPTER 7: TROUBLESHOOTING

7.1 Stuck on a Grey Screen

This guide explains how I managed to fix and recover my Rocky Linux 9 virtual machine in Oracle VirtualBox after it got stuck on a gray screen and later showed a “No bootable device” error. Every step below follows the actual process I took, along with screenshots:

1. Problem - Rocky VM Stuck on a Gray Screen

When I started my Rocky Linux VM, it only displayed a gray screen with no login or desktop.

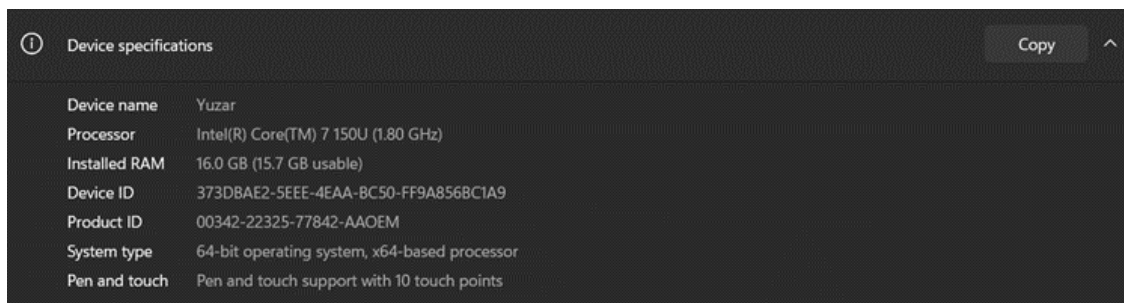
Figure 52 shows Rocky Linux 9 stuck on grey screen while booting up



2. Checking my Host Machine

Before troubleshooting the VM itself, I checked if my Windows host machine had enough resources to run it smoothly (Intel i7, 16GB RAM).

Figure 53 shows Host Device Specifications in Windows Settings



3. Realization – Rocky VM was Booting from the Installer ISO

On restart, I noticed the Rocky Linux installer **GRUB menu** appeared. That meant the ISO was still attached and the VM was booting from it instead of the installed system.

Figure 54 shows Rocky Linux 9 GRUB Bootloader



Introduction to GRUB

GRUB stands for **GRand Unified Bootloader**.

It is the standard, primary bootloader for most modern Linux distributions, including Rocky Linux. A bootloader is the first software program that runs when a computer (or VM) starts up, after the BIOS/UEFI firmware initializes the hardware.

Key Functions of GRUB:

- ❖ Locating the Kernel: Its main job is to locate the operating system kernel (the core of the OS) on the hard drive.
- ❖ Loading the Kernel: It then loads the kernel into the computer's memory.
- ❖ Transferring Control: Finally, it transfers control to the kernel, which then takes over the startup process and loads the rest of the operating system.
- ❖ Boot Menu: It presents a menu to the user, allowing them to choose which operating system or kernel version to start (figure 10). This is especially useful if you have multiple operating systems installed (a dual-boot setup) or need to access recovery options.

Rocky Boot Process Overview

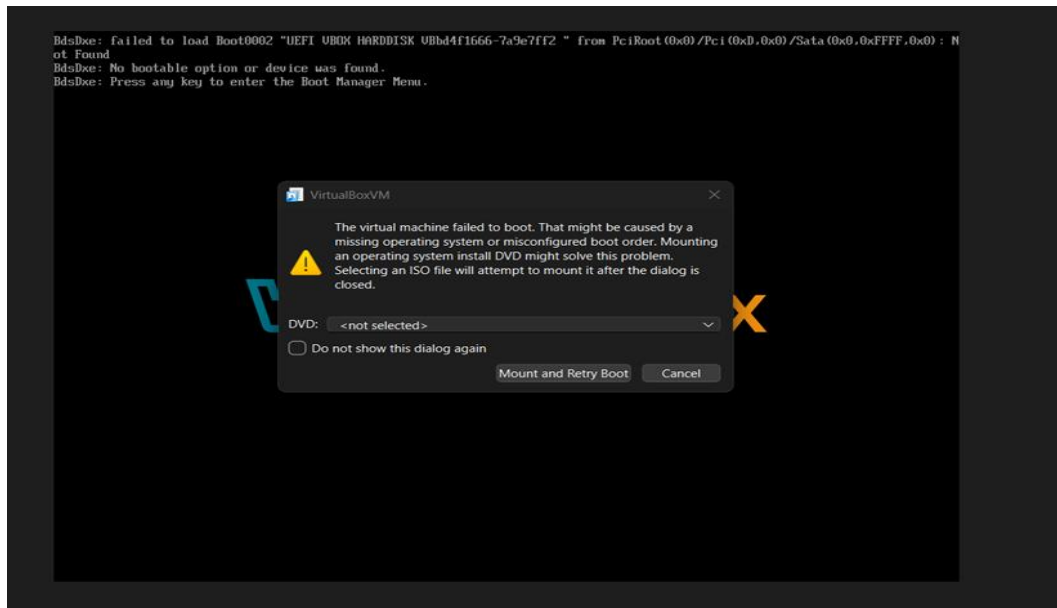
The simplified boot process usually follows these steps:

1. BIOS/UEFI runs.
2. BIOS/UEFI reads the boot order and loads the first sector of the boot device.
3. GRUB is loaded and appears on the screen.
4. GRUB loads the Rocky Linux Kernel.
5. The Kernel loads the rest of the Operating System.

4. “No Bootable Device” Error

After detaching the **ISO** and rebooting, VirtualBox showed “No bootable device found.” This meant the OS was installed, but the bootloader (EFI/GRUB) was broken.

Figure 55 shows VirtualBox prompting the user to mount an Operating System ISO Image



What is an ISO Image?

An ISO image is a type of disk image file that contains an identical, sector-by-sector copy of all the data and structure found on an optical disc, such as a CD, DVD, or Blu-ray.

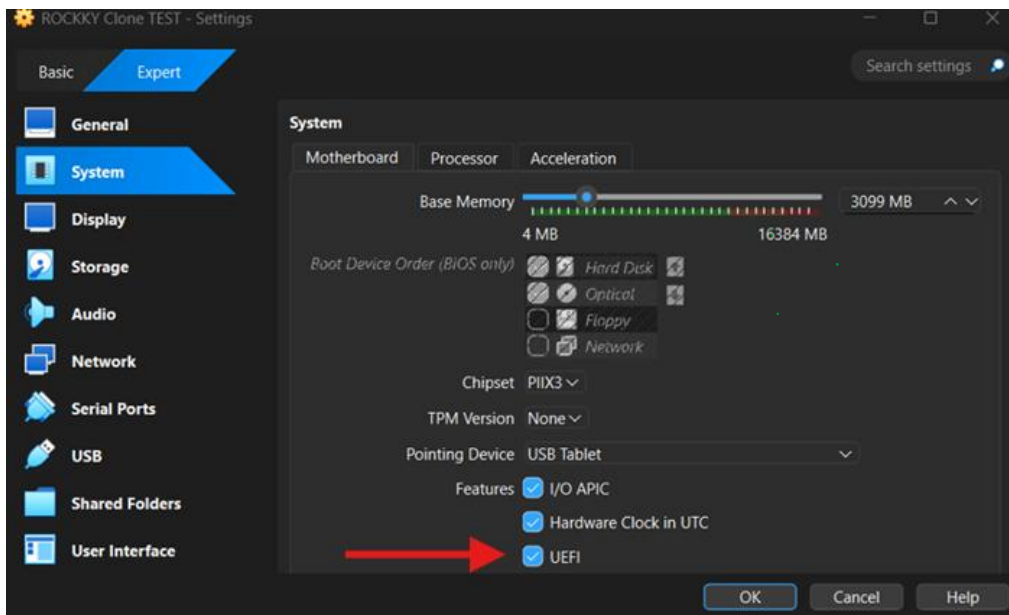
Key Characteristics of an ISO File:

- ❖ Complete Copy: It captures not only all the individual files and folders but also the disc's original file system, metadata, and boot information, ensuring it is a perfect digital replica.
- ❖ Uncompressed: Unlike a ZIP or RAR file, an ISO image is typically not compressed. It contains the raw, binary data, meaning its size is nearly identical to the original physical disc's content.

5. Checking VirtualBox Settings

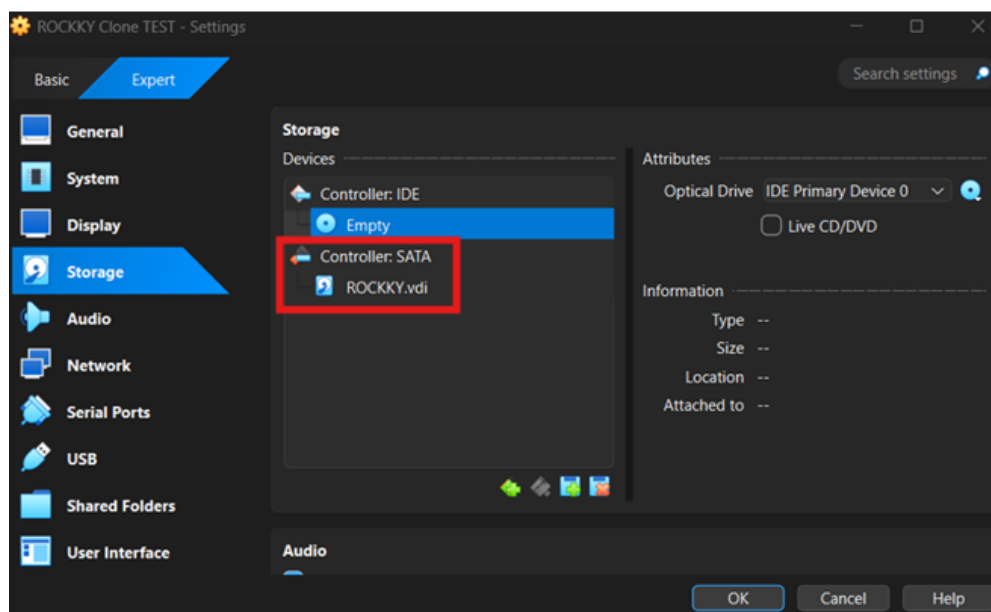
I opened the VM settings to confirm everything was still configured properly:

Figure 56 shows UEFI enabled for Rocky Linux Virtual Machine



In the System tab, UEFI boot was turned on.

Figure 57 shows VirtualBox Storage Settings for Rocky Linux



The Storage tab showed that the primary .vdi virtual disk was still properly attached.

I reattached the Rocky ISO and selected “Rescue a Rocky Linux system” from the boot menu. This allows repairing the system from within the rescue environment.

Figure 58 shows Rescue Shell Environment finding the Linux Installation and mounting it

```
Starting installer, one moment...
anaconda 34.25.3.8-1.el9.rocky.0.3 for Rocky Linux 9.3 started.
 * installation log files are stored in /tmp during the installation
 * shell is available on TTY2
 * when reporting a bug add logs from /tmp as separate text/plain attachments
=====
Rescue

The rescue environment will now attempt to find your Linux installation and
mount it under the directory : /mnt/sysroot. You can then make any changes
required to your system. Choose '1' to proceed with this step.
You can choose to mount your file systems read-only instead of read-write by
choosing '2'.
If for some reason this process does not work choose '3' to skip directly to a
shell.

1) Continue
2) Read-only mount
3) Skip to shell
4) Quit (Reboot)

Please make a selection from the above: 1
=====
Rescue Shell

Your system has been mounted under /mnt/sysroot.

If you would like to make the root of your system the root of the active system,
run the command:

    chroot /mnt/sysroot

When finished, please exit from the shell and your system will reboot.

Please press ENTER to get a shell:
bash-5.1#
bash-5.1# chroot /mnt/sysimage
grub2-install: error: /usr/lib/grub/x86_64-efi/modinfo.sh doesn't exist. Please specify --target or --directory.
bash-5.1# grub2-install /dev/sda
grub2-install: error: /usr/lib/grub/x86_64-efi/modinfo.sh doesn't exist. Please specify --target or --directory.
bash-5.1# _

[anaconda1:main* 2:shell 3:log 4:storage-log 5:program-log
```

7. First Attempt at chroot - Failed

I tried running 'chroot /mnt/sysimage', but it failed because the correct path wasn't mounted yet.

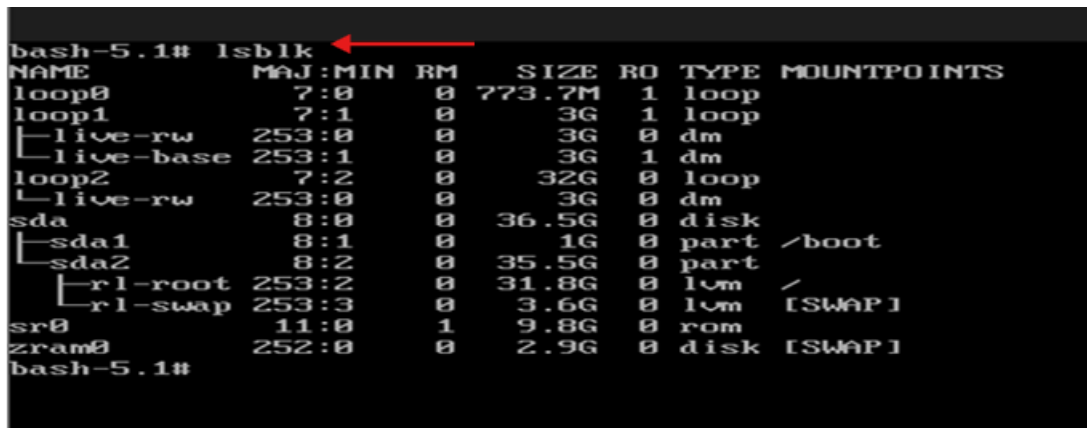
Figure 59 shows chroot /mnt/sysimage failed because the directory doesn't exist.

```
bash-5.1# chroot /mnt/sysimage
chroot: cannot change root directory to '/mnt/sysimage': No such file or directory
bash-5.1# _
```

8. Checking Disk and LVM Setup

To understand the layout, I ran the command '**lsblk**'. It showed /dev/sda1 as /boot and **rl-root** (**LVM** volume) as the main root filesystem.

Figure 60 shows information about all available block devices



```
bash-5.1# lsblk
NAME                MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0                 7:0      0 773.7M 1 loop
loop1                 7:1      0    3G   1 loop
└─live-rw             253:0     0    3G   0 dm
└─live-base           253:1     0    3G   1 dm
loop2                 7:2      0   32G   0 loop
└─live-rw             253:0     0    3G   0 dm
sda                   8:0      0  36.5G  0 disk
├─sda1                 8:1      0    1G   0 part /boot
├─sda2                 8:2      0  35.5G  0 part
│   └─rl-root           253:2     0  31.8G  0 lvm /
│       └─rl-swap        253:3     0    3.6G  0 lvm [SWAP]
sr0                   11:0      1   9.8G  0 rom
zram0                 252:0     0    2.9G  0 disk [SWAP]
bash-5.1#
```

lsblk Command

lsblk stands for "**list block devices**."

lsblk is a Linux command-line utility used to display information about all available block devices (storage devices) on your system. It presents the output in a clean, tree-like format, showing the relationship between a physical disk, its partitions, and any logical volumes created on them.

LVM (Logical Volume Manager)

LVM is a storage management technology that creates a flexible layer of abstraction over physical storage devices. Instead of directly interacting with physical partitions, LVM allows you to create logical volumes that can be easily resized, moved, and managed.

10. Repairing GRUB (and encountering errors)

I started repairing GRUB, but a few mistakes and obstacles happened along the way:

- I Combined two commands which lead to the error: **invalid option -- 'o'**

Figure 61 shows invalid option -- 'o' error

```
bash-5.1# grub2-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=rocky --recheck grub2-mkconfig -o /boot/efi/EFI/rocky/grub.cfg
grub2-install: invalid option -- 'o'
Try 'grub2-install --help' or 'grub2-install --usage' for more information.
bash-5.1#
```

- Running grub2-install failed because **modinfo.sh is missing**, which means the EFI GRUB modules are not present.

Figure 62 shows modinfo.sh doesn't exist error

```
bash-5.1# grub2-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=rocky --recheck
grub2-install: error: /usr/lib/grub/x86_64-efi/modinfo.sh doesn't exist. Please specify --target or --directory.
bash-5.1#
```

- The attempt to run **dnf reinstall grub2-efi** didn't work because there was no internet connection.

Figure 63 shows error: Failed to download metadata for repo 'baseos'

```
bash-5.1# dnf reinstall grub2-efi-x86_64 grub2-efi-x86_64-modules shim-x86_64 -y
Rocky Linux 9 - BaseOS
Errors during downloading metadata for repository 'baseos':
 - Curl error (6): Couldn't resolve host name for https://mirrors.rockylinux.org/mirrorlist?arch=x86_64&repo=BaseOS-9
Error: Failed to download metadata for repo 'baseos': Cannot prepare internal mirrorlist: Curl error (6): Couldn't resolve host: mirrors.rockylinux.org
bash-5.1#
```

- The **grub2-install** command still fails even when the **--no-nvram** option is used.

Figure 64 shows error: modinfo.sh does not exist

```
bash-5.1# mkdir -p /boot/grub2/x86_64-efi
bash-5.1# grub2-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=rocky --recheck --no-nvram
grub2-install: error: /usr/lib/grub/x86_64-efi/modinfo.sh doesn't exist. Please specify --target or --directory.
bash-5.1#
```

11. Realization: Wrong Shell Context

When running 'lvm vgchange -ay', it complained about /proc not being available — meaning I was in the wrong shell.

Figure 65 shows fopen failed: No such file or directory error in bash-5.1 terminal

```
bash-5.1# lvm vgchange -ay
/proc/mounts: _get_sysfs_dir fopen failed: No such file or directory
/proc/devices: fopen failed: No such file or directory
bash-5.1# _
```

12. The Fix

Exit to the rescue menu, choose Option 1 (Continue) so the system mounts under /mnt/sysimage, then run the following commands:

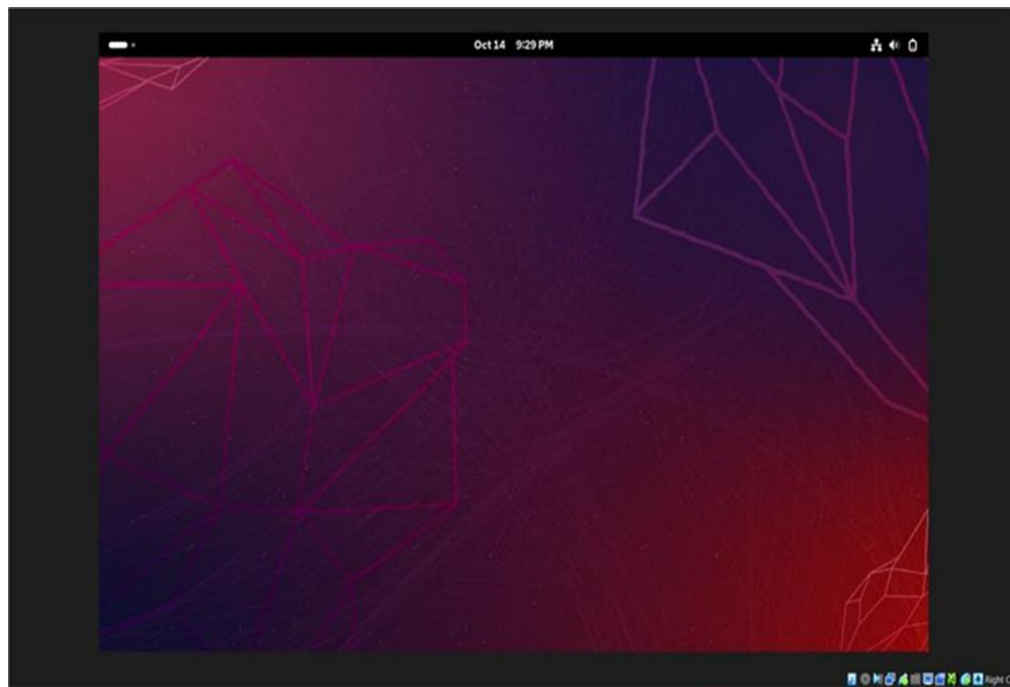
```
chroot /mnt/sysimage
```

```
grub2-install
```

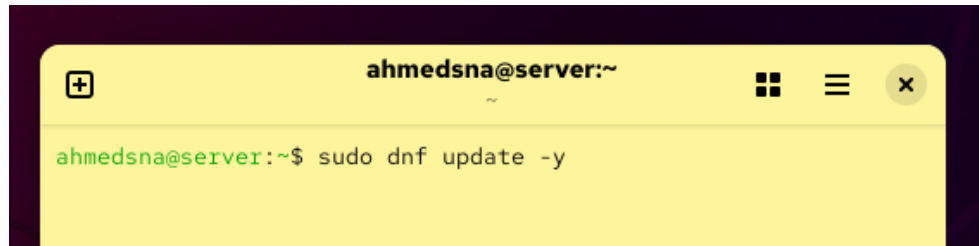
```
grub2-mkconfig -o /boot/grub2/grub.cfg
```

After rebooting, Rocky Linux finally loaded correctly and brought up the GNOME desktop.

Figure 66 shows Rocky Linux GNOME desktop loading appropriately



13. Final Stabilization and Updates



To ensure system stability and avoid gray screen issues, I ran updates and installed the following necessary tools:

i. **sudo dnf update -y**

This command securely updates all installed software packages on the system to their newest versions without requiring the user to confirm the action.

ii. **sudo dnf install -y kernel-devel kernel-headers gcc make perl bzip2 elfutils-libelf-devel dkms**

This command Installs the necessary development tools and kernel sources (kernel-devel, kernel-headers) required to compile software, like the official VMware Tools kernel modules.

iii. **sudo dnf install -y open-vm-tools open-vm-tools-desktop**

This command Installs the Open-VM-Tools package and its graphical components. This is the recommended open-source alternative to proprietary VMware Tools for improved performance and features in a virtual machine.

iv. **sudo systemctl enable --now vmtoolsd**

This command Enables and starts the Open-VM-Tools service (vmtoolsd) immediately and ensures it starts automatically on future reboots.

v. **sudo systemctl set-default graphical.target**

This command Sets the system to boot into the graphical environment (GUI) by default, instead of the command-line interface.

7.2 Unable to install dhcp-server

1. The Issue

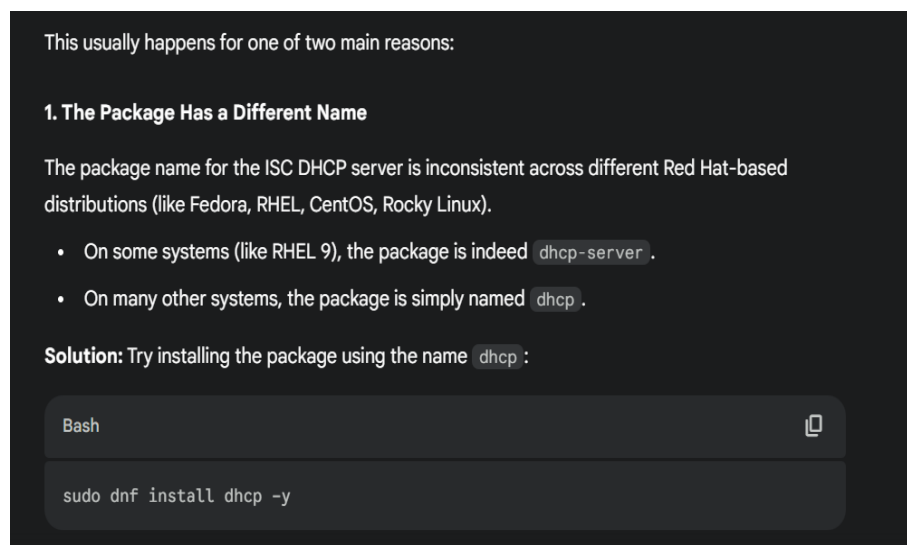
I am Unable to install the DHCP on Rocky Linux because the package dhcp-server fails to download or resolve during installation.

```
ahmedsna@server:~$ sudo dnf install dhcp-server -y
Last metadata expiration check: 0:00:55 ago on Wed 05 Nov 2025 06:17:13 PM.
No match for argument: dhcp-server
Error: Unable to find a match: dhcp-server
ahmedsna@server:~$
```

As shown in the above picture there is no match for argument: dhcp-server

2. Identifying the Cause

Let's upload this image to Gemini to brainstorm solutions



The 1st suggestion says to run the dhcp-server package with a different name “dhcp” (not dhcp-server). So, let's try installing the dhcp package with the new package name:

“sudo dnf install dhcp -y”

```
ahmedsna@server:~$ sudo dnf install dhcp -y
[sudo] password for ahmedsna:
Last metadata expiration check: 0:13:44 ago on Wed 05 Nov 2025 06:17:13 PM.
No match for argument: dhcp
Error: Unable to find a match: dhcp
ahmedsna@server:~$
```

hmm, same issue again... Unable to find a match: dhcp

Let's move on to the 2nd reason as to why this could be happening

2. The Package is Obsolete (Replaced by Kea)

The traditional ISC DHCP server (the package you're trying to install) was officially declared **End-of-Life (EOL)** in late 2022.

Many modern Linux distributions are in the process of replacing it with a new, actively maintained DHCP server called **Kea**. If your system is very up-to-date, the old `dhcp-server` package may no longer be in the repositories at all.

Solution: If the command above also fails, you should search for and install Kea instead:

Bash

```
# First, search for the Kea packages
dnf search kea

# Then, install the relevant packages (the names might be kea, kea-dhcp4, etc.)
sudo dnf install kea -y
```

Running the command “dnf search kea”

```
ahmedsna@server:~$ dnf search kea
Extra Packages for Enterprise Linux 10 - x86_64 502 kB/s | 4.8 MB      00:09
Rocky Linux 10 - CRB                          265 kB/s | 532 kB      00:02
===== Name Exactly Matched: kea =====
kea.x86_64 : DHCPv4, DHCPv6 and DDNS server from ISC
===== Name & Summary Matched: kea =====
kea-doc.noarch : Documentation for Kea DHCP server
kea-hooks.x86_64 : Hooks libraries for kea
kea-keama.x86_64 : Experimental migration assistant for Kea
kea-libs.x86_64 : Shared libraries used by Kea DHCP server
===== Name Matched: kea =====
golang-github-bgentry-speakasy-devel.noarch : Cross-platform golang helpers for
...: reading password input without cgo
ahmedsna@server:~$ █
```

Okay I have found kea packages that support DHCPv4

2. Verifying OS Version

```

ahmedsna@server:~$ cat /etc/os-release
NAME="Rocky Linux"
VERSION="10.0 (Red Quartz)"
ID="rocky"
ID_LIKE="rhel centos fedora"
VERSION_ID="10.0"
PLATFORM_ID="platform:el10"
PRETTY_NAME="Rocky Linux 10.0 (Red Quartz)"
ANSI_COLOR="0;32"
LOGO="fedora-logo-icon"
CPE_NAME="cpe:/o:rocky:rocky:10::baseos"
HOME_URL="https://rockylinux.org/"
VENDOR_NAME="RESF"
VENDOR_URL="https://resf.org/"
BUG_REPORT_URL="https://bugs.rockylinux.org/"
SUPPORT_END="2035-05-31"
ROCKY_SUPPORT_PRODUCT="Rocky-Linux-10"
ROCKY_SUPPORT_PRODUCT_VERSION="10.0"
REDHAT_SUPPORT_PRODUCT="Rocky Linux"
REDHAT_SUPPORT_PRODUCT_VERSION="10.0"
ahmedsna@server:~$

```

By running the command: `cat /etc/os-release`, I realized that I have installed Rocky Linux 10 which does not support dhcp-server package anymore. The dhcp-server package has now been superseded by the new Kea Packages in Rocky Linux 10. Since the dhcp-server package is not supported by Rocky Linux 10 anymore, I'll proceed with the installation of Kea DHCPv4.

3. Into the Teams Troubleshooting Forum

Troubleshooting Forum Posts Files Notes

ELYAS ABDURABU NASSER 10/25 12:08 PM
[SOLVED]

Reply

ABDUBOKIEV MIRZOKHAMDAM ISKANDAR UGLI 10/27 12:16 PM

New DHCP commands on Rocky 10

If you are using rocky 10, you might face some issues installing dhcp, because command changed to the kea, so you need to type different commands than slides in according to install it.

Task	Old Command (Rocky 9)	New Command (Rocky 10)
Install Package	<code>sudo dnf install dhcp-server -y</code>	<code>sudo dnf install kea -y</code>
Service Name	dhcpd.service	kea-dhcp4.service
Start & Enable	<code>sudo systemctl enable --now dhcpd</code>	<code>sudo systemctl enable --now kea-dhcp4</code>
Check Status	<code>sudo systemctl status dhcpd</code>	<code>sudo systemctl status kea-dhcp4</code>
Restart Service	<code>sudo systemctl restart dhcpd</code>	<code>sudo systemctl restart kea-dhcp4</code>
Config File Path	<code>/etc/dhcp/dhcpd.conf</code>	<code>/etc/kea/kea-dhcp4.conf</code>

After scrolling through the Troubleshooting Forum on teams I found that someone has already uploaded new Commands to install and configure DHCP Kea service on Rocky Linux 10.

Task	Old Command (Rocky 9)	✓ New Command (Rocky 10)
Install Package	<code>sudo dnf install dhcp-server -y</code>	<code>sudo dnf install kea -y</code>
Service Name	<code>dhcpd.service</code>	<code>kea-dhcp4.service</code>
Start & Enable	<code>sudo systemctl enable --now dhcpd</code>	<code>sudo systemctl enable --now kea-dhcp4</code>
Check Status	<code>sudo systemctl status dhcpd</code>	<code>sudo systemctl status kea-dhcp4</code>
Restart Service	<code>sudo systemctl restart dhcpd</code>	<code>sudo systemctl restart kea-dhcp4</code>
Config File Path	<code>/etc/dhcp/dhcpd.conf</code>	<code>/etc/kea/kea-dhcp4.conf</code>

With the help of this table, I can map the dhcp-server commands of Rocky 9 in lab to the new Kea DHCPv4 Rocky 10 commands.

3. Installing Kea DHCPv4 server

I check if the new Kea DHCPv4 is installed on my Rocky Linux 10, by running the command: `'sudo dnf install kea -y'`.

```
ahmedsna@server:~$ sudo dnf install kea -y
Extra Packages for Enterprise Linux 10 - x86_64 902 B/s | 9.8 kB 00:11
Extra Packages for Enterprise Linux 10 - x86_64 78 kB/s | 4.8 MB 01:03
Rocky Linux 10 - BaseOS 98 B/s | 4.3 kB 00:45
Rocky Linux 10 - BaseOS 451 kB/s | 23 MB 00:52
Rocky Linux 10 - AppStream 1.7 kB/s | 4.3 kB 00:02
Rocky Linux 10 - CRB 4.0 kB/s | 4.3 kB 00:01
Rocky Linux 10 - CRB 318 kB/s | 532 kB 00:01
Rocky Linux 10 - Extras 2.5 kB/s | 3.1 kB 00:01
Package kea-2.6.3-1.el10_0.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
```

After Installation of Kea DHCPv4 server I now need to Configure the kea DHCPv4 conf file

4. Creating the kea Configuration file

To create the kea configuration file, I type in the command:

- `sudo nano /etc/kea/kea-dhcp4.conf`

```

ahmedsna@server:~ – sudo nano /etc/kea/kea-dhcp4.conf
GNU nano 8.1 /etc/kea/kea-dhcp4.conf
{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [
        "enp0s3"
      ]
    },
    "control-socket": {
      "socket-type": "unix",
      "socket-name": "/run/kea/kea4-ctrl-socket"
    },
    "lease-database": {
      "type": "memfile",
      "lfc-interval": 3600
    },
    "valid-lifetime": 3600,
    "max-valid-lifetime": 86400,
    "subnets": [
      {
        "id": 1,
        "subnet": "192.168.10.0/24",

```

Terminal window showing the nano text editor editing the file `/etc/kea/kea-dhcp4.conf`. The file content is in JSON format, defining DHCPv4 configuration including interfaces, control socket, lease database, and a subnet. A red bracket highlights the 'subnets' section.

After opening the file, I noticed that the file's configurations are all in JavaScript Object Notation (JSON) format.

This is a key characteristic of the Kea DHCP server, as it relies on JSON for all its configuration settings, which allows for a flexible, human-readable, and machine-parsable structure, unlike the plain text or key-value format used by some older DHCP servers.

```

GNU nano 8.1 /etc/kea/kea-dhcp4.conf
{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [
        "enp0s3"
      ]
    },
    "control-socket": {
      "socket-type": "unix",
      "socket-name": "/run/kea/kea4-ctrl-socket"
    },
    "lease-database": {
      "type": "memfile",
      "lfc-interval": 3600
    },
    "valid-lifetime": 3600,
    "max-valid-lifetime": 86400,
    "subnet4": [
      {
        "id": 1,
        "subnet": "192.168.10.0/24",

```

Terminal window showing the Kea DHCP4 configuration file in nano editor. The file is /etc/kea/kea-dhcp4.conf. The configuration is in JSON format. Red arrows point to specific parts: 'enp0s3' in the interfaces array, 'lease-database' object, and the 'subnet4' array.

The Kea Configuration file shows several standard JSON structures:

- **Objects:** The configuration is built around nested **objects** enclosed in curly braces `{ }` like "Dhcp4", "interfaces-config", and "subnet4".
- **Key-Value Pairs:** Each configuration setting is a **key-value pair** separated by a colon `:`, such as `"type": "memfile"` or `"subnet": "192.168.10.0/24"`.
- **Arrays:** The configuration for multiple interfaces or subnets is handled using **arrays** enclosed in square brackets `[ ]`, such as the `["enp0s3"]` under "interfaces" and the array of subnet objects under "subnet4".

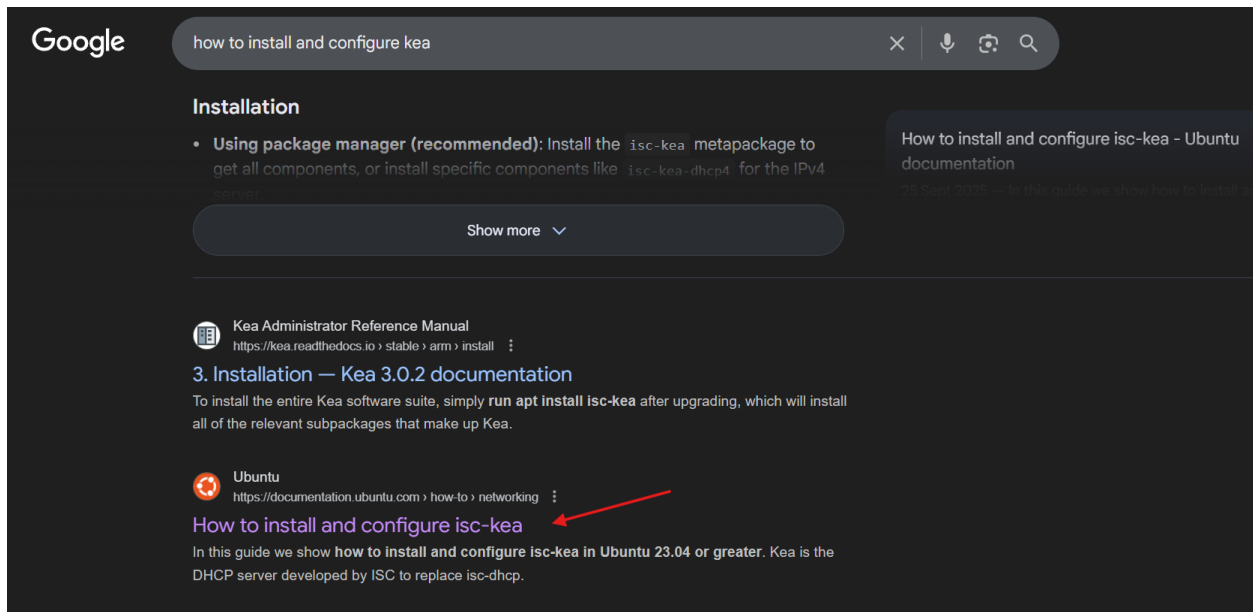
5. Kea-dhcp4 JSON Template

To get the Template for kea-dhcp4.conf file, I searched on google:

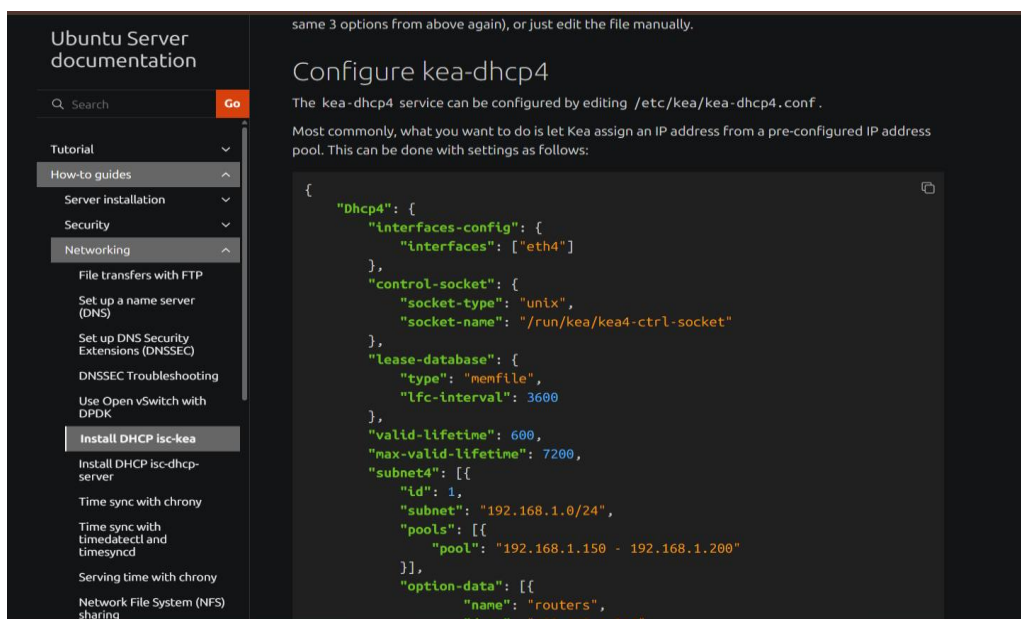
“How to install and configure kea” and clicked on the following link:

URL: <https://documentation.ubuntu.com/server/how-to/networking/install-isc-kea/>

Note: Open the URL inside the Mozilla Firefox Browser of Rocky Server only



After scrolling down the Ubuntu Server documentation webpage, I found the template to configure kea-dhcp4 service.



But the given template does not have same values as per my Network Configuration so to solve this I again make use of Gemini by copying the JSON Configuration Template and adding the following prompt to it:

```
#####
Reconfigure the above JSON Configuration with the following Specifications

default-lease-time 3600;
max-lease-time 86400;
authoritative;
subnet 192.168.10.0 netmask 255.255.255.0 {
range 192.168.10.80 192.168.10.90;
option routers 192.168.10.1;
option subnet-mask 255.255.255.0;
option domain-name-servers 192.168.10.4, 8.8.8.8;
network interface enp0s3
```

Below is the JSON configuration for my Kea-dhcp4.conf file

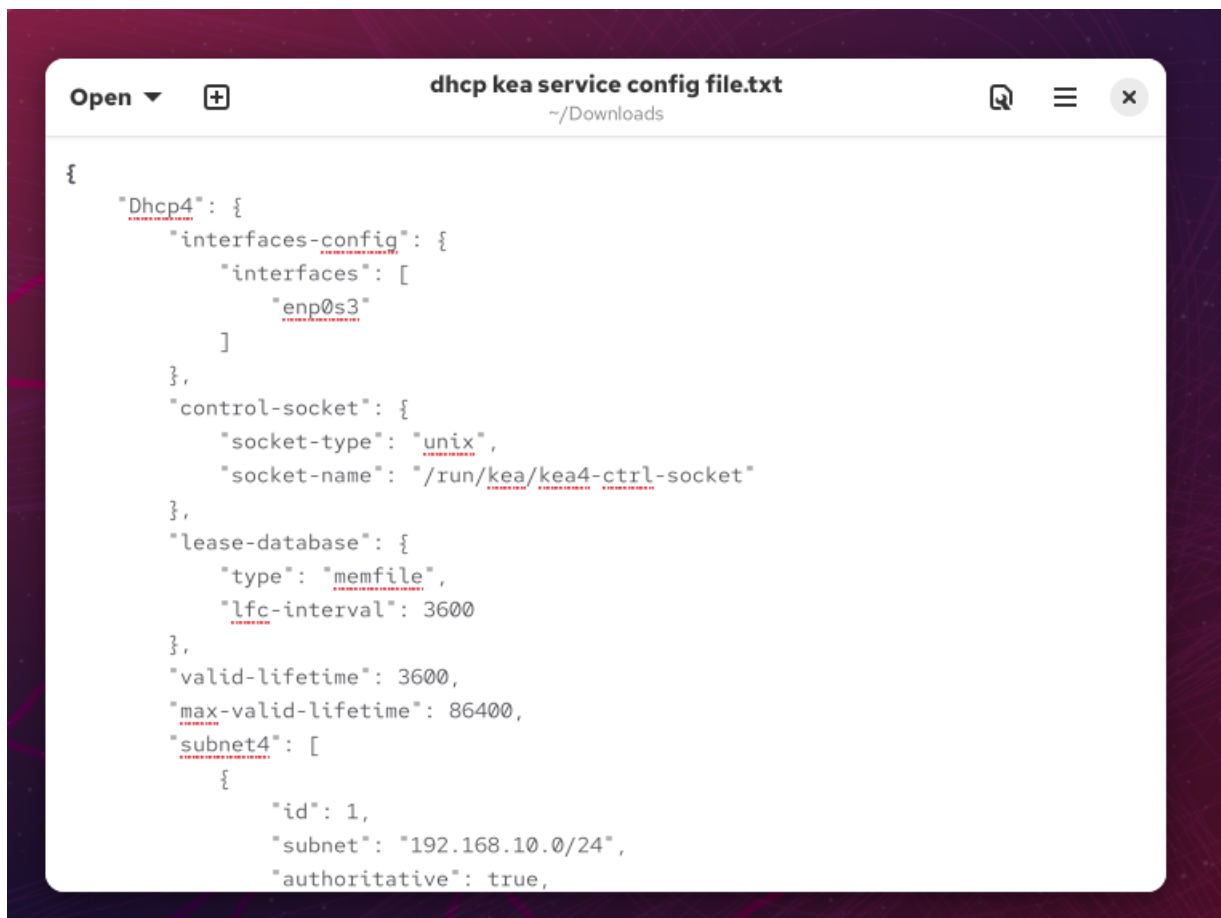
```
{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [
        "enp0s3"
      ]
    },
    "control-socket": {
      "socket-type": "unix",
      "socket-name": "/run/kea/kea4-ctrl-socket"
    },
    "lease-database": {
      "type": "memfile",
      "lfc-interval": 3600
    },
    "valid-lifetime": 3600,
    "max-valid-lifetime": 86400,
    "subnet4": [
      {
        "id": 1,
        "subnet": "192.168.10.0/24",
        "authoritative": true,
        "pools": [
          {
            "pool": "192.168.10.80 - 192.168.10.90"
          }
        ]
      }
    ],
    "option-data": [
```

```

{
  "name": "routers",
  "data": "192.168.10.1"
},
{
  "name": "subnet-mask",
  "data": "255.255.255.0"
},
{
  "name": "domain-name-servers",
  "data": "192.168.10.4, 8.8.8.8"
}
]
}
]
}
}

```

To keep the JSON formatting clean and valid, I copied the configuration from Gemini into a local text editor on Rocky first.



The screenshot shows a text editor window titled "dhcp kea service config file.txt" with the file path "~/Downloads". The editor contains a JSON configuration for a DHCP service. The configuration includes settings for the DHCP4 service, such as interfaces configuration, control socket, lease database, valid lifetime, and a subnet configuration.

```

{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [
        "enp0s3"
      ]
    },
    "control-socket": {
      "socket-type": "unix",
      "socket-name": "/run/kea/kea4-ctrl-socket"
    },
    "lease-database": {
      "type": "memfile",
      "lfc-interval": 3600
    },
    "valid-lifetime": 3600,
    "max-valid-lifetime": 86400,
    "subnet4": [
      {
        "id": 1,
        "subnet": "192.168.10.0/24",
        "authoritative": true,

```

After confirming Gemini's output to match my network specifications, I went back to the Rocky terminal, opened **kea-dhcp4.conf**, and pasted the JSON configuration into it.

```

ahmedsna@server:~ -- sudo nano /etc/kea/kea-dhcp4.conf
GNU nano 8.1 /etc/kea/kea-dhcp4.conf
{
  "subnet4": [
    {
      "id": 1,
      "subnet": "192.168.10.0/24",
      "authoritative": true,
      "pools": [
        {
          "pool": "192.168.10.80 - 192.168.10.90"
        }
      ],
      "option-data": [
        {
          "name": "routers",
          "data": "192.168.10.1"
        },
        {
          "name": "subnet-mask",
          "data": "255.255.255.0"
        },
        {
          "name": "domain-name-servers",
          "data": "192.168.10.4, 8.8.8.8"
        }
      ]
    }
  ]
}
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify

```

id: 1 (Subnet identifier).

subnet: 192.168.10.0/24 (The network address and CIDR prefix length).

authoritative: true. This means the DHCP server is the primary source of configuration for this subnet and should respond to requests even if they appear to be invalid or for expired leases.

IP Address Pool: The range of IP addresses that the server will dynamically assign to clients, which is 192.168.10.80 - 192.168.10.90

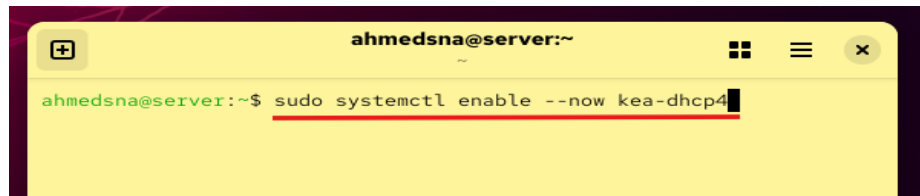
routers (Default Gateway): 192.168.10.1

subnet-mask: 255.255.255.0 (Corresponds to the /24 prefix).

domain-name-servers (DNS Servers): 192.168.10.4, 8.8.8.8 (The client will use these two addresses for DNS resolution).

6. Running the Kea-dhcp4 Service

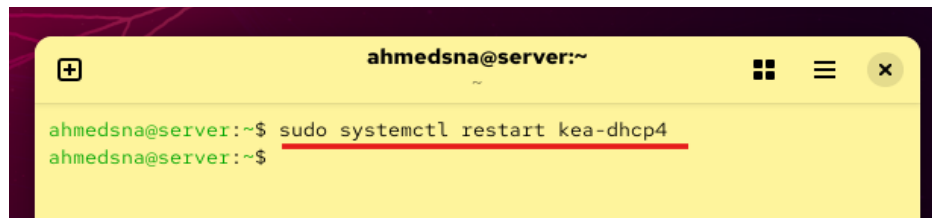
After configuring the kea-dhcp4.conf file, I now need to enable the kea-dhcp4 service.

A terminal window titled 'ahmedsna@server:~' with a yellow background. The command 'sudo systemctl enable --now kea-dhcp4' is entered and highlighted with a red underline. The cursor is at the end of the command.

```
ahmedsna@server:~$ sudo systemctl enable --now kea-dhcp4
```

- `sudo systemctl enable --now kea-dhcp4`

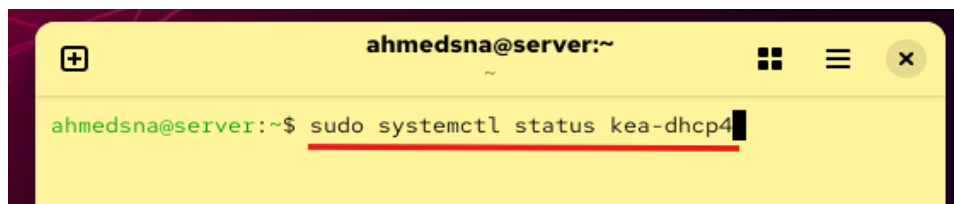
Explanation: This command permanently **enables** the kea-dhcp4 service, ensuring it starts automatically every time the system boots. The added flag, `--now`, tells systemctl to start the service immediately as well, without needing a separate start command.

A terminal window titled 'ahmedsna@server:~' with a yellow background. The command 'sudo systemctl restart kea-dhcp4' is entered and highlighted with a red underline. The prompt 'ahmedsna@server:~\$' is visible on the next line.

```
ahmedsna@server:~$ sudo systemctl restart kea-dhcp4
ahmedsna@server:~$
```

- `sudo systemctl restart kea-dhcp4`

Explanation: This command **restarts** the kea-dhcp4 service immediately. It's typically used after making changes to the KEA configuration file to ensure the running service reloads the new settings.

A terminal window titled 'ahmedsna@server:~' with a yellow background. The command 'sudo systemctl status kea-dhcp4' is entered and highlighted with a red underline. The cursor is at the end of the command.

```
ahmedsna@server:~$ sudo systemctl status kea-dhcp4
```

- `sudo systemctl status kea-dhcp4`

Explanation: This command **displays the current operational status** of the kea-dhcp4 service. It provides details like whether the service is **active** (running) or stopped, its process ID (PID), recent log output, and how long it has been running. This is the main way to check the health of the DHCP server.

```

ahmedsna@server:~ - sudo systemctl status kea-dhcp4
● kea-dhcp4.service - Kea DHCPv4 Server
   Loaded: loaded (/usr/lib/systemd/system/kea-dhcp4.service; enabled; preset: disabled)
   Active: active (running) since Mon 2025-11-17 00:10:19 +08; 2min 42s ago
 Invocation: 365c0f4081fb4bceb652c05273778b6d
    Docs: man:kea-dhcp4(8)
   Main PID: 9598 (kea-dhcp4)
     Tasks: 11 (limit: 67504)
    Memory: 2.6M (peak: 6M)
       CPU: 228ms
    CGroup: /system.slice/kea-dhcp4.service
            └─9598 /usr/sbin/kea-dhcp4 -c /etc/kea/kea-dhcp4.conf

Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.513 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.514 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.520 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.525 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.525 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.525 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.525 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.555 INFO [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.560 WARN [kea-d>
Nov 17 00:10:19 server.distinction.club kea-dhcp4[9598]: 2025-11-17 00:10:19.561 INFO [kea-d>
~
~

```

The **Active (running)** status confirms KEA DHCPv4 server is now successfully running on Rocky Linux 10 and configured to start whenever the system boots.

THE END 😊

7.3 Mail Server Troubleshooting

7.3.1 Failed Mail Delivery

During setup, several common issues were encountered. **Initially, mail delivery failed because Postfix was delivering messages to the old 'var/spool/mail' location.** This was corrected by setting 'home_mailbox = Maildir/' in *main.cf*. Permissions for each user's Maildir were fixed using *chown* and *chmod*.

Figure 67 Creation of Maildir directories for user accounts

```
[moha@server ~]$ cat /etc/passwd | grep home
moha:x:1000:1000:moha:/home/moha:/bin/bash
[moha@server ~]$ sudo useradd user1
[moha@server ~]$ sudo passwd user1
Changing password for user user1.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[moha@server ~]$ sudo useradd user2
[moha@server ~]$ sudo passwd user2
Changing password for user user2.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[moha@server ~]$ sudo mkdir -p /home/user1/Maildir/{cur,new,tmp}
sudo mkdir -p /home/user2/Maildir/{cur,new,tmp}
sudo chown -R user1:user1 /home/user1/Maildir
sudo chown -R user2:user2 /home/user2/Maildir
sudo chmod -R 700 /home/user*/Maildir
chmod: cannot access '/home/user*/Maildir': No such file or directory
[moha@server ~]$
```

Figure 68 Postfix and Dovecot active ports

```
[moha@server ~]$ sudo postconf | grep home_mailbox
home_mailbox = Maildir/
[moha@server ~]$ sudo postconf -e "home_mailbox = Maildir/"
[moha@server ~]$ sudo systemctl restart dovecot postfix
[moha@server ~]$ sudo postconf | grep home_mailbox
home_mailbox = Maildir/
[moha@server ~]$ sudo -u user1 /usr/bin/mail -s "Maildir Test" user2@server.distinction.club <<< "Testing maildir delivery"
sudo -u user2 /usr/bin/mail
s-nail: /var/spool/mail/moha: Permission denied
[moha@server ~]$
```

Figure 69 Confirmed Maildir message files

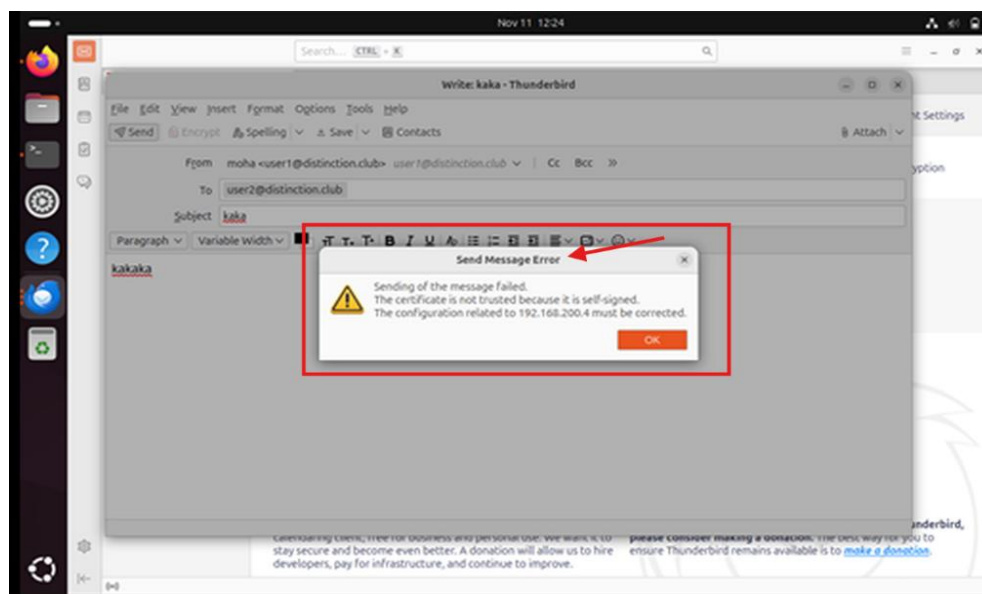
```
[moha@server ~]$ sudo postconf | grep home_mailbox
home_mailbox = Maildir/
[moha@server ~]$ sudo postconf | grep home_mailbox
home_mailbox = Maildir/
[moha@server ~]$ sudo ls -l /home/user2/Maildir
total 4
drwx-----. 2 user2 user2 6 Nov 11 11:51 cur
drwx-----. 2 user2 user2 4096 Nov 11 12:08 new
drwx-----. 2 user2 user2 6 Nov 11 12:08 tmp
[moha@server ~]$ sudo ls -l /home/user2/Maildir/new
total 16
-rw-----. 1 user2 user2 501 Nov 11 11:55 1762833304.Vfd00I116058eM726608.server.distinction.club
-rw-----. 1 user2 user2 501 Nov 11 11:56 1762833410.Vfd00I11605b7M214961.server.distinction.club
-rw-----. 1 user2 user2 491 Nov 11 12:04 1762833879.Vfd00I11605aeM13683.server.distinction.club
-rw-----. 1 user2 user2 496 Nov 11 12:08 1762834111.Vfd00I11605a7M308464.server.distinction.club
[moha@server ~]$ sudo -u user1 /usr/sbin/sendmail user2@server.distinction.club <<Eor
Subject: Maildir Delivery Test
From: user1@server.distinction.club

This is a test message to check Maildir delivery.
EOF
[moha@server ~]$ sudo ls -l /home/user2/Maildir/new
total 20
-rw-----. 1 user2 user2 501 Nov 11 11:55 1762833304.Vfd00I116058eM726608.server.distinction.club
-rw-----. 1 user2 user2 501 Nov 11 11:56 1762833410.Vfd00I11605b7M214961.server.distinction.club
-rw-----. 1 user2 user2 491 Nov 11 12:04 1762833879.Vfd00I11605aeM13683.server.distinction.club
-rw-----. 1 user2 user2 496 Nov 11 12:08 1762834111.Vfd00I11605a7M308464.server.distinction.club
-rw-----. 1 user2 user2 476 Nov 11 12:14 1762834493.Vfd00I11605a9M67003.server.distinction.club
[moha@server ~]$
```

7.3.2 Thunderbird Self-signed Certificate Warning

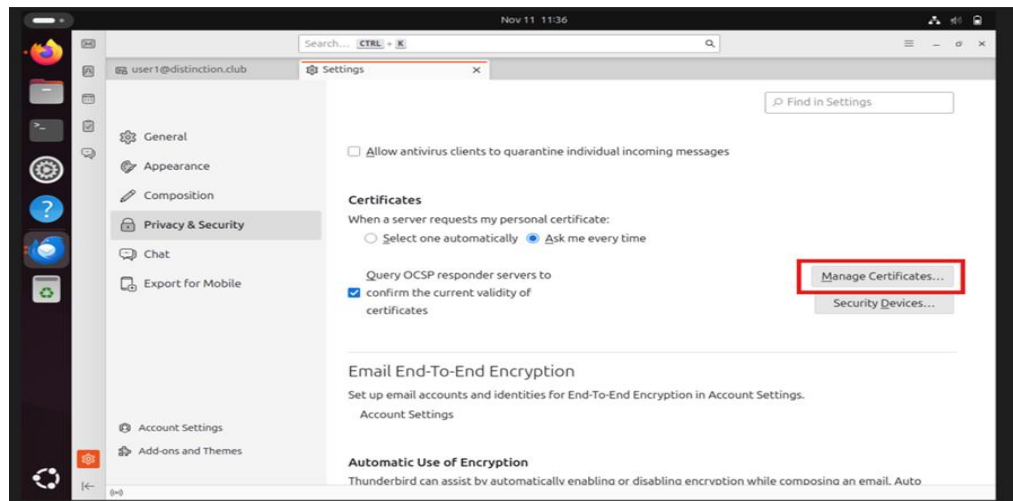
When attempting to send an email, Thunderbird displayed a 'Send Message Error' warning. This occurred because the mail server is using a self-signed certificate, which Thunderbird cannot verify against a known Certificate Authority. As a result, Thunderbird blocked the message for security reasons.

Figure 70 Thunderbird Self-Signed Certificate warning Message



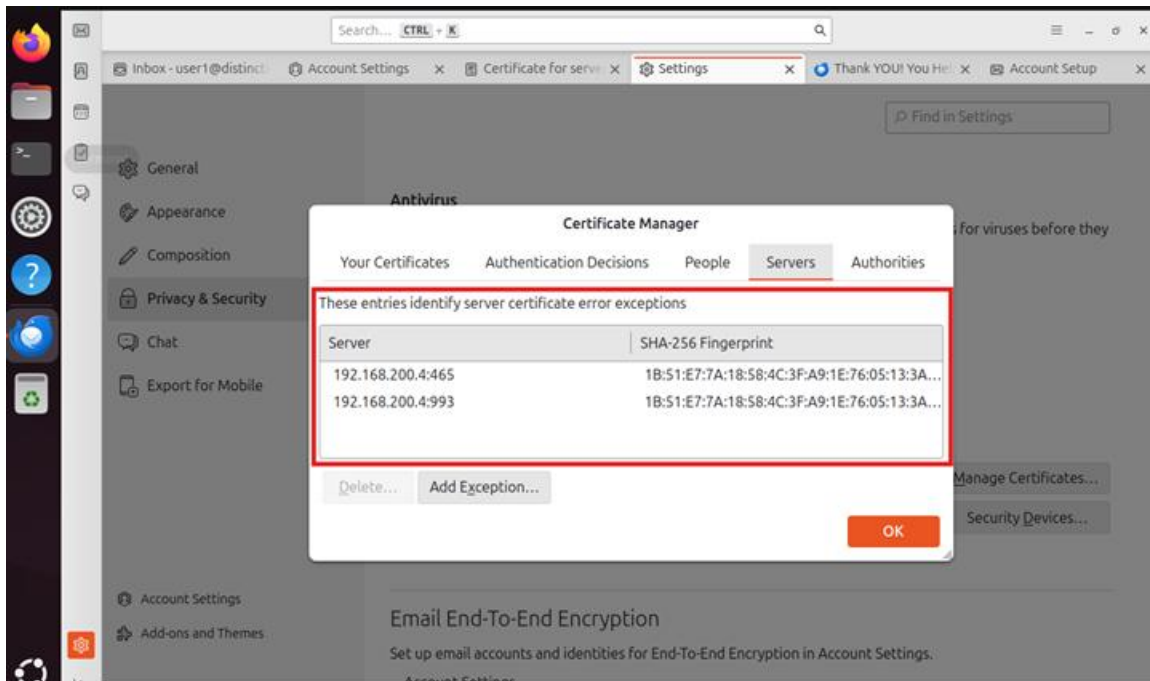
To resolve the warning, the user navigated to Thunderbird's settings. Within the 'Privacy & Security' tab, Thunderbird provides access to certificate management tools. By selecting the 'Manage Certificates' button, the user can add exceptions or import trusted certificates.

Figure 71 Accessing the Certificate Manager from Thunderbird settings.



Inside the Certificate Manager, the user selected the 'Servers' tab. This tab displays server-side certificate exceptions. The mail server's IP and ports (465 and 993) appear here once an exception is added. Clicking 'Add Exception' allows Thunderbird to retrieve the certificate from the server, display its fingerprint, and optionally trust it.

Figure 72 Thunderbird Certificate Manager with server exceptions added



By manually adding a security exception, Thunderbird now trusts the mail server's self-signed certificate. This allows emails to be sent and received securely over SSL/TLS within the lab environment.

Note: This process is common in internal networks where certificates are generated locally rather than signed by a public Certificate Authority.

Verification and Testing

To verify the configuration, Thunderbird (user1@distinction.club) sent a message to user2@distinction.club. The message was successfully received and stored in user2's Maildir.

Figure 73 IMAPS connection verified via OpenSSL client.

```
moha@client:~$ openssl s_client -connect server.distinction.club:993 -quiet
depth=0 C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.dis
verify error:num=18:self-signed certificate
verify return:1
depth=0 C = MY, ST = KualaLumpur, L = APU, O = DistinctionClub, OU = IT, CN = server.dis
verify return:1
* OK [CAPABILITY IMAP4rev1 SASL-IR LOGIN-REFERRALS ID ENABLE IDLE LITERAL+ AUTH=PLAIN AU
dy.

* BAD Error in IMAP command received by server.

* BAD Error in IMAP command received by server.
```

The message headers confirmed encrypted SMTP delivery via Postfix and retrieval via Dovecot using IMAP over SSL.

Figure 74 Maildir listing showing successfully received messages.

```
[user2@server ~]$ pwd
/home/user2
[user2@server ~]$ ls -R ~/Maildir | cat
/home/user2/Maildir:
cur
new
tmp

/home/user2/Maildir/cur:

/home/user2/Maildir/new:
1762833304.Vfd00I116058eM726608.server.distinction.club
1762833410.Vfd00I11605b7M214961.server.distinction.club
1762833879.Vfd00I11605aeM13683.server.distinction.club
1762834111.Vfd00I11605a7M308464.server.distinction.club
1762834493.Vfd00I11605a9M67003.server.distinction.club
1762835329.Vfd00I11605aaM688862.server.distinction.club

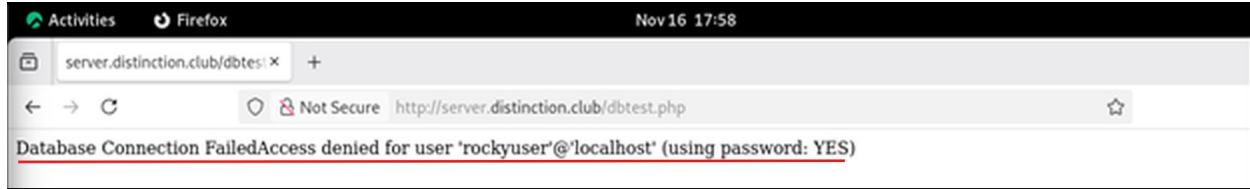
/home/user2/Maildir/tmp:
[user2@server ~]$
```

The message was successfully received and stored in user2's Maildir.

7.4 MariaDB Troubleshooting

Error: Access denied for user 'rockyuser'@'localhost'

Figure 75 Database Connection failed for user 'rockyuser'@'localhost'

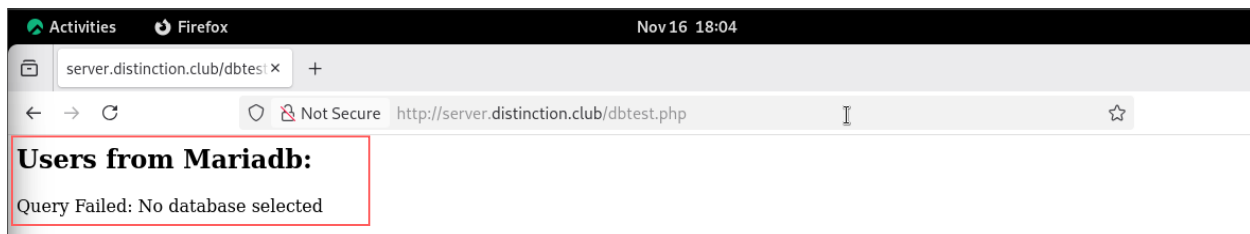


Cause:

- The MariaDB user was created, but permissions were not granted correctly.
- The user did not have access to the database.
- In some cases, the wrong host (localhost instead of %) was used.

Fix: We logged into MariaDB and granted proper permissions.

Error: Query Failed — No Database Selected



Cause:

- The PHP script connected to the server, but no database was explicitly selected.
- The original connection code did not include \$database correctly.

Fix: Adding \$database to the mysqli_connect() function ensured the DB was selected automatically.

CONCLUSION

Assembling all the configured services demonstrated us the way a full Linux based infrastructure works when all the other components are reliant on one another. What began as individual settings such as DNS, DHCP, Email, HTTPS, Samba, and MariaDB; ultimately created a single integrated environment that acted as an actual enterprise network. This experience has made it clear that the following issues are essential in this process; planning, consistency, and the comprehension of the role of each layer of a system in contributing to the overall stability and security.

Another lesson realized in the project was the importance of teamwork. All members were concerned with particular services, though none of the arrangements could exist independently; each component has to be compatible with all the other components of the system. This compelled us to align our decisions, trouble-shooting measures, and ensure that one service did not fail another when it was being configured. This method of work helped us not only to improve our technical knowledge but also to be able to work as a team in a realistic administrative environment.

More to the point, the assignment gave us a better understanding of how professional system administrator tackles the issues by studying the behavior, proving the assumptions, and comprehending the reasoning behind each service instead of making assumptions. The experience eventually enabled us to develop, establish, and operate a working server-client network with confidence.

REFERENCES

- CLOUDFLARE. (n.d.). *What is a mail server?* Retrieved from CLOUDFLARE:
<https://www.cloudflare.com/learning/email-security/what-is-a-mail-server/>
- FORTINET. (n.d.). *What Is Dynamic Host Configuration Protocol (DHCP)?* Retrieved from FORTINET: <https://www.fortinet.com/uk/resources/cyberglossary/dynamic-host-configuration-protocol-dhcp>
- GeeksforGeeks. (2025, July 15). *Introduction of MariaDB*. Retrieved from GeeksforGeeks:
<https://www.geeksforgeeks.org/dbms/introduction-of-mariadb/>
- Girvin, D. (2025, February 18). *What is Apache? In-Depth Overview of Apache Web Server*. Retrieved from <https://www.sumologic.com/blog/apache-web-server-introduction>:
<https://www.sumologic.com/blog/apache-web-server-introduction>
- Mary E. Shacklett, J. B. (2025, July 2025). *What is the domain name system (DNS)?* Retrieved from TechTarget: <https://www.techtarget.com/searchnetworking/definition/domain-name-system>
- Pares, N. (2024, December 10). *Learn to Install Oracle VirtualBox and Run Virtual Machines*. Retrieved from Oracle Linux Blog: <https://blogs.oracle.com/linux/learn-to-install-oracle-virtualbox-and-run-virtual-machines>
- Pratama, D. R. (2023, May 11). *What Is Samba?* Retrieved from Alibaba Cloud:
https://www.alibabacloud.com/blog/what-is-samba_599981
- Rathod, A. (2025, July 12). *What is Dovecot?* Retrieved from LinkedIn:
<https://www.linkedin.com/pulse/what-dovecot-aniruddhsinh-rathod-wio0f/>
- Rathod, A. (2025, July 19). *What is Postfix?* Retrieved from LinkedIn:
<https://www.linkedin.com/pulse/what-postfix-aniruddhsinh-rathod-j5zdf/>

APPENDIX A: GROUP PHOTO

A.1 Group Photo 1



A.2 Group Photo 2



A.3 Group Photo 3



APPENDIX B: WORKLOAD MATRIX

No	TP	Name	Component
1	TP084807	AHMED MIRAHUSAIN ALVI	○ Installing, Configuring & Setting up MariaDB Database ○ Initial Network Configuration ○ Overall Assignment Documentation & Formatting ○ References ○ Workload Matrix
2	TP082459	MOHAMED ABDIFATAH ALI	● Installing & Configuring an Email Server ● Troubleshooting Email server
3	TP085327	SULTAN ABDULLA OMAR TAKRORI	● Installing & Configuring a Web Server ● Configuring Additional Apache Security Features ● Troubleshooting DHCP ● Introduction ● Conclusion
4	TP085031	OMAR KONDAKJI	● Installing & Configuring a DNS Server ● Installing & Configuring Samba File Server
5	TP080822	ABDUL AZIZ VAYANI	● Initial Network Configuration ● Installing, Configuring & Setting up MariaDB Database ● Troubleshooting MariaDB